CPPS2026

TRUE NORTH POWER PLATFORM EXPERIENCE

CANADIAN SUMMIT 2026

Microsoft

Designing Beautiful Apps for Functional Makers

A Masterclass in Fusing Power Apps Capabilities

PLATINUM SPONSOR

mscrm-addons.com
Your company for MS-CRM ADD-ONS!

GOLD SPONSORS

CGI

THE MENTORING HUB

SILVER SPONSORS

MISSION IMPACT PARTNERS

convverge

ecLearn by engineered code

COMMUNITY SPONSORS

DATA PRESS

Endeavor4
Knowledge. Integrity. Trust. Accountability.

upzoids
Microsoft Solutions, Tailored for You

Solulan

CANADIAN SUMMIT 2026

MANY THANKS TO OUR SPONSORS

AGENDA

Introductions / Housekeeping (10 minutes)

Module 1: Overview of Key Topics (20 minutes)
Lab 0: Setup lab environment (15 minutes)

Module 2: Model-Driven Apps (30 minutes)
Lab 1: Create a Model-Driven App (60 minutes)

Module 3: Working with Custom Pages (60 minutes)
Lab 2: Creating and working with custom pages (60 minutes)
Lab 3: Integrating custom pages with model-driven Power Apps (30 minutes)

Module 4: Going Further - Advanced Concepts and Best Practices (30 minutes)
Lab 4: Test and deploy your solution (30 minutes)

#NoFuglyApps!

Head of Power Platform 🚀 **Point Taken**

Business Applications **MVP**

Norway

github.com/cathrinebruvold

linkedin.com/in/bruvold

CATHRINE BRUVOLD

CEO @ proMX UK Limited

Agentologist, trainer, #PowerPlatformFreak, #ProjectOperationsLover, geek, cheesecake consumer

United Kingdom

github.com/JJGriffin

linkedin.com/in/joejgriffin/

JOE GRIFFIN

But it's not all about us...

Who are you and why are you here?

Please tell us about yourself!

https://www.menti.com/alsc95eooh4b

HOUSEKEEPING

Break times:
- 09:30-09:45 (Timmies Run #1 ☕ 🍁)
- 12:00-13:00 (Lunch)
- 14:30-14:45 (Timmies Run #2 ☕ 🍁)

Wi-Fi Details:
- **SSID**: MSFT-GUEST
- **Attendee Code**: msevent183zw

Today's workshop has **four** hands-on lab exercises:
- You should have already received an email with the instructions.
  - Did you complete the form? 😳 --------->
- If completing in your own environments, Lab 0 should have already been completed.
- Don't have an environment? Don't worry; we will provide access and give you time to complete the lab.
- Links on the next slide to access everything – lab instructions, environment credentials and this slide deck 🙌

CONTENT AND LINKS

Lab Instructions
https://bit.ly/BeautifulPowerAppsLabs

Environment Credentials
https://bit.ly/BeautifulPowerApps-Credentials

Presentation
https://bit.ly/BeautifulPowerApps-Presentation

Module 1: Overview of Key Topics

A Powerful Platform

A suite of tools meant to improve and simplify business processes

Copilot Studio
Task-oriented agents and natural language conversation about data.

AI Builder
Build robust, dynamic automations. Invocation of AI models in business processes

Power Apps
Custom applications for solving business needs. UI for exploring organisational data

Power Automate
Automation of workflows and manual processes

Power Pages
Portal and web interface for accessing Dataverse externally

Dataverse
Data storage, security layer and relational database.

The platform is more than just *apps, AI* and *flows*

You need data, security, connectivity, control and logic...

Power Fx

The language in Power Platform. Formulas to execute logic. Canvas apps, Copilot Studio, Dataverse, Power Pages, Power Automate.

Connectors

Access external data sources and services. Custom connectors (security, reusability, custom API).

Managed Environments

Overview, control, usage insights, governance, security features and Power Platform Pipelines

Structure, *security* and *deployment*

How your work is organised, protected and promoted

Solutions

Packaging and deployment containers. Move components between environments (dev, test, prod) with ALM best practices.

Environments

Isolated spaces for building and running apps. Separate dev, test and production to control change and access.

Security Model

Roles, business units and Dataverse table-level permissions. Control who sees and edits what.

People, *process* and *governance*

Who builds, who controls and how the organisation scales

Citizen Developer / Pro Developer

Low-code for business users, code-first extensibility for developers. PCF controls, plug-ins and Azure Functions bridge the gap.

Governance & CoE

Centre of Excellence toolkit, DLP policies and admin controls. Manage adoption at scale without limiting innovation.

Ahh, the app synergies

Build solid solutions using different application types

Canvas Apps

Task focused. Mobile Apps. Drag and drop. Custom UI. Connectors to access external data.

Model-driven apps

Dataverse driven. Structured data in views and forms. Business processes. JavaScript.

Custom Pages

Custom UI and connectors in MDA. Canvas apps capabilities. Modern approach to embedding UI in MDA

Generative pages/Vibe

React. Prompt and natural language. AI-driven. Modern UI in MDA.

Code Apps

Code first approach. Standalone app. Full control. React - TypeScript

Choosing the right tool for your scenario

A quick guide

Need custom quickly → **Generative Pages/Vibe**
Data driven → **Model-driven apps**
Multiple data sources → **Canvas**
MDA functional gaps → **Custom page**
Pro-dev capabilities → **Code Apps**

Choosing the right app type

When to use each — and when to combine them

Model-driven apps

- Dataverse is your primary data source
- Standardised forms, views and dashboards
- Complex data model with relationships
- Security roles control access per table

Canvas apps

- Highly customised UI and pixel-perfect layout
- Multiple or non-Dataverse data sources
- Mobile-first or task-specific experiences
- Simple data models, focused workflows

+

Custom pages

- Canvas-style UI inside a model-driven app
- Replace dialogs, wizards or complex forms
- Combine Dataverse context with custom layout
- Best of both worlds — embedded flexibility

Code-first apps

- Requirements exceed low-code capabilities
- Full developer control (React, PCF, Azure)
- Complex integrations or real-time processing
- Dedicated dev team for build and maintain

Combine them

MDA + Custom Pages for flexible forms · Canvas embedded in MDA for task UIs · PCF controls inside any app

Generative pages

Describe a page → AI writes the code → you refine it

What are they?

AI-generated pages inside **model-driven apps**. Describe what you want in plain language, attach a sketch or wireframe, select your Dataverse tables — and an AI agent generates **real React & TypeScript code**.

Why it matters

- Concept to functional page in **minutes, not weeks**
- No drag-and-drop limitations — full creative freedom
- Powered by GPT-5 for high-quality code generation
- Full code transparency — view, edit & extend the React code
- Solution-aware — managed ALM across environments
- Now **generally available** (since November 2025)

✓ **Generally available**

Available in model-driven apps · Dataverse tables only · Up to 6 tables per page

How it works

1 **Describe**
Write a natural language prompt

↓

2 **Connect**
Select Dataverse tables + attach sketch

↓

3 **Generate**
AI writes React & TypeScript code

↓

4 **Refine**
Iterate via conversation or edit code

Think of it as a **reimagined Custom Page** — no Power Fx needed, no drag-and-drop limits, just describe and go

… And know when to avoid the app type

A quick guide

Generative Pages → Complex workflows and Multiple data sources, **not just Dataverse**

Model-driven apps → Dataverse is not the data source and a need for highly **customised UI**

Canvas Apps → There are heavy **data operations** and complex data model

Custom page → Need for a **standalone app** and there are heavy data operations

Code Apps → No developer capabilities, which makes a **low code** project too complex

Power Apps is not dead… yet

What's new and exciting heading into 2026

01 Vibe Coding
Build apps by describing them in plain language at vibe.powerapps.com — AI generates React code, data models and UI automatically

02 Code Apps (GA)
Deploy React, Vue or any web framework directly to Power Platform with full ALM, governance and enterprise guardrails

03 MCP Server & Agent Feed
AI agents fill forms, create records and hand off to humans for review — with a shared supervision workspace

04 M365 Copilot in Apps
Copilot chat embedded in model-driven apps — ask questions, summarise data and take action without leaving the app

05 Plan Designer
Describe a business problem and a team of AI agents generates requirements, data model, app and Power BI reports

06 Modern Controls & Cards
New responsive Card control, Fluent UI dialogs, theme copy-paste across apps and refreshed MDA navigation

The platform is evolving from **low-code** to **AI-first** — where humans and agents build, run and manage apps together

What are we building today

Fusing Power Apps capabilities and creating a beautiful app.

Scenario

Coho Winery needs to access and work with purchase orders from Power Platform. They have data stored in an ERP database and they have relevant documents in OneDrive.

The task

Develop an end-to-end solution that integrates data from Coho Winerys ERP database into a model-driven app to make purchase order information more accessible for the users.

You will:

- Utilise a custom API for connecting to ERP data
- Create a model-driven app that has key information about purchase orders
- Develop a custom UI in two different custom pages to serve as a **landing page** and as a **side pane** to display purchase order PDFs and embed them in the MDA.

Todays Build

Extending model-driven apps with pixel-perfect custom UI

What is a Custom Page?

A canvas-based page that runs inside a model-driven app. It uses Power Fx and the canvas designer, giving you full control over layout, styling and behaviour — all within your MDA.

Why use them?

Pixel-perfect UI
Full canvas designer control over layout, colours and components

Embedded experiences
Display as full page, side pane or dialog — all within the MDA

Power Fx + connectors
Access 1,000+ connectors and custom APIs alongside Dataverse

Solution-aware
Managed ALM — deploy across environments with your MDA solution

In our workshop

Coho Winery — Purchase Order app

Model-driven app

Custom Page — **Landing Page**
Dashboard-style overview with KPIs, charts and navigation tiles

Custom Page — **Side Pane**
Displays purchase order PDFs alongside the form — no new tabs

Model-driven forms
Tables, views, forms, dashboards — Dataverse-driven

Dataverse · Custom API · OneDrive · ERP connector

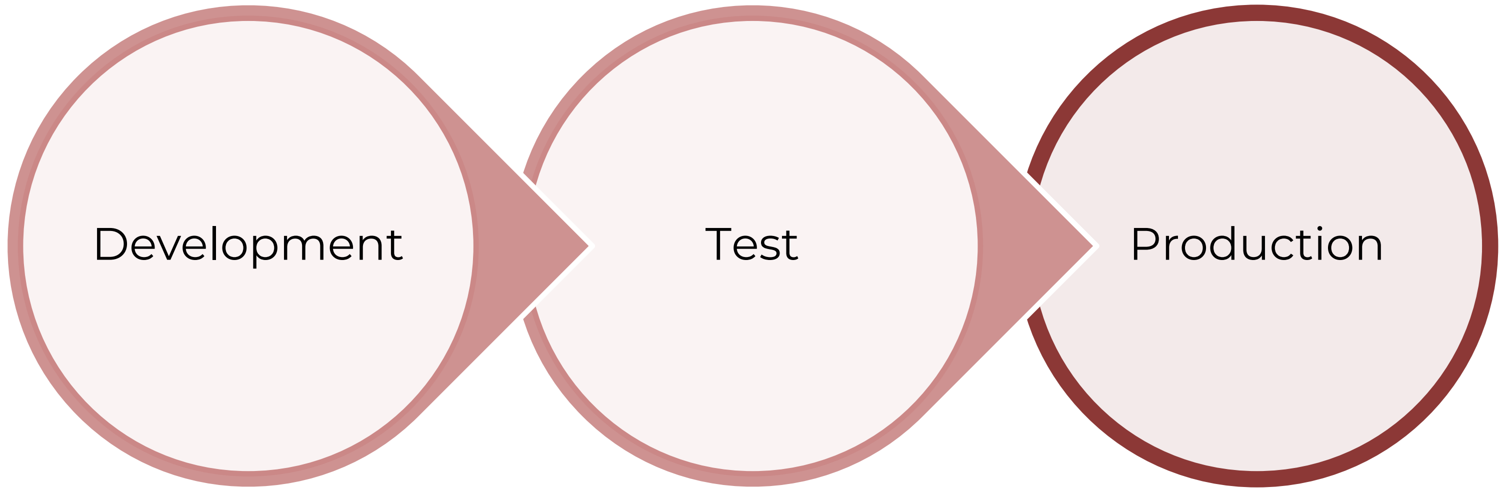
Custom pages bridge the gap between canvas flexibility and model-driven structure — one app, best of both worlds

DEMO: What we will build today

First, we need a solid foundation

The importance of Solutions and Application Lifecycle Management (ALM)

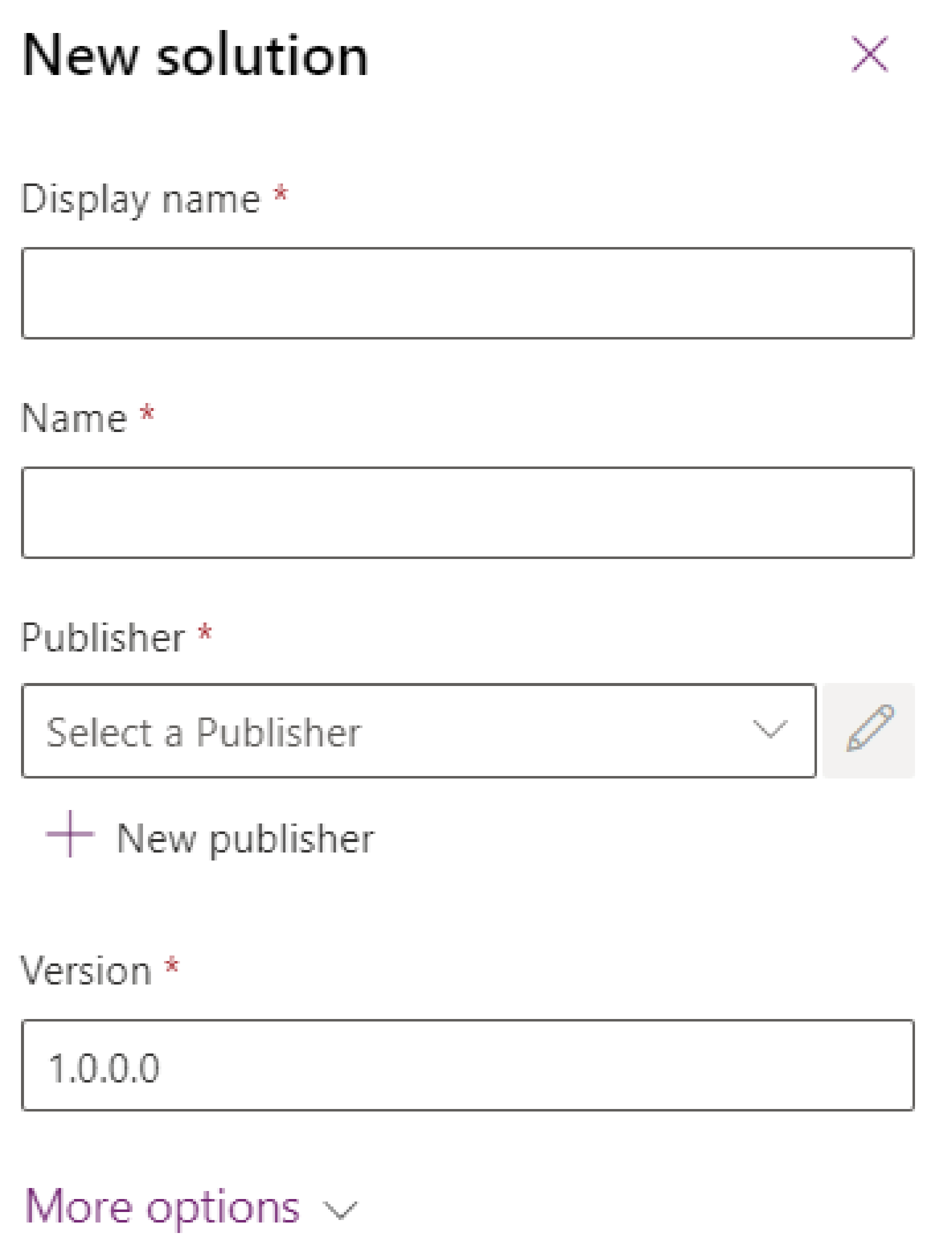
- Solutions are containers for Power Platform components 📦
- Transport customizations from one environment to another
- Part of an overall strategy for application lifecycle management (ALM)
- Unmanaged versus managed solutions



So easy, so no excuse

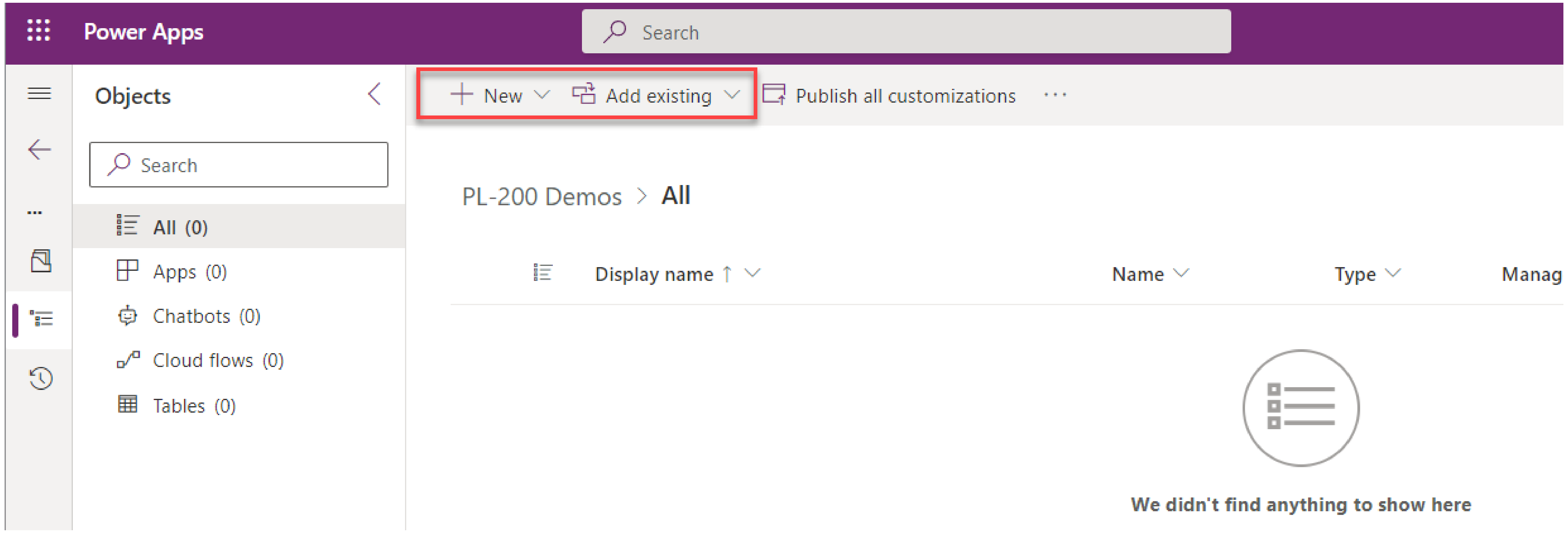
Creating a solution

- **Display name** used in Maker portal
- **Name** is the schema name and cannot be changed and cannot have spaces
- **Publisher** defines the prefix for new components
- **Version** number to track updates



What's inside there? 🔍

Solution components



- Create new components in solutions
- Add existing components to solutions
  - Avoid **Include all objects** for Common Data Model tables (Account, Contact etc.)

Caring is sharing 🤝

Tips and tricks when working with solutions

- Solution aware versus non-solution aware components
- Always remember to KISS 😽
  - Less is more
  - Dependencies and deployment order
  - Multiple managed layers
- Handle cloud flows and connection references via a dedicated solution and service account
  - Prevents flows from switching off post-deployment
- Consider automation for more complex scenarios:
  - Power Platform Pipelines
  - Power Platform (PAC) CLI
  - Azure DevOps

Lab 0 – CONFIGURE LAB ENVIRONMENT

In this lab, you will do the following:

- Redeem your Developer plan license for Power Apps, to create an environment to build your solution - this will be your Development environment.
- Set up an environment to deploy your solution - this will be your Production environment.
- Create a publisher and solution to store your components.
- Prepare your OneDrive for Business folder and upload the required document template.

Expected completion time: 15 minutes

Module 2: Model-Driven Apps

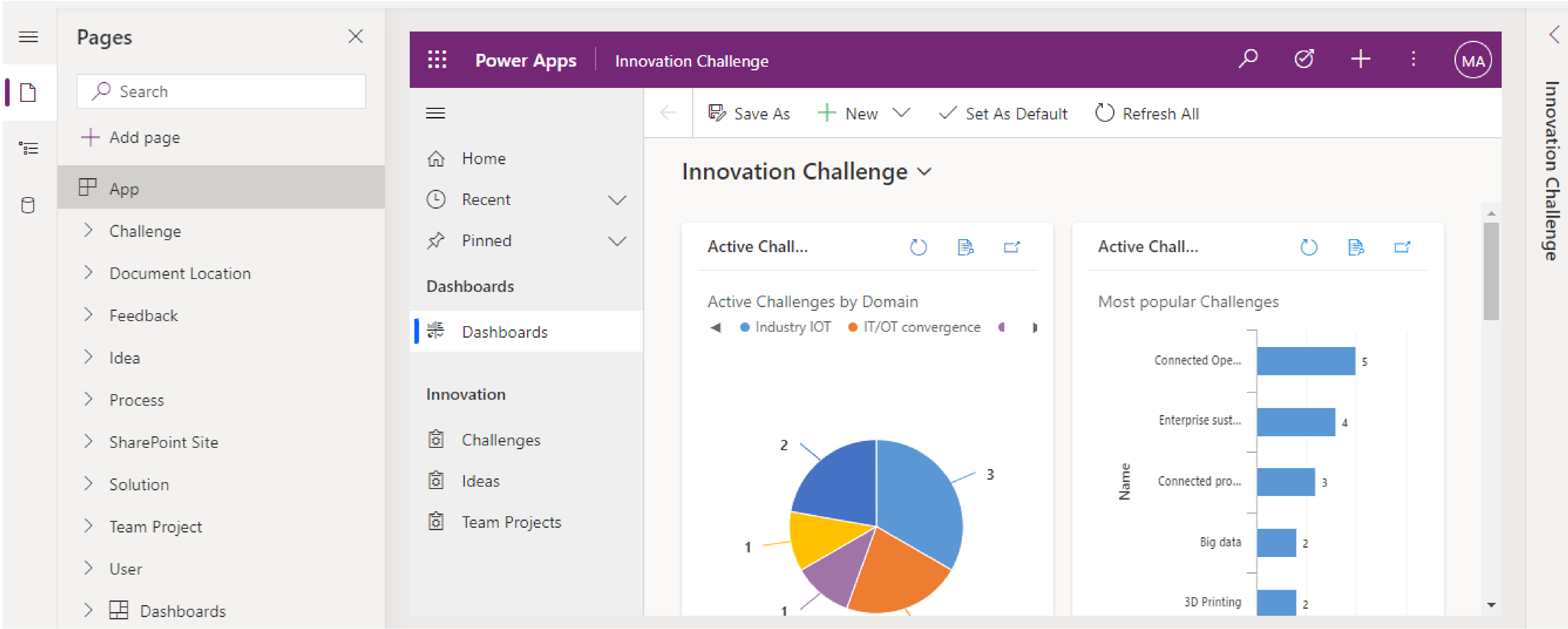
Model-Driven App Development

Before creating your app for the first time, consider the following:

- Working with the modern designer (and why the "classic" designer might be necessary)
- How the sitemap works
- Key app components:
  - **Tables**
  - **Forms**
  - **Views**
  - Charts
  - Dashboards

We like it modern

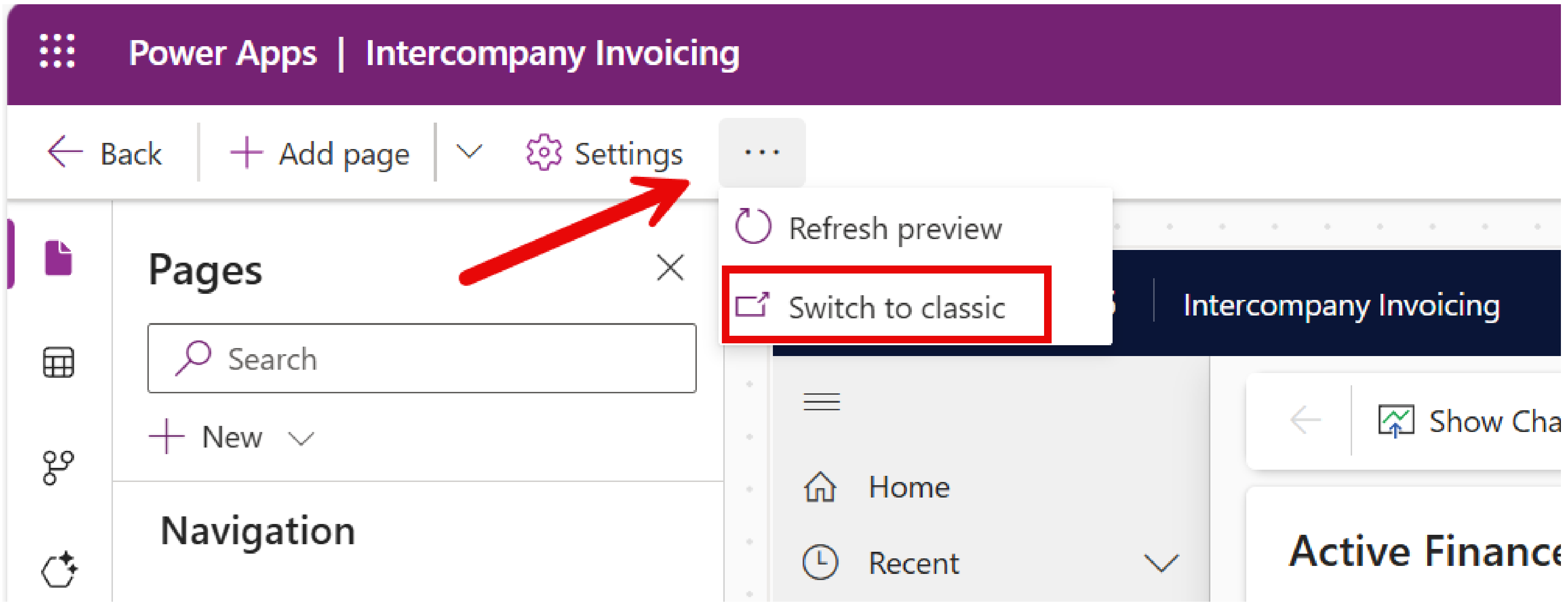
Modern app designer



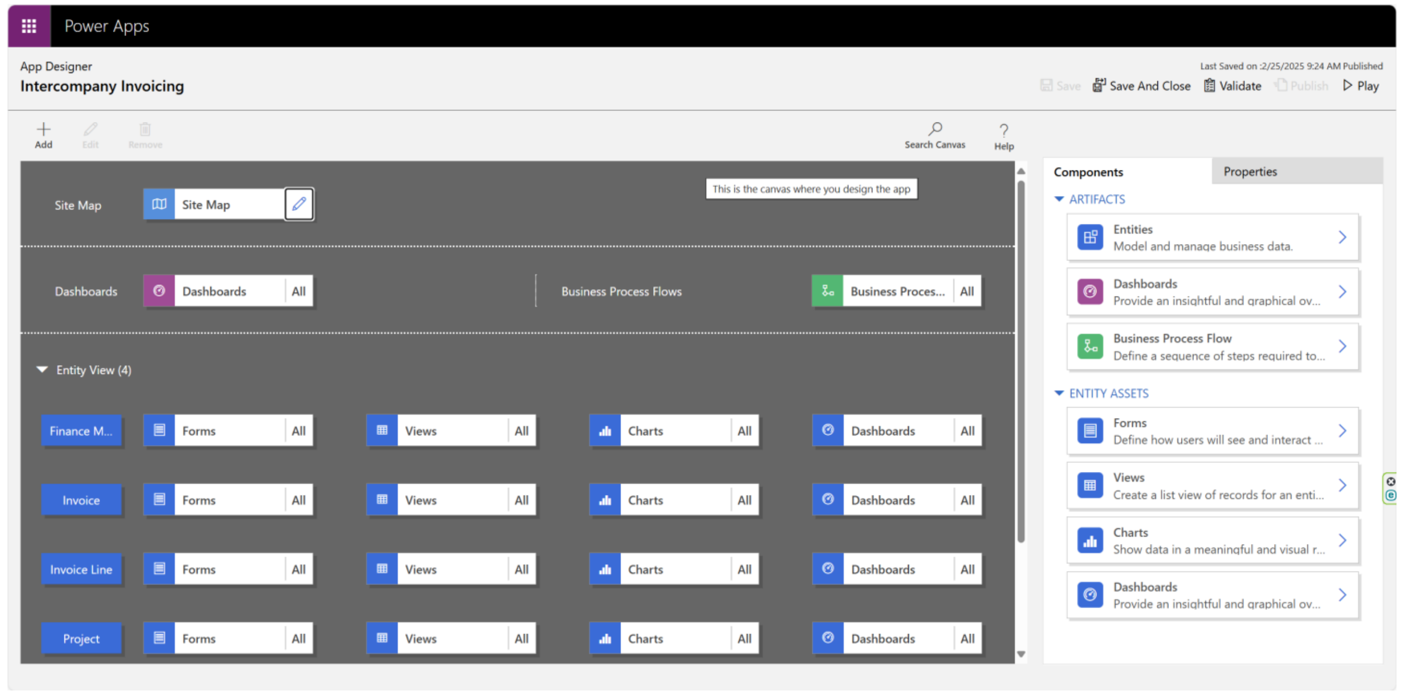
After modelling data, you build your app by selecting and setting up the pages you need in the App Designer.

Classics never die

“Classic” designer

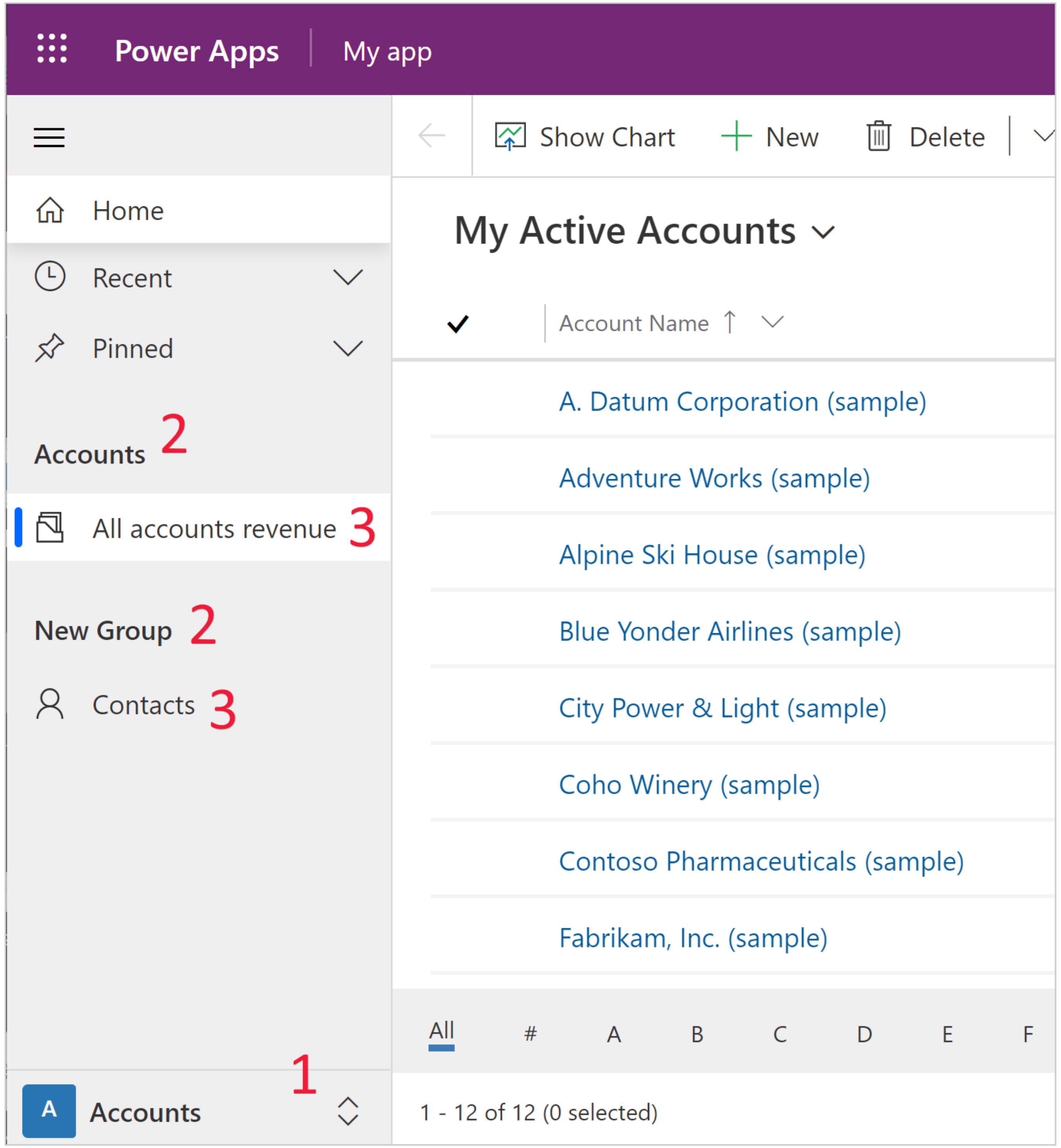


- The old way of building apps 👴
- Still available for backwards compatibility; will eventually be removed.
- Can’t do something in the modern designer? Go “classic”!
- Only use if strictly required.



Need some directions

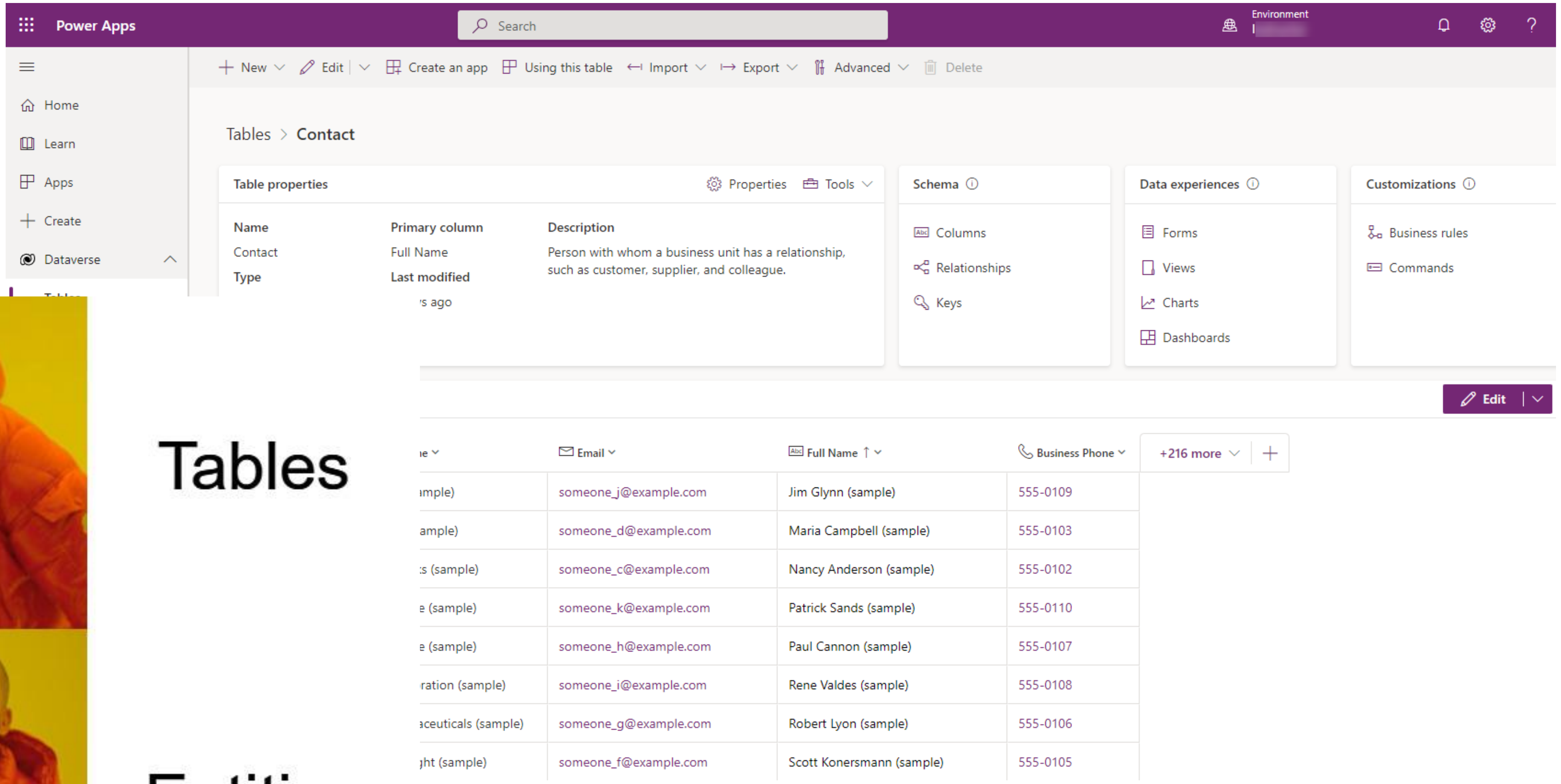
Sitemap



- Every model-driven app has a Sitemap that controls the left-hand navigation
- Structure of a site map:
  1. Area
  2. Group
  3. Subarea
- Pages are added as subareas

Relational data FTW

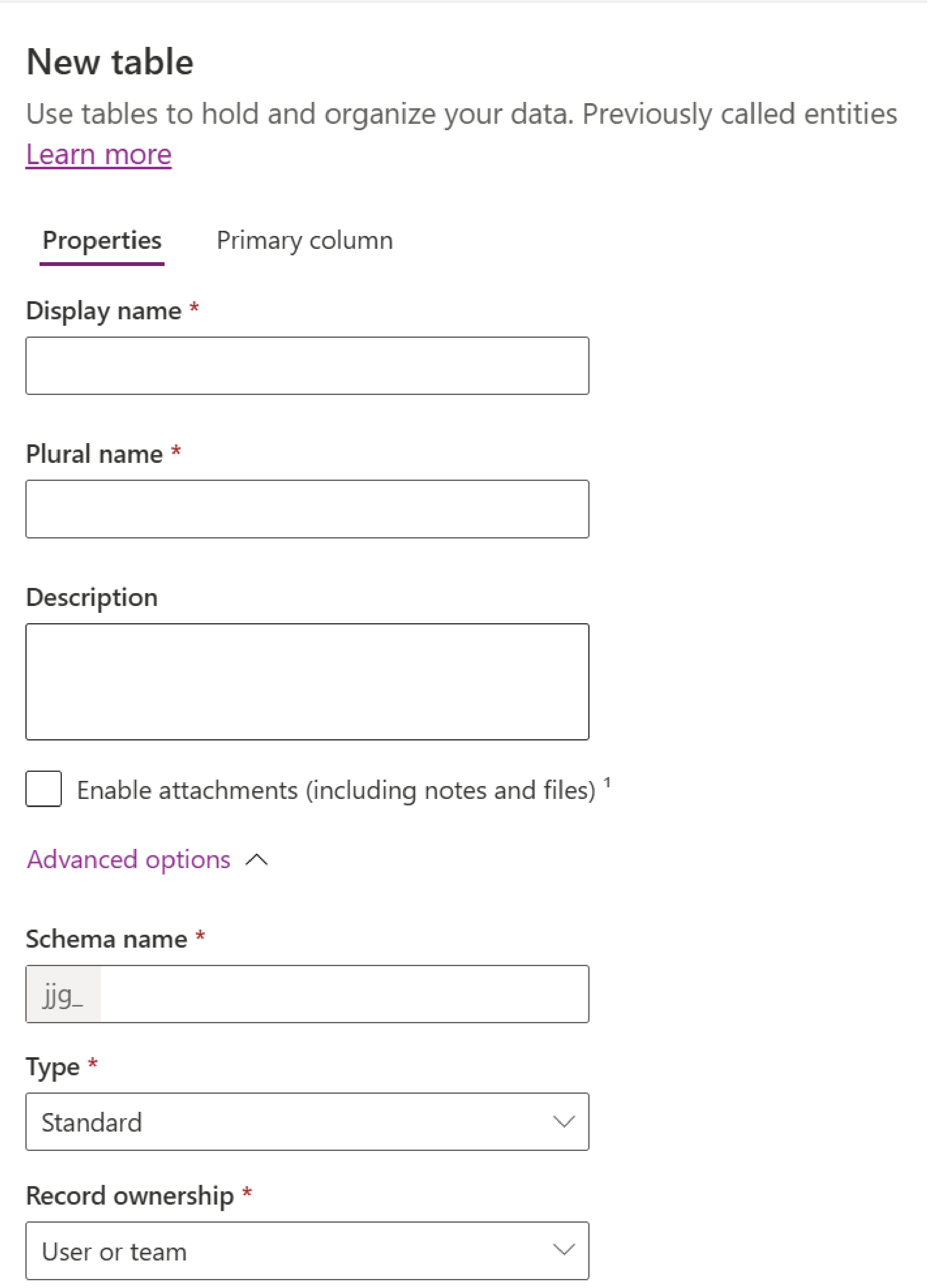
A table is a logical structure containing rows and columns that represents a set of data.



Bling up your data model

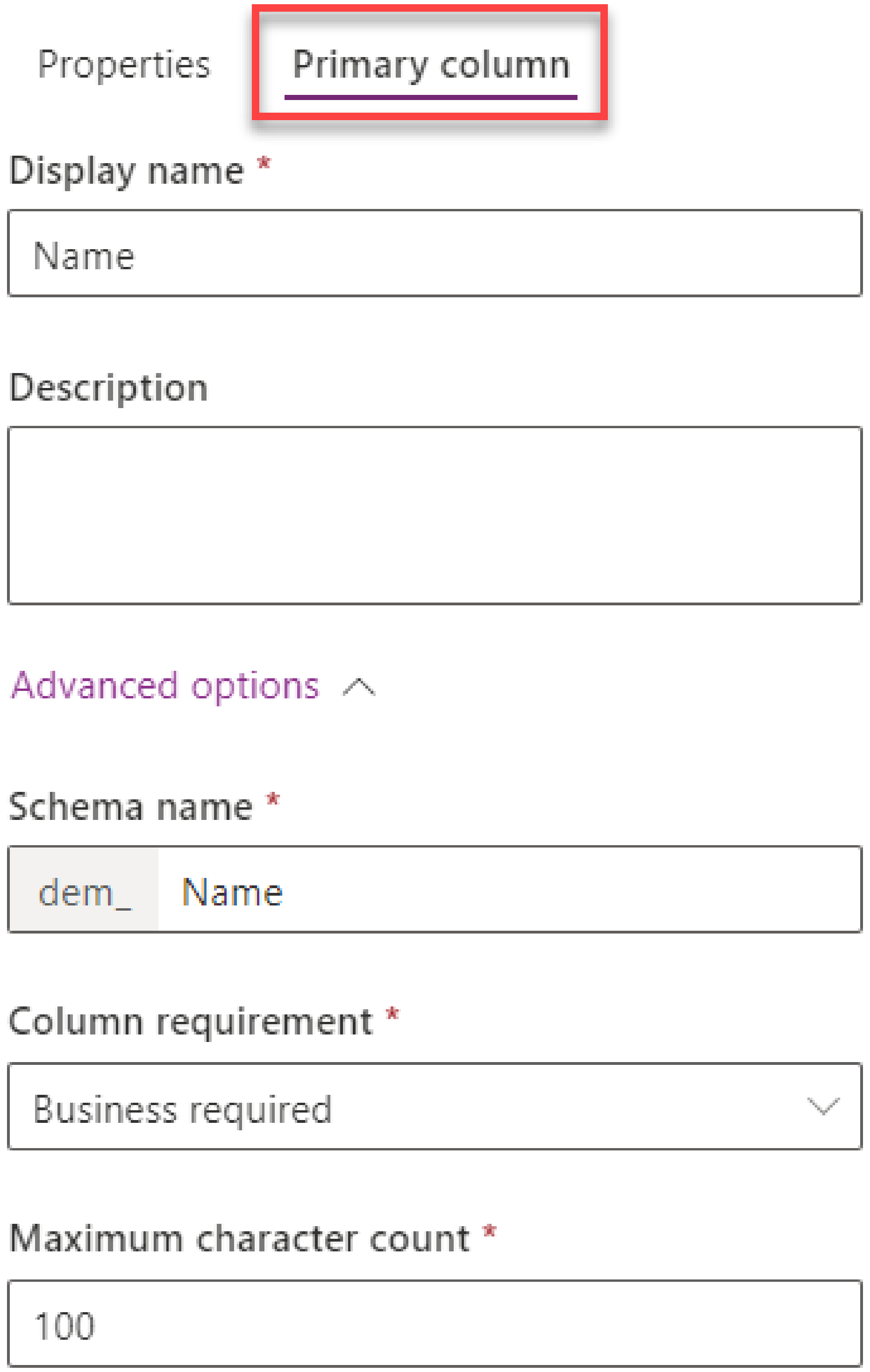
Table characteristics

- Microsoft Dataverse is designed to let you quickly and easily create a data model for your application, based on the tables and the table metadata that you include in your app.
  - Display name
  - Plural name
  - Schema name
  - Primary column
  - Table type
  - Record ownership
  - Advanced options



What’s in a name?

Primary column



- Every table has a Primary Name column
- Single line of text
- Used by lookup columns to display the name of the related row
- Displayed in form heading and window title in model-driven apps

This is getting out of hand. Now there are four of them

Types of table

Standard
Create and edit rows

Activity
Used to track interactions on standard tables

Virtual
Connect to tables in an external data source

Elastic
Handle large volumes of data

DEMO: Creating a custom table

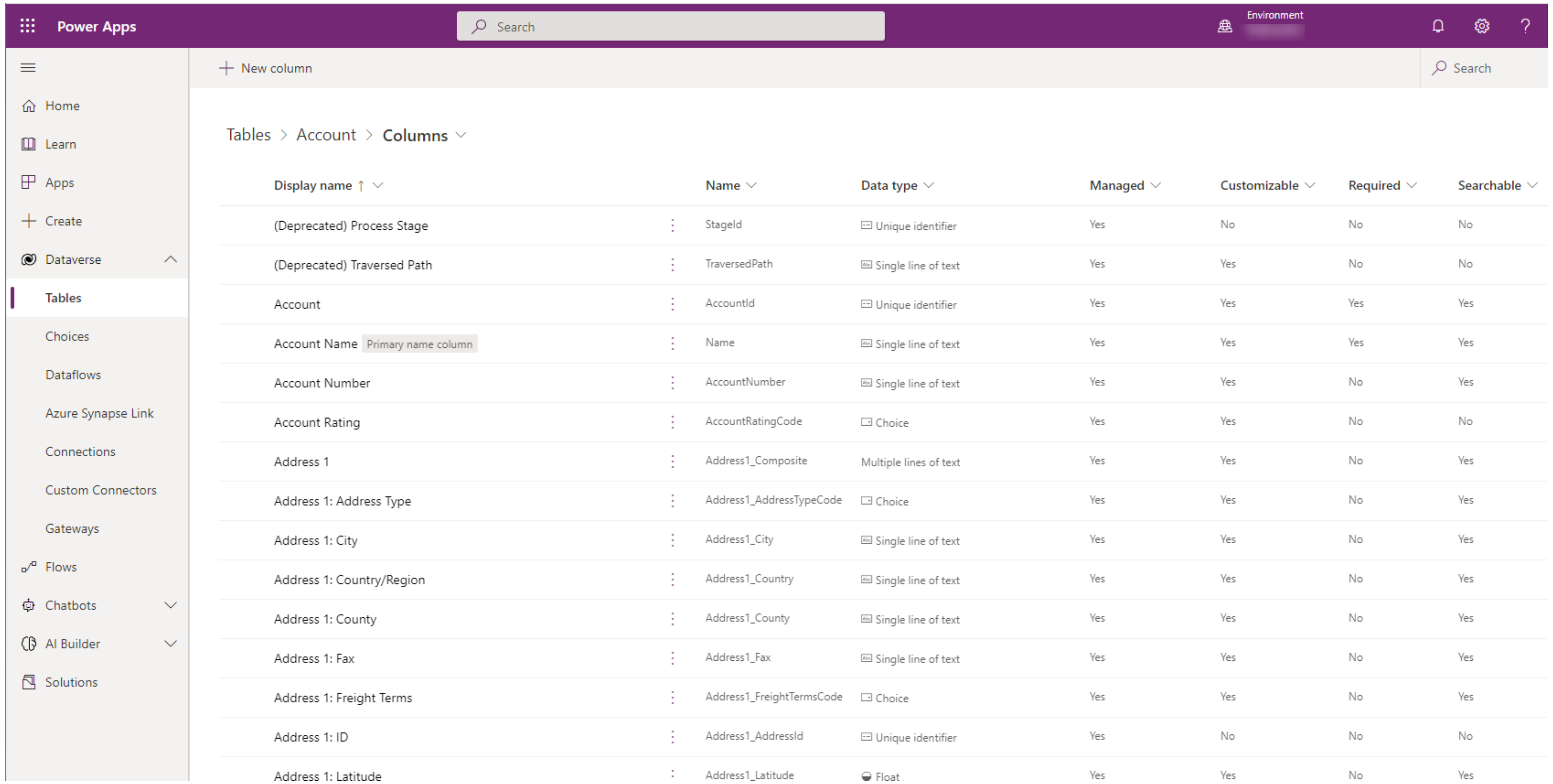
Decisions, decisions...

Standard versus custom tables

- **Dataverse comes with several of standard tables that support core business application capabilities.**
  - To change the display name, you can edit the table. You don’t have to create a new table.
  - You can’t delete standard tables, but you can hide them. To hide a standard table, change the security role privileges for your users to remove the Read privilege for that table. This will remove the table from most parts of the application.

A columnar cornucopia

Columns are a way to store a discrete piece of information within a row in a table.



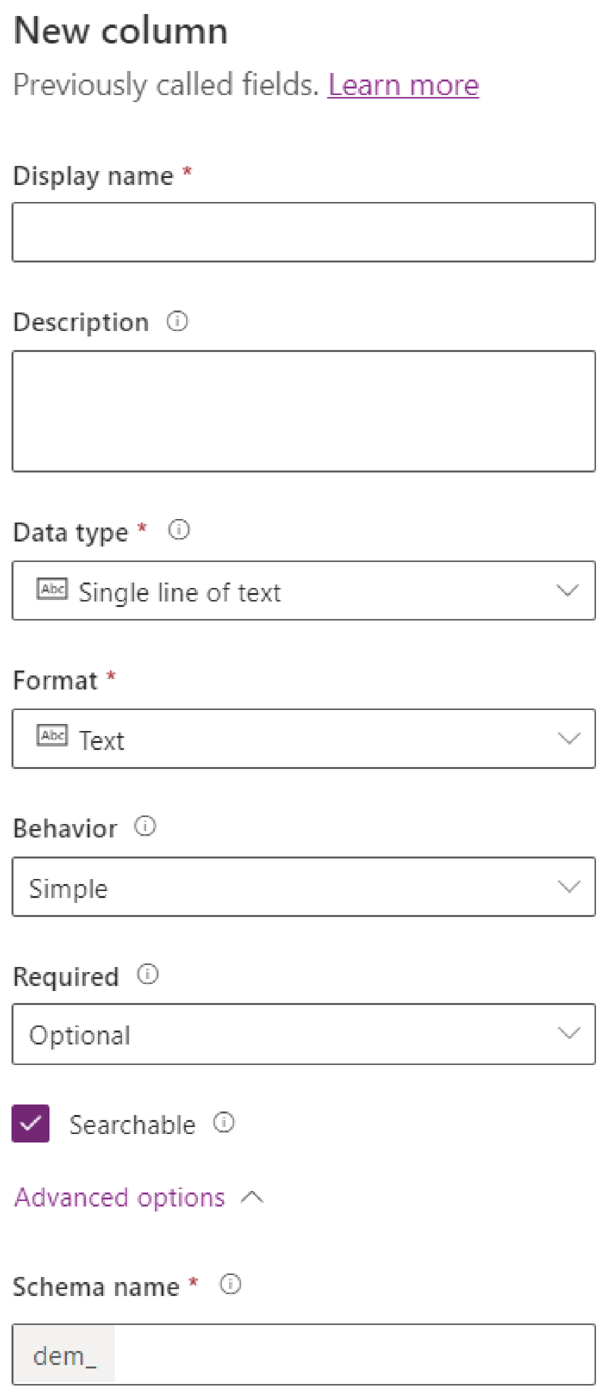
Display name ↑	Name	Data type	Managed	Customizable	Required	Searchable
(Deprecated) Process Stage	StageId	Unique identifier	Yes	No	No	No
(Deprecated) Traversed Path	TraversedPath	Single line of text	Yes	Yes	No	No
Account	AccountId	Unique identifier	Yes	Yes	Yes	Yes
Account Name Primary name column	Name	Single line of text	Yes	Yes	Yes	Yes
Account Number	AccountNumber	Single line of text	Yes	Yes	No	Yes
Account Rating	AccountRatingCode	Choice	Yes	Yes	No	No
Address 1	Address1_Composite	Multiple lines of text	Yes	Yes	No	Yes
Address 1: Address Type	Address1_AddressTypeCode	Choice	Yes	Yes	No	Yes
Address 1: City	Address1_City	Single line of text	Yes	Yes	No	Yes
Address 1: Country/Region	Address1_Country	Single line of text	Yes	Yes	No	Yes
Address 1: County	Address1_County	Single line of text	Yes	Yes	No	Yes
Address 1: Fax	Address1_Fax	Single line of text	Yes	Yes	No	Yes
Address 1: Freight Terms	Address1_FreightTermsCode	Choice	Yes	Yes	No	Yes
Address 1: ID	Address1_AddressId	Unique identifier	Yes	No	No	No
Address 1: Latitude	Address1_Latitude	Float	Yes	Yes	No	Yes

1,024 column table, here I come...

Add a column to a table

You can add columns when you create a new custom table, or you can add columns to an existing table at any time.

- Display name
- Descriptions
- Data type
- Format
- Behavior
- Required
- Searchable
- Schema name
- Advanced options



Gotta store 'em all

Column data types

- Single line of text
- Multiple lines of text
- Whole number
- Floating point number
- Decimal number
- *Currency*
- Date and time
- Lookup
- Customer
- Choice
- Choices
- Yes/No
- File
- Image

You mean there's more??

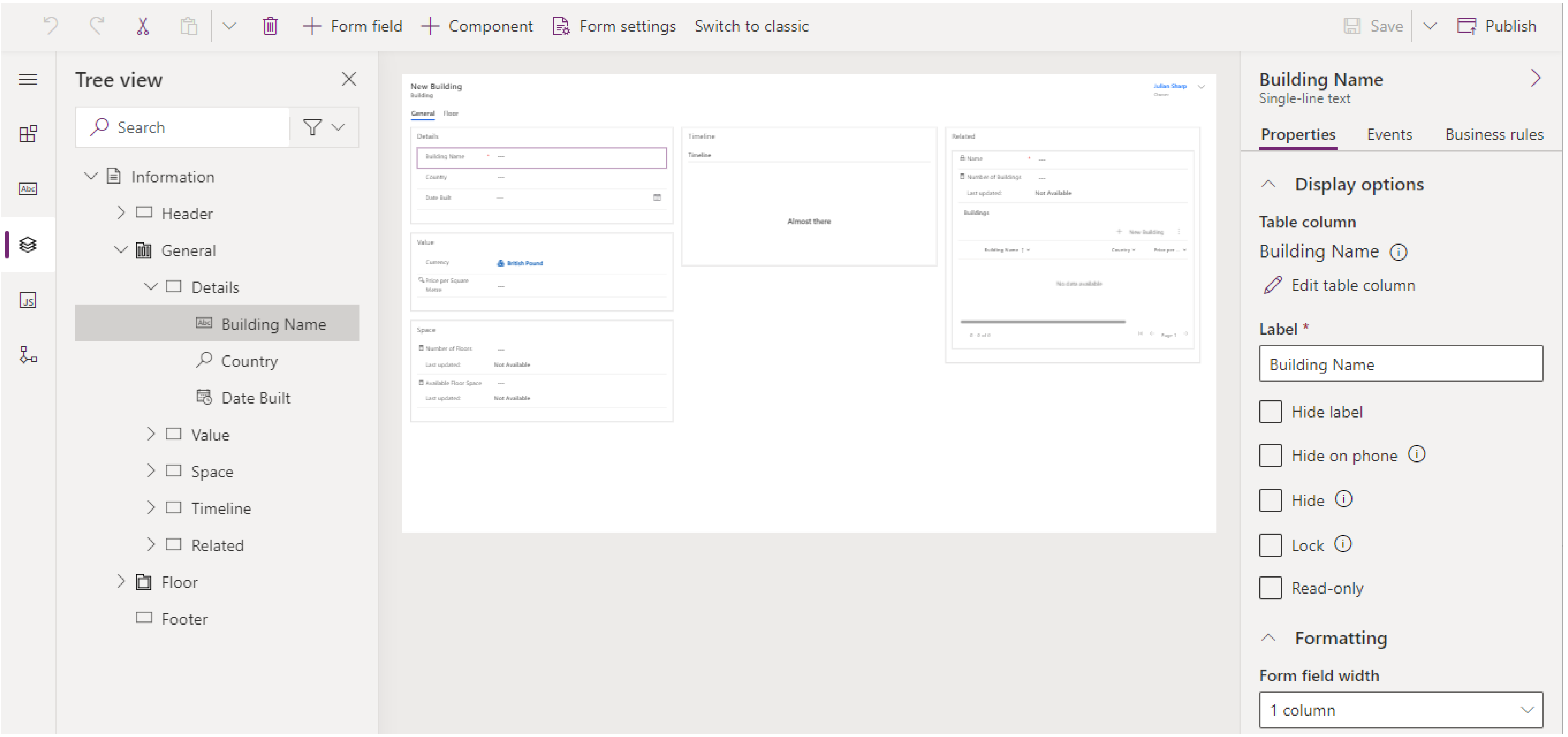
Other column details

- Column display formats
- Column properties
- Column requirement
- Searchable
- Column (field) security
- Auditing
- Description

DEMO: Working with custom columns

Perfectly formed

The Form Designer



Model-driven forms are how data for a row is presented to the user during editing and viewing.

What's your type?

Form types

Main

Quick View

Quick Create

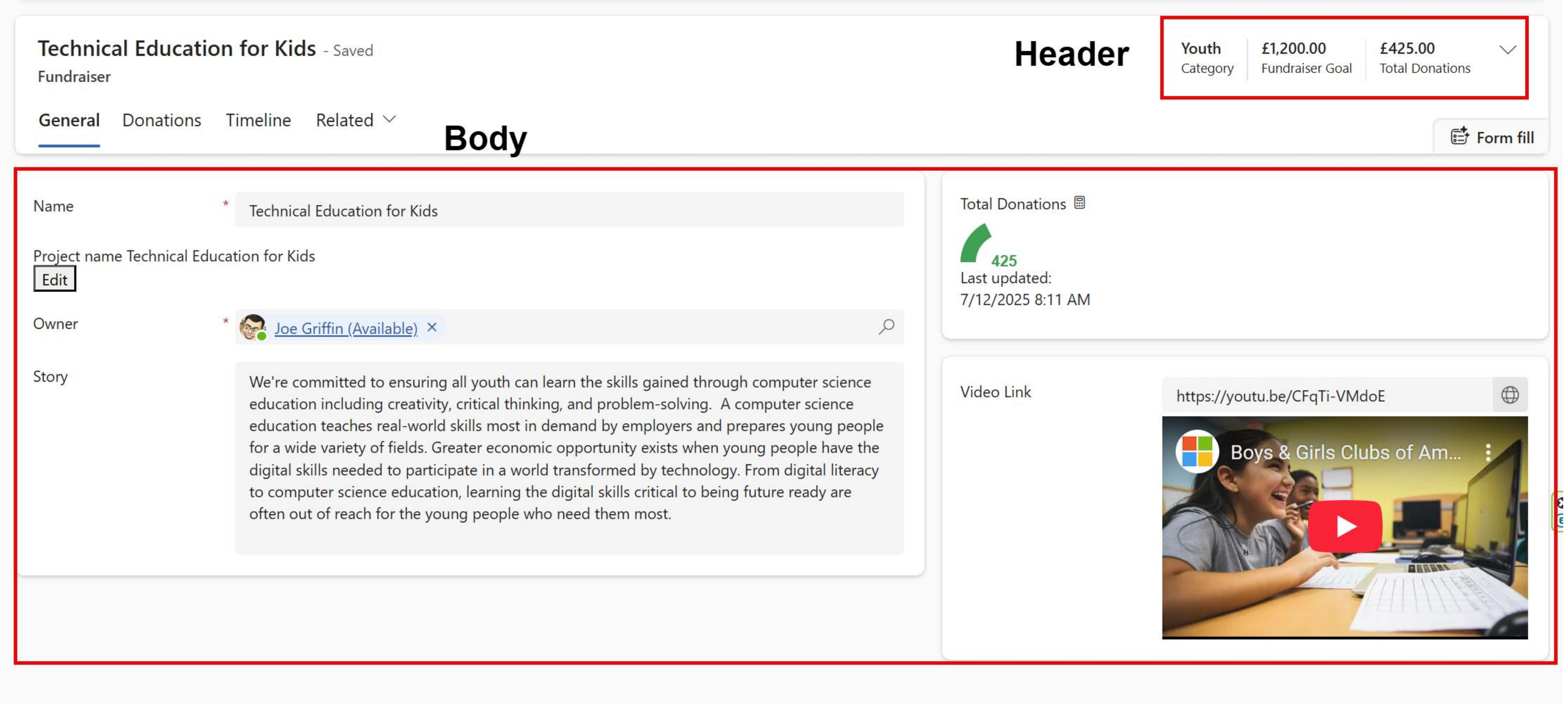
Card

You just need structure

Form structure

Model-driven app forms have a defined structure

1. **Header** – contains up to 4 columns.
2. **Body** – contains main content of the form.
3. **Footer** – no longer used.

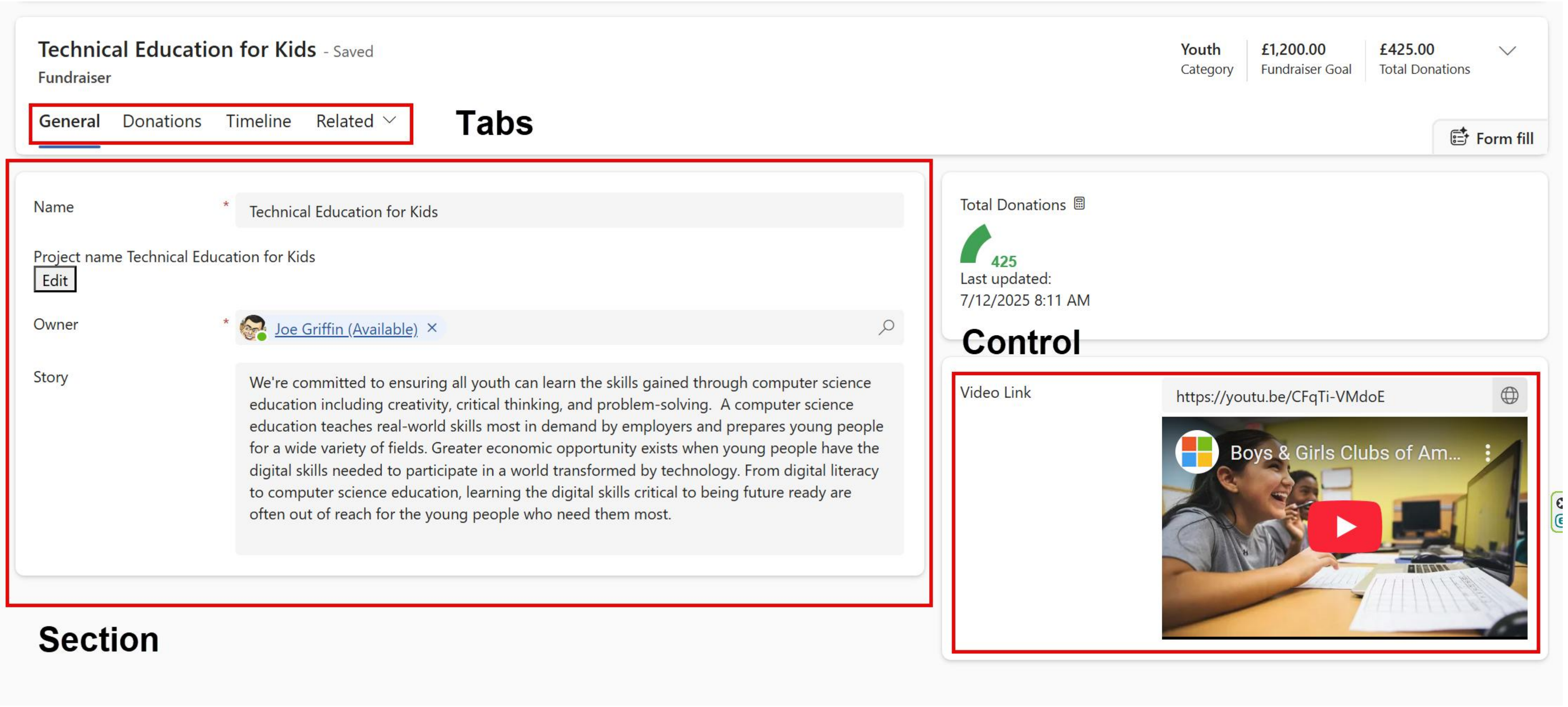


You just need structure

Form structure

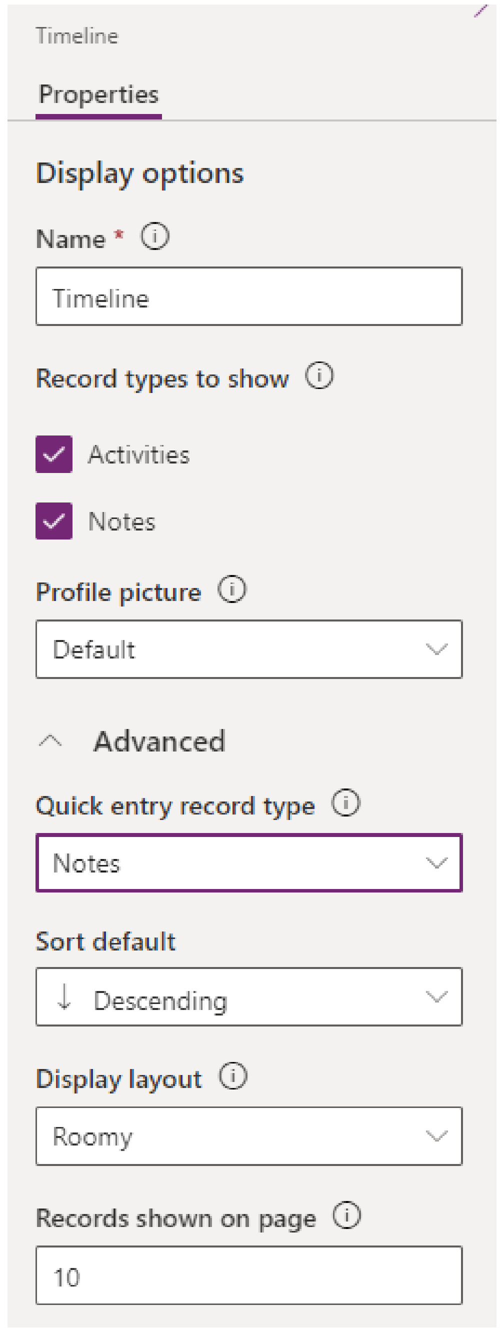
The body in a model-driven app form has a defined structure:

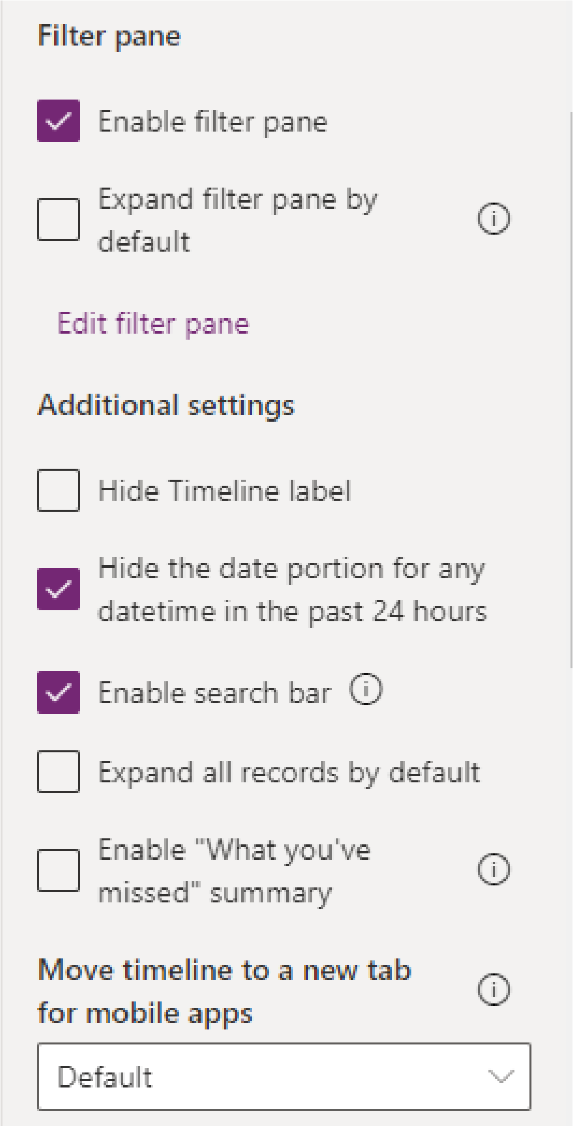
1. **Tabs** – tabs can be 1, 2, or 3 column layout and contain sections.
2. **Sections** – sections should be 1 column and contain controls.
3. **Controls** – Controls are table columns or other specialized components.

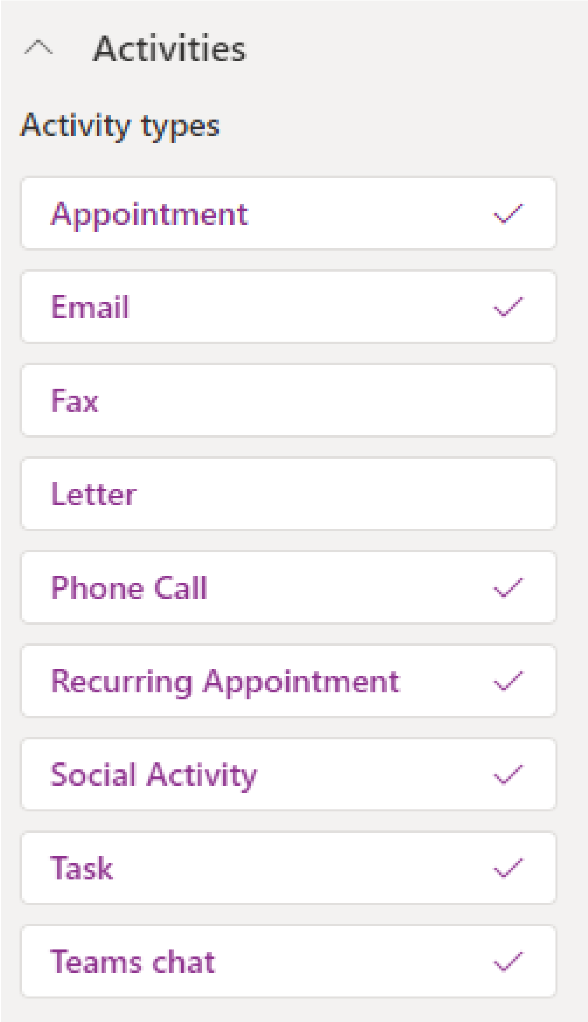


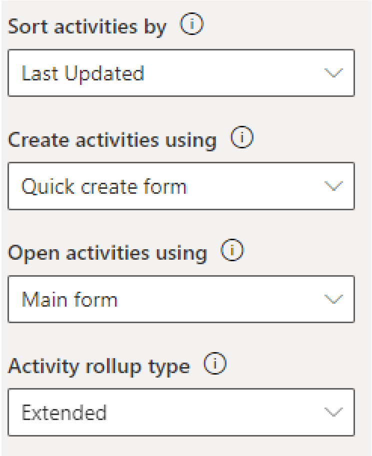
I’m special. So special.

Specialized form components









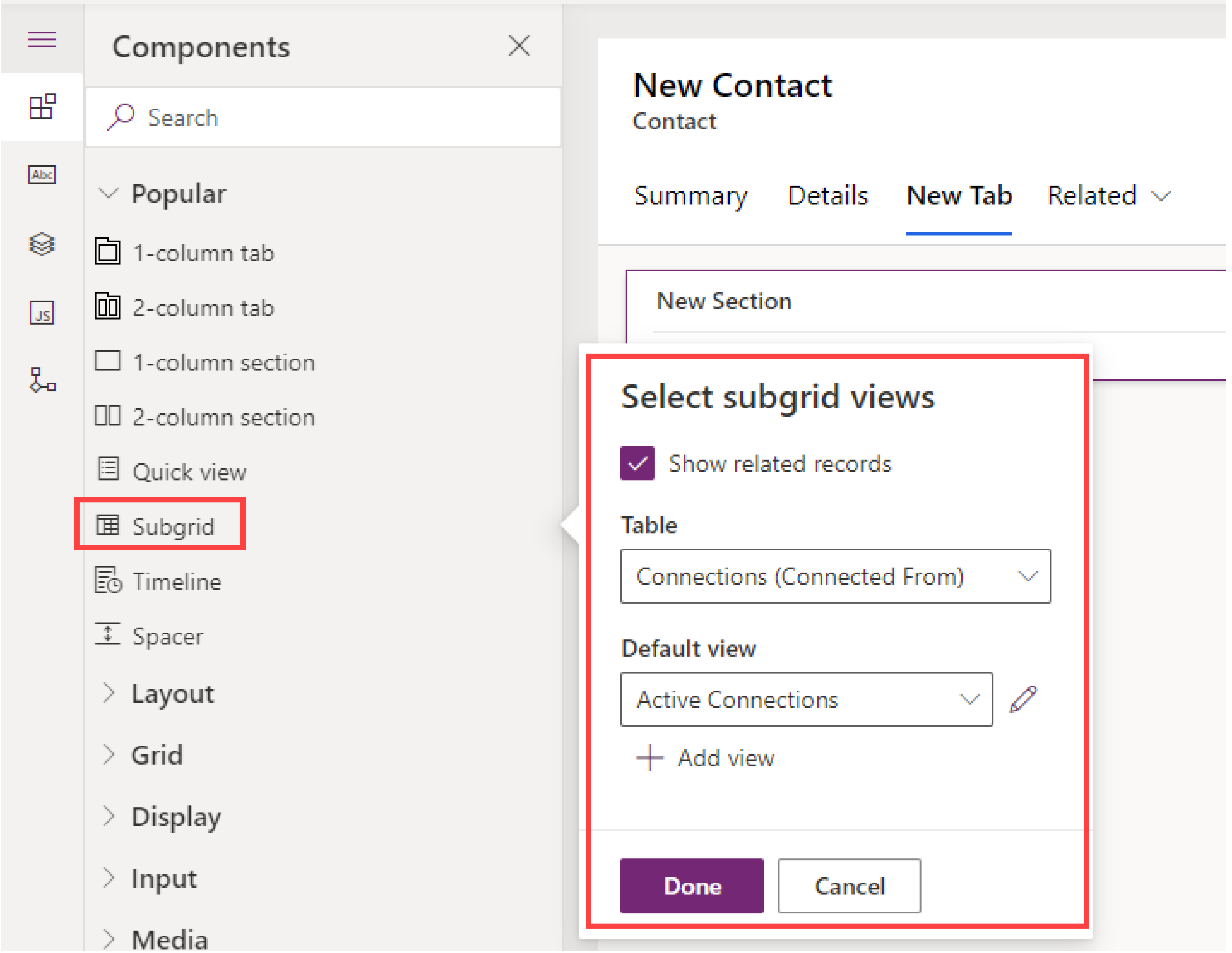
- Timeline
- Subgrid
- Quick view
- Reference panel
- Timer control
- IFrame

We are family

Show related data in a form

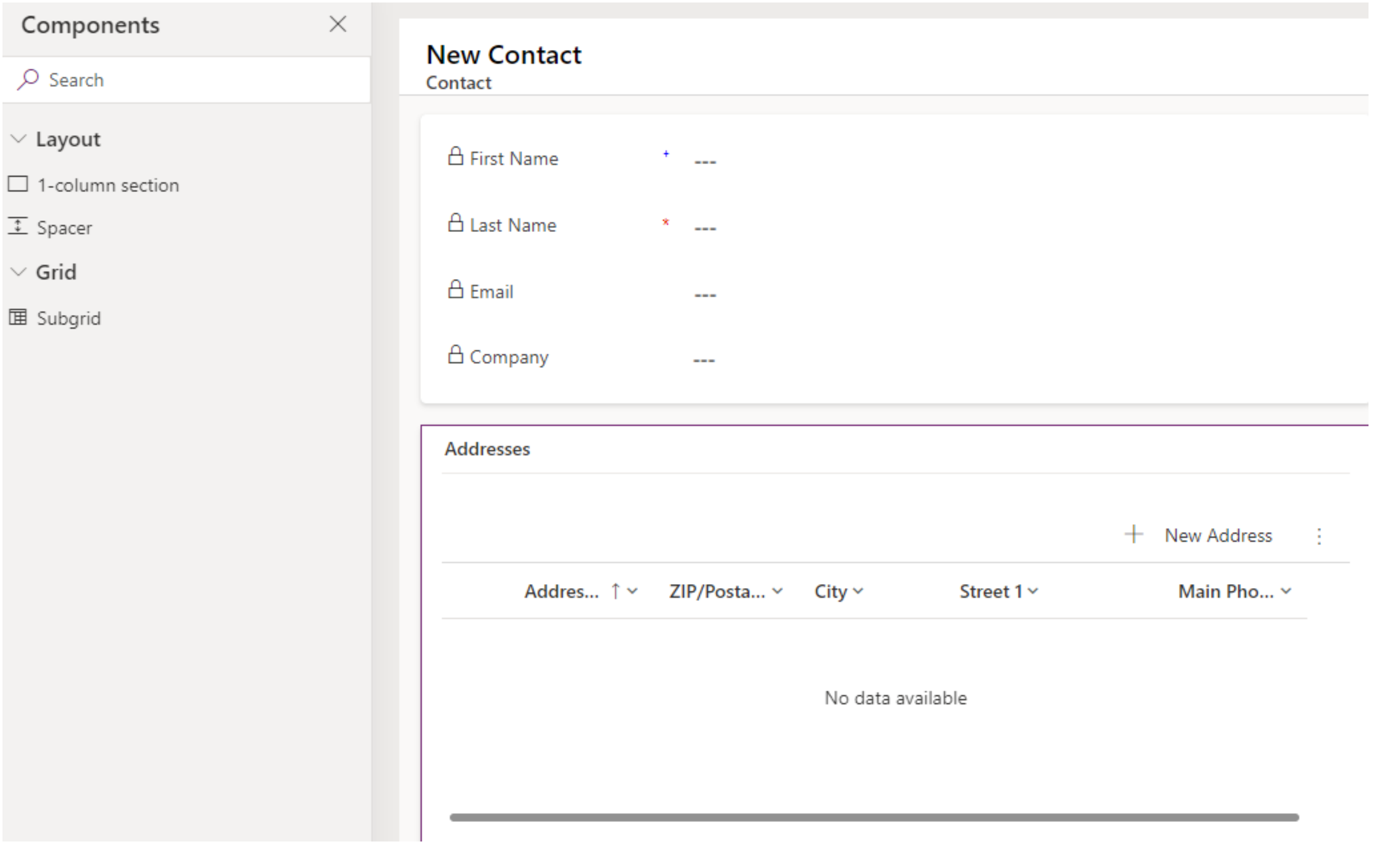
Subgrid Control

- Shows one-to-many and many-to-many relationship rows.
- Select the related table.
- Select a view to use to display data.



We are family

Show related data in a form



Quick view form control

- Shows columns from a many-to-one relationship.
- Select the lookup column.
- Select the Quick View form for the related table.
- Columns, 1-column sections, and subgrid controls can be added to Quick View forms.

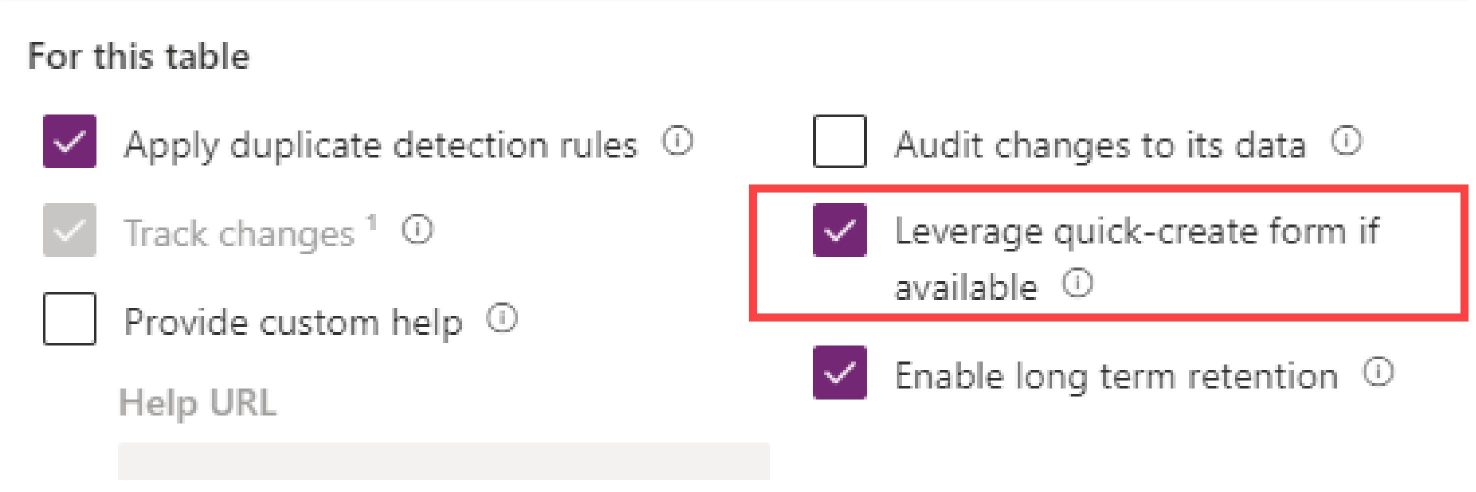
DEMO: Working with the form designer

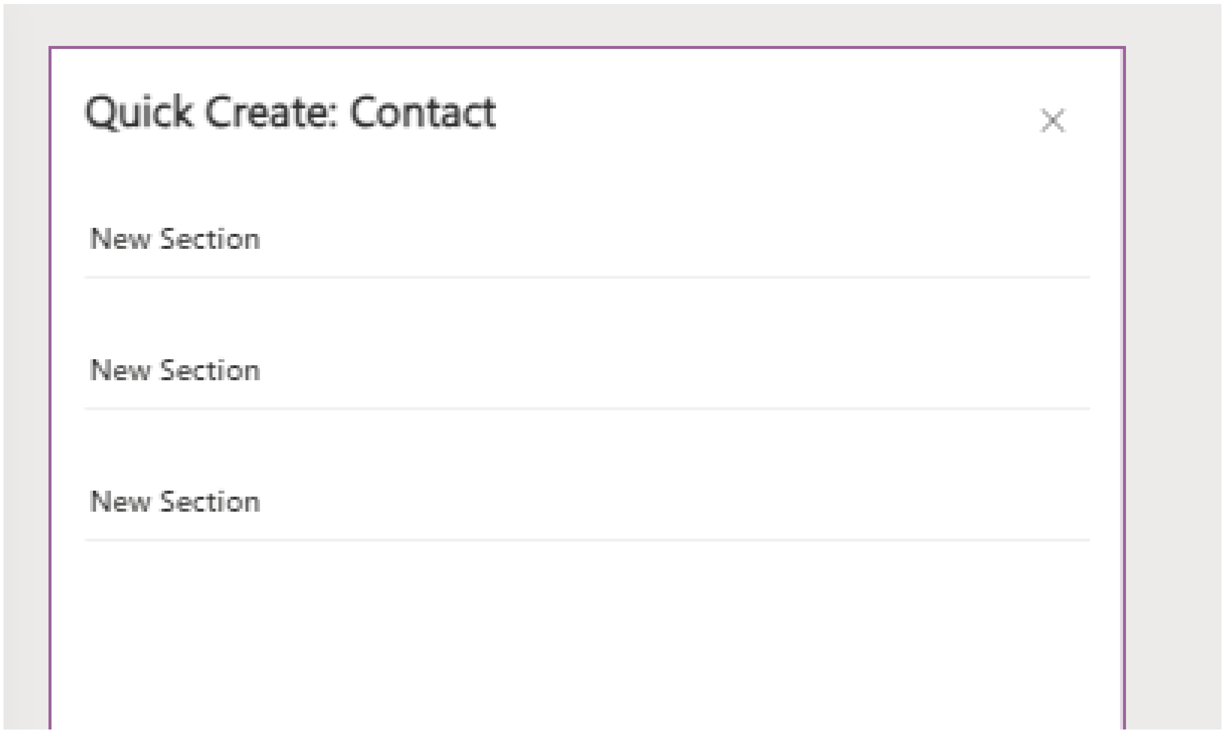
Quickly now

Quick Create form

Configure Quick Create

- Enable Quick Create for a table in table properties.
- Create a new form.
- Quick Create forms have a single 1-column tab with 3 sections.
- Can only add columns to a Quick Create form.

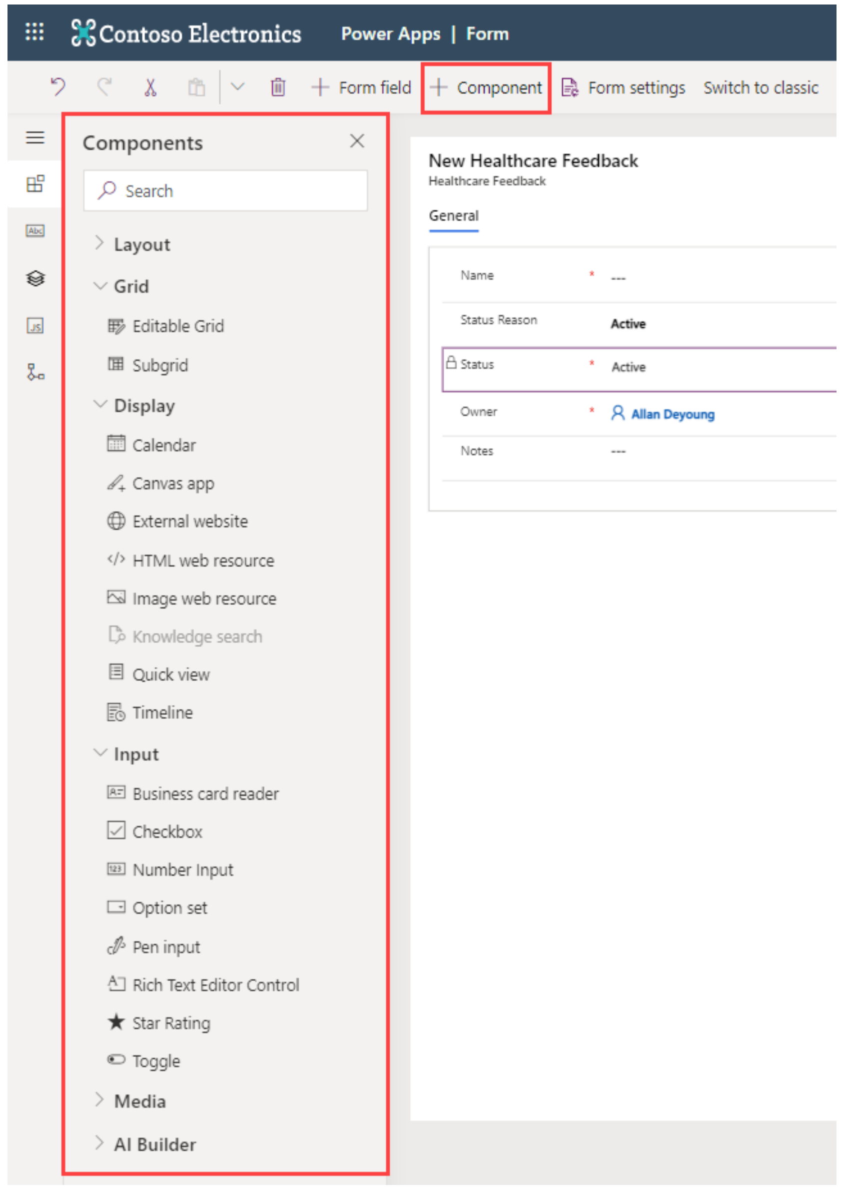


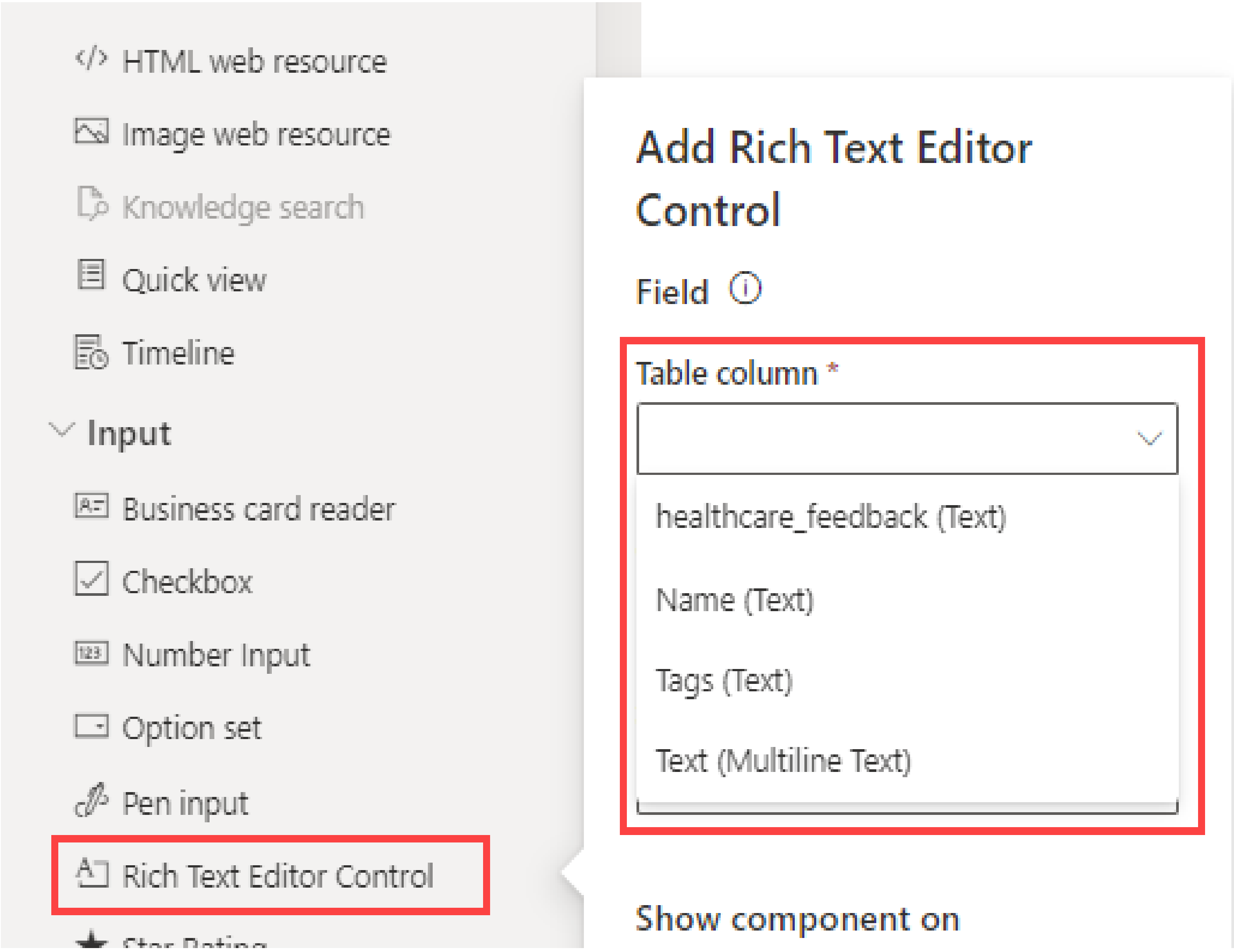


FCC FTW

Form component controls

Components can be added to columns on the forms to display how the control is displayed



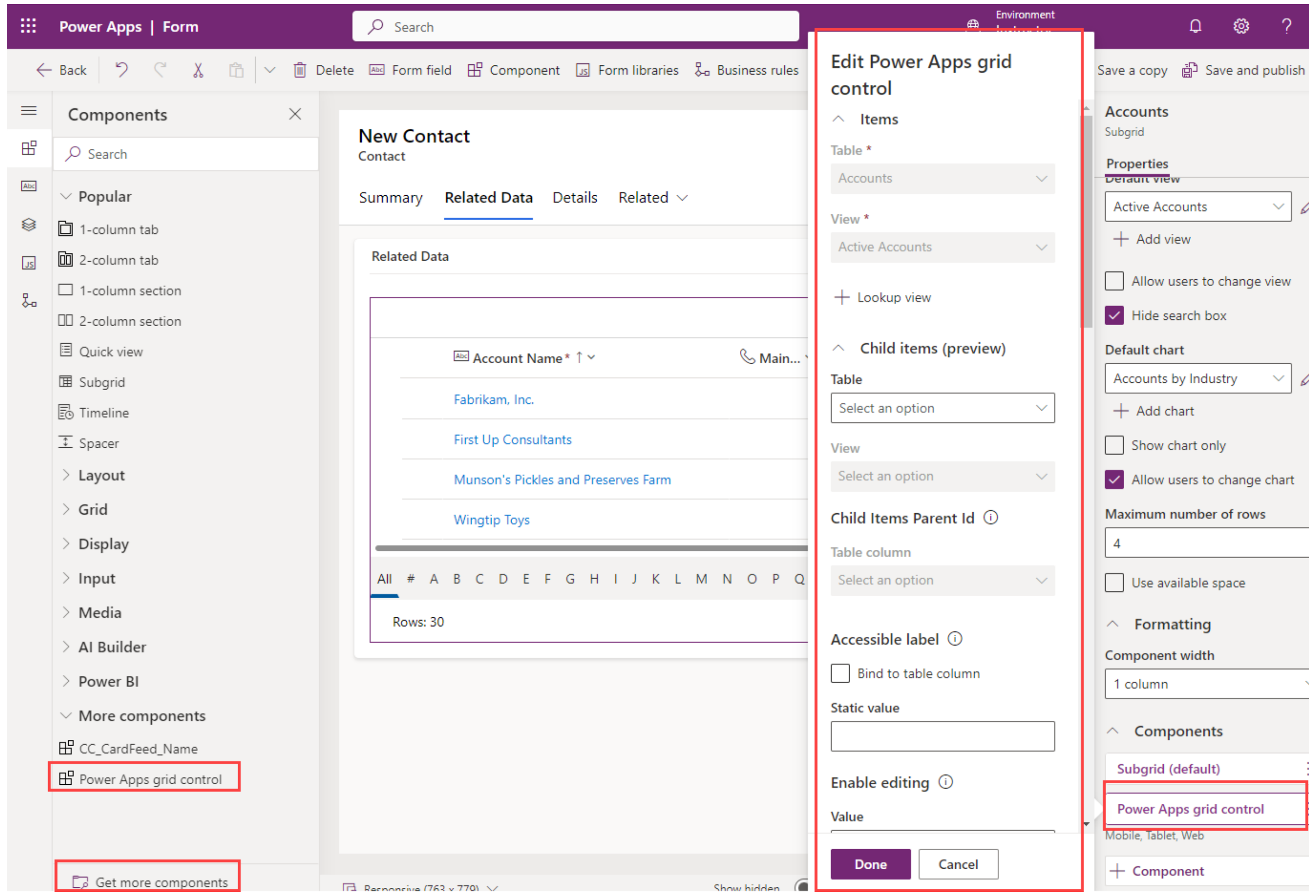


Hey, that's pretty grid

Power Apps grid control

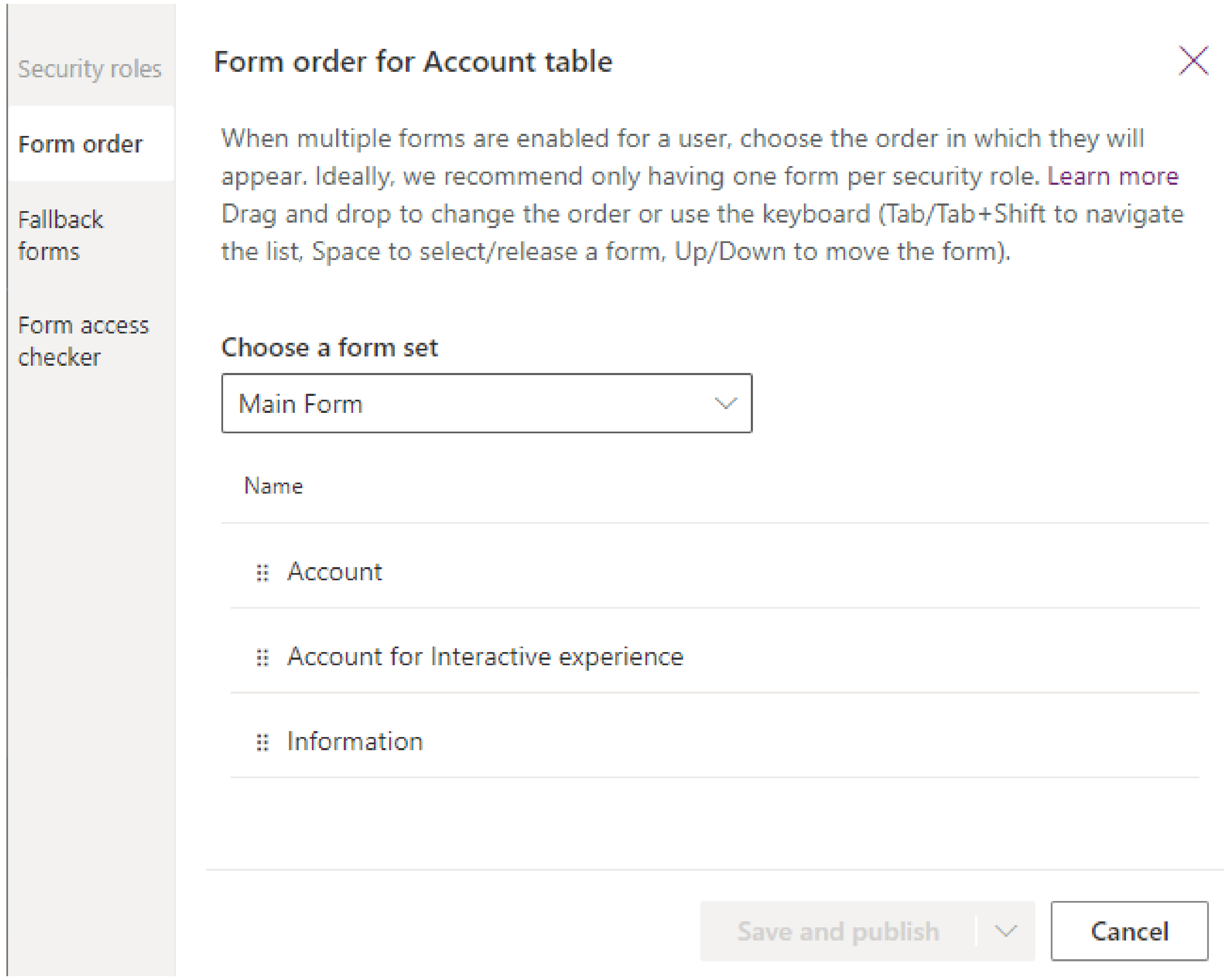
By default, grids are read-only

- Editable grids and Power Apps grid controls are special controls for editing columns in a view without needing to open a form.
- Add the Power Apps grid control to the form.
- Add Power Apps grid control as a component to a subgrid.
- Features:
- Read-only or Editable.
- Control reflow behaviour.
- Filtering.
- Sorting.
- Jump bar.
- Data type icons.



Hey, that's pretty grid

Power Apps grid control



- Main forms can be associated with security roles.
- Users will see the form that matches their security role.
- If the user has more than one form, the Form order defines which form the user will see.
- If, through security, the user has no forms then the user will see the form designated for fallback.

Let's get the form party started

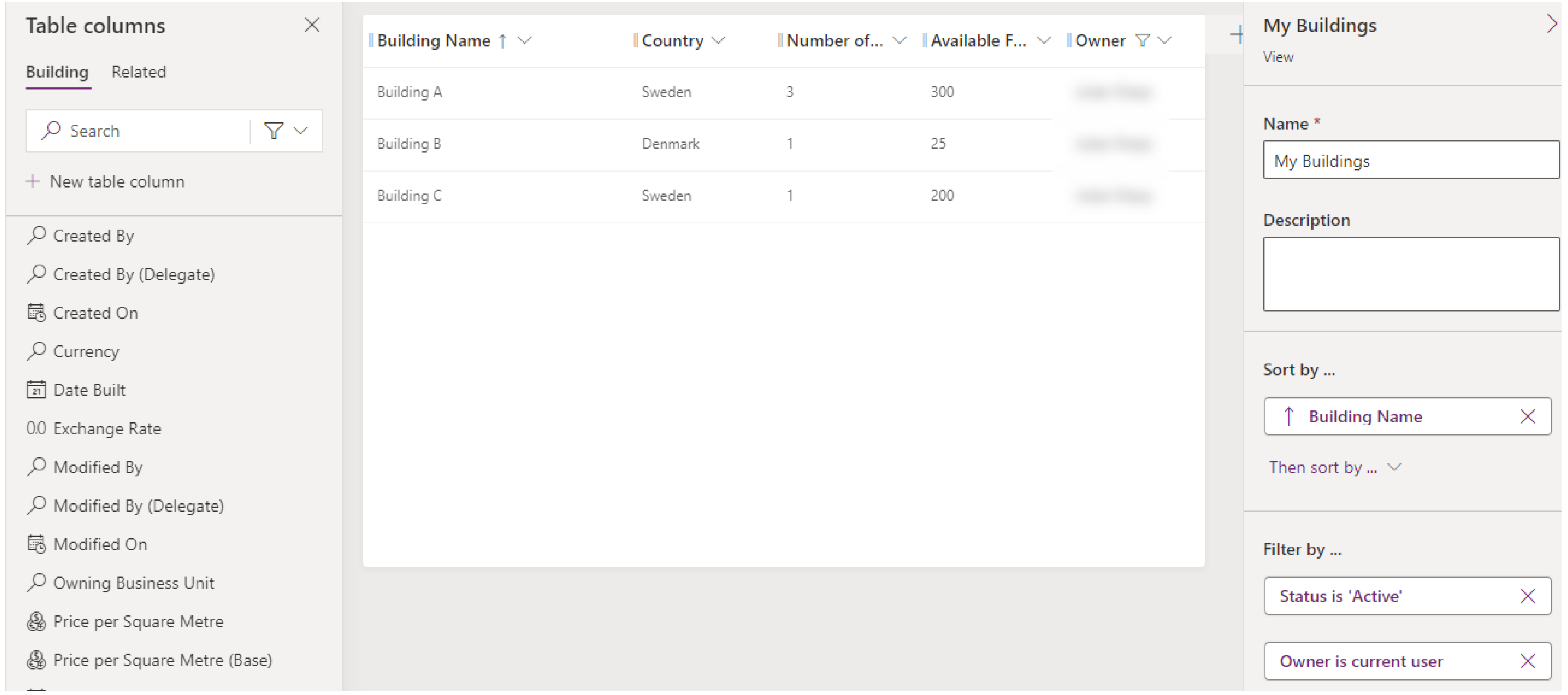
Configure multiple forms

- Only Main forms can be assigned to security roles.
- When a user has access to multiple forms, a form selector will be available near the top of the form. If a user has access to only a single form for a given table, there will be no form selector visible.
- You can designate a main form as inactive. This will make it not visible to all users, regardless of security roles.
- Controlling user access to forms is not necessarily a secure means to prevent access to data. Sometimes users have other ways to interact with data such as advanced find or automation.

What a view!

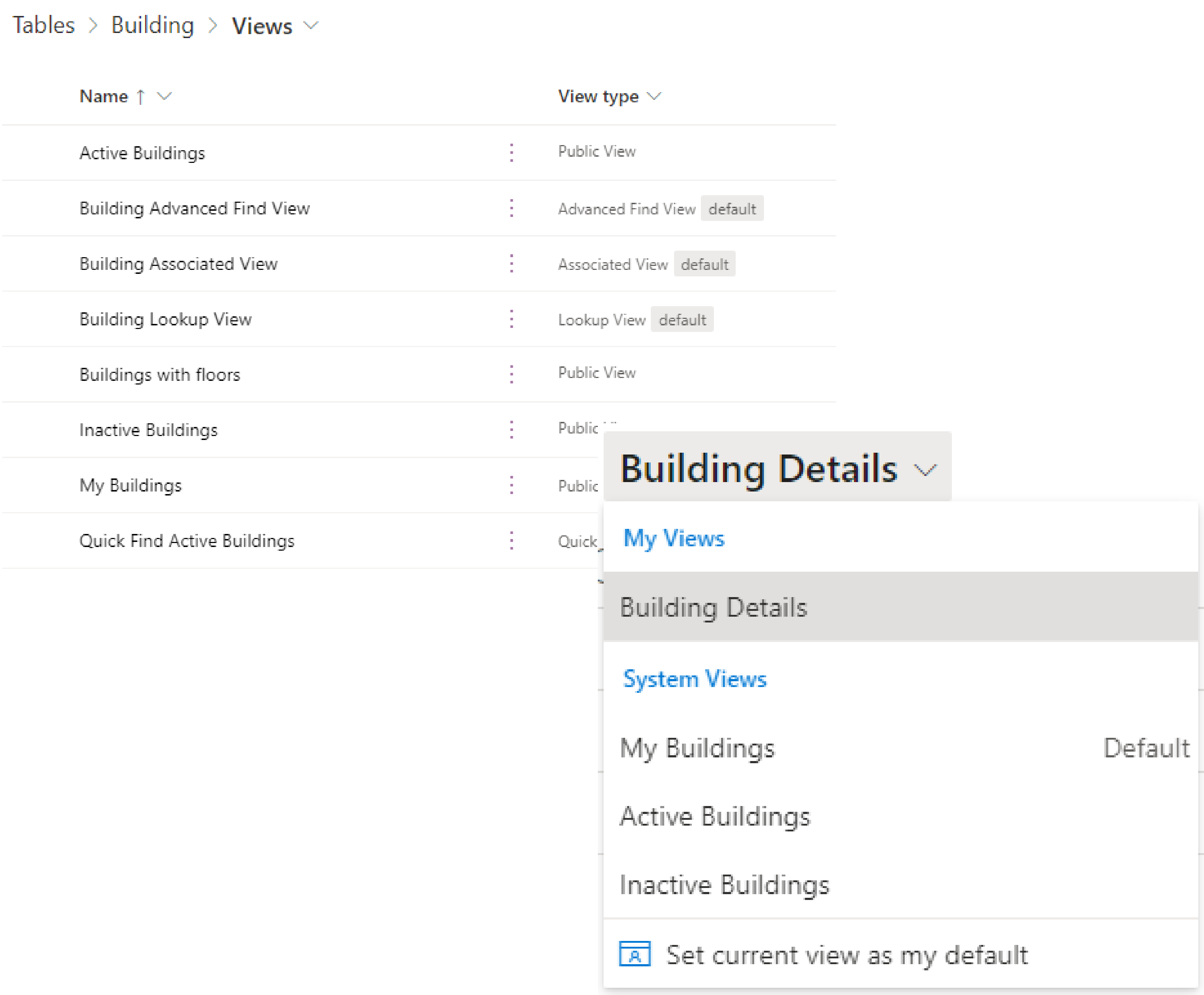
Views Overview

- **Views are lists of data from a Dataverse table**
  - Columns from table
  - Columns from many-to-one relationships
  - Sorting
  - Filtering



What a view!

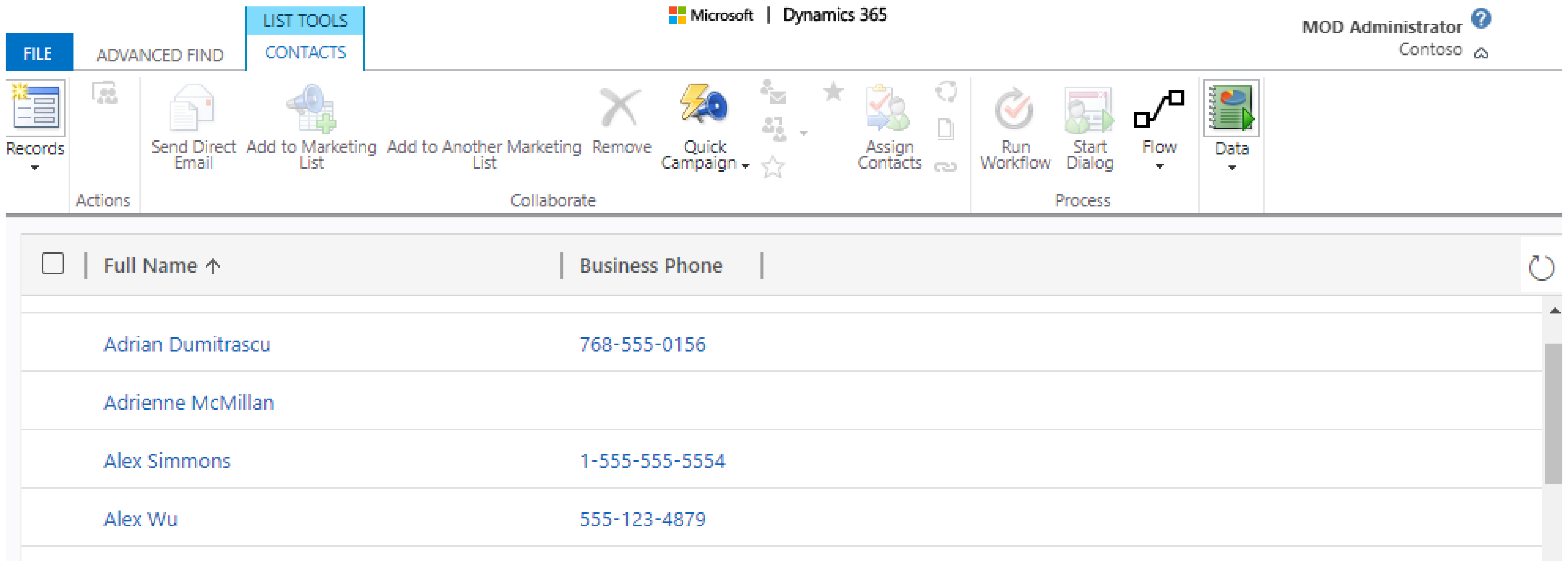
Public vs Personal views

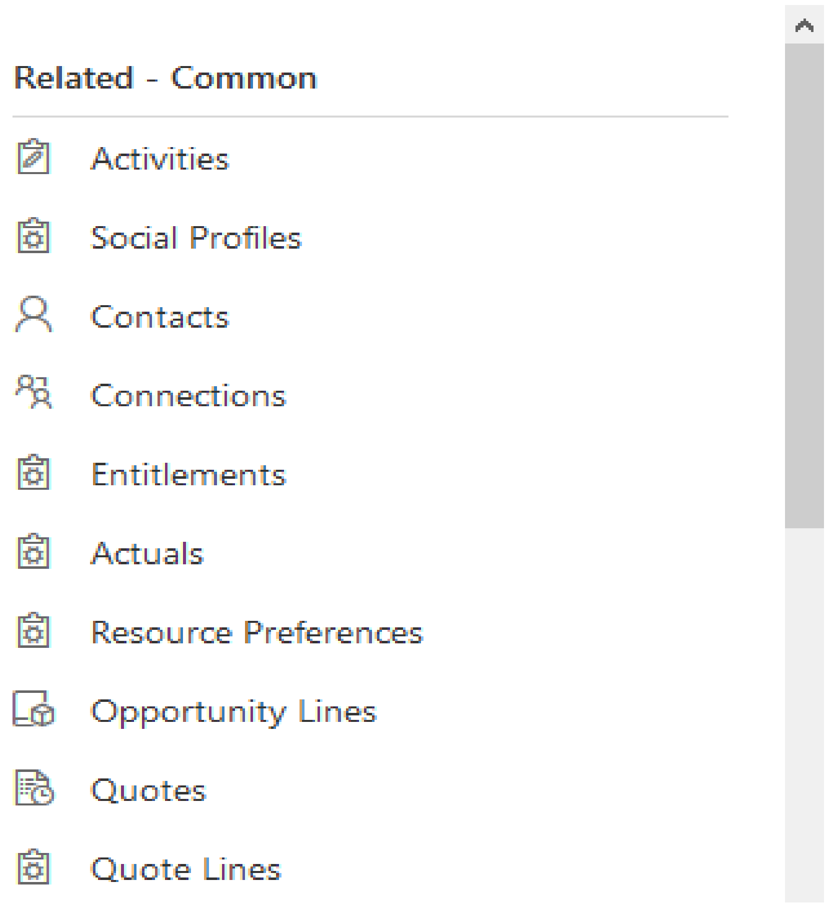


- **Public (System) views are managed in the Maker portal**
  - Public views are not controlled by security
  - Public views can be turned off (deactivated)
  - Public views can be included/excluded in an app
- **Personal (My) views are created in a model-driven app**
  - Personal views can be shared with users and teams

Chancellor, Views are our specialty

Specialty views



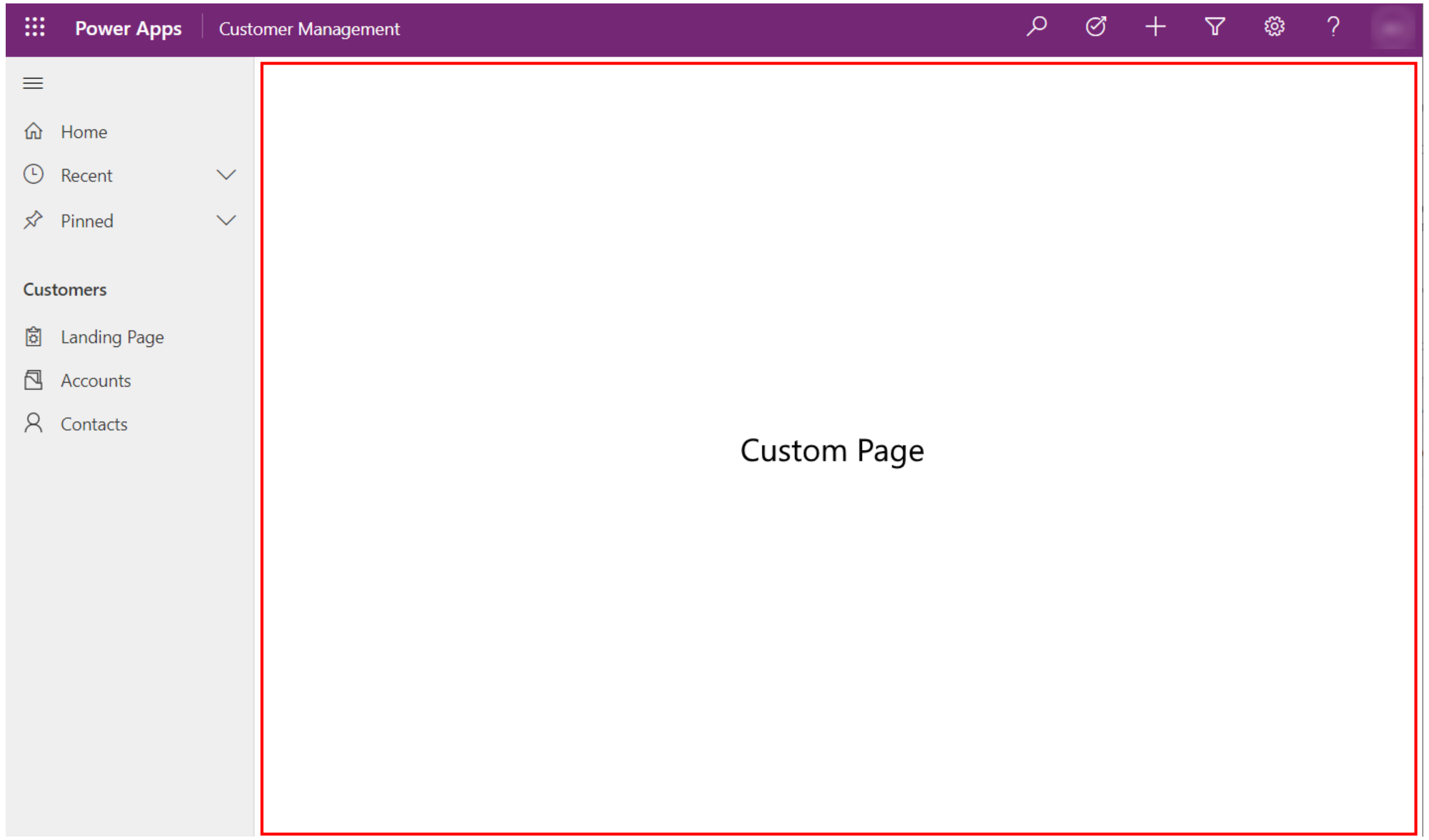


There are four types of specialty view; Lookup, Associated, Quick Find, Advanced Find (legacy)

DEMO: Working with views

A beautiful, blank canvas

Custom pages



What is a custom page?

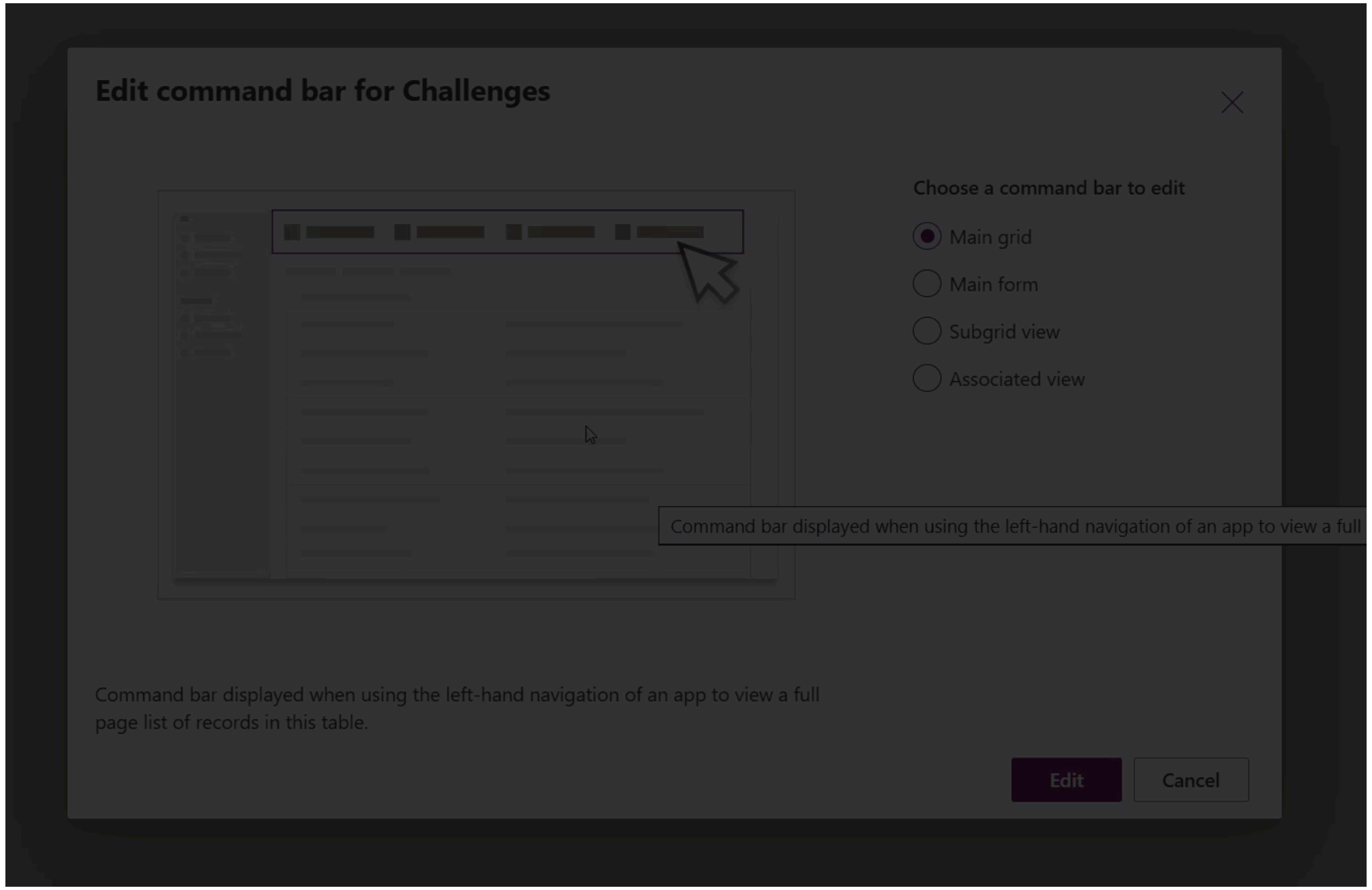
- A new page experience using the canvas app capabilities.
- Added to the site map in a model-driven app.
- Uses connectors to access data external to Dataverse inside a model-driven app.
- Recommend to use instead of embedding canvas apps.
- Separate component in a solution.

On my command, unleash Hell

Command bar

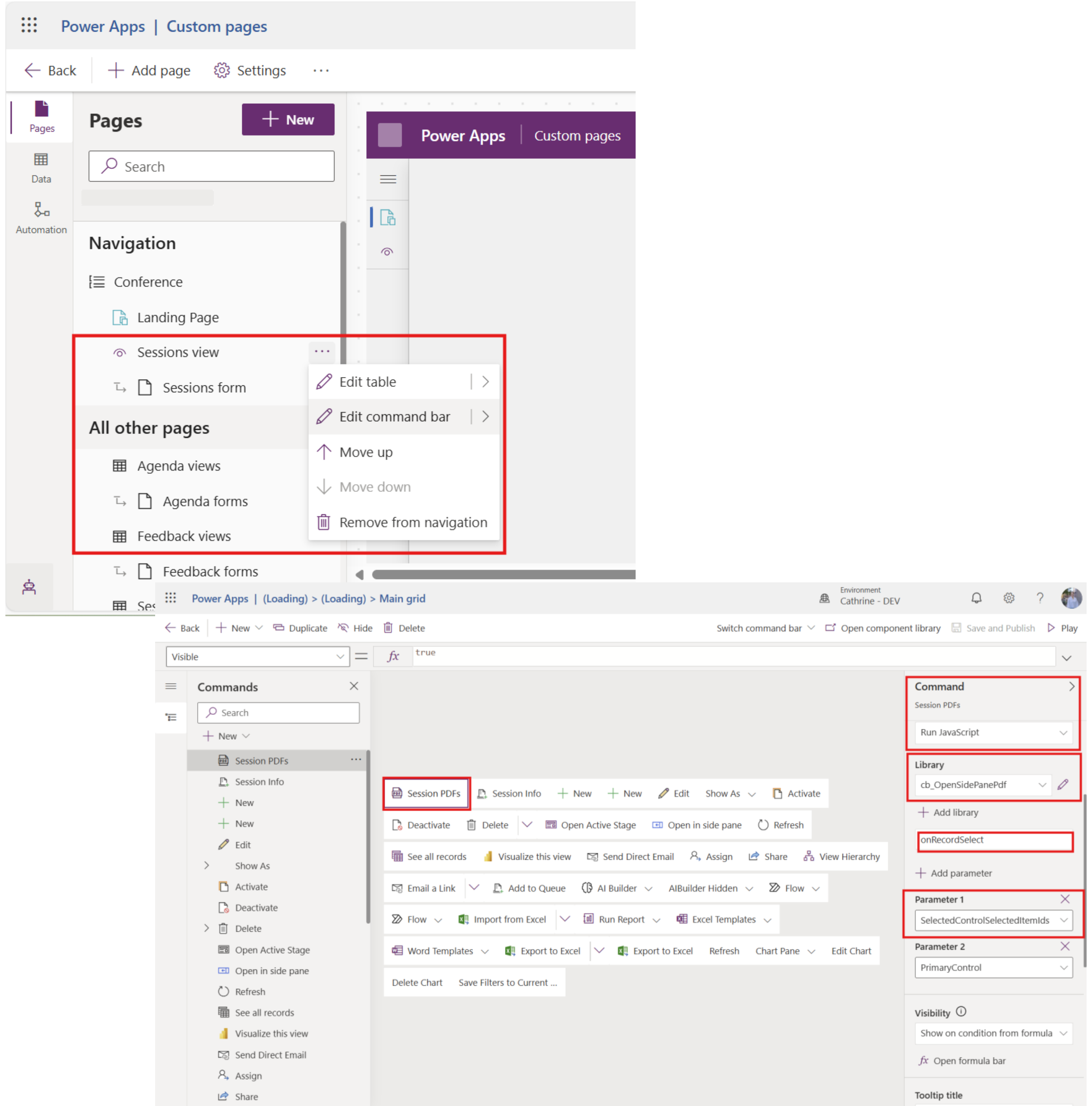
Locations

1. Main grid
2. Main form
3. Subgrid view
4. Associated view



What is this I don’t even

Creating a Command



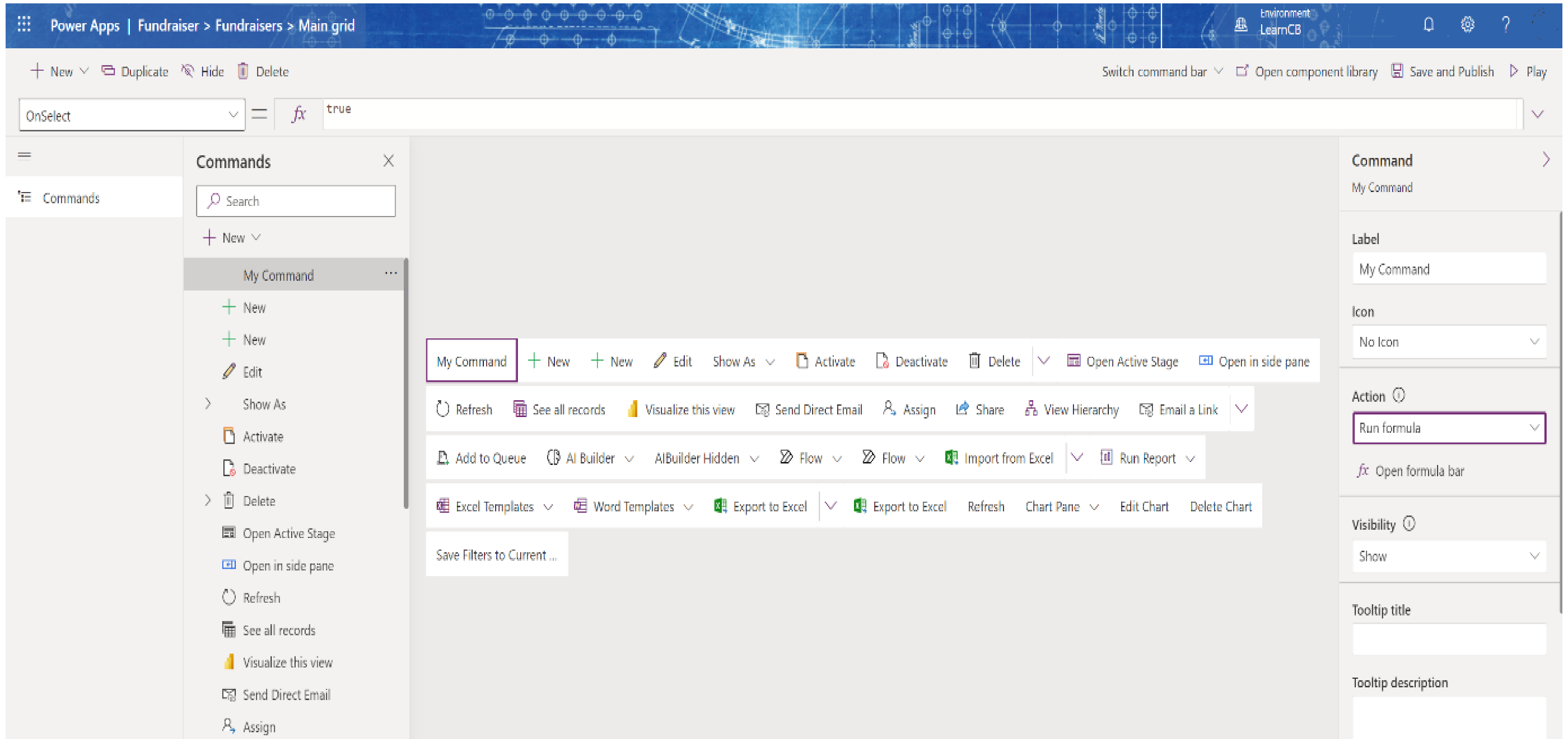
1. Select your table and the three dots on the right
2. Select «Edit Command bar»
3. Choose whether the button should be visible on the form or from the Main Grid
4. Select «+New» - «Command»
5. On the right side, provide a name for the button
6. Select «Run JavaScript»
7. Select or search for the web resource as the library
8. Set the function name «JavaScriptFunction»
9. Set parameters
10. Save and publish – Play the app to test

RIP Ribbon Workbench

Command bar designer

Edit in model-driven app

1. Choose command bar
2. Visual representation
3. Add new commands
4. Modify existing commands



Select Edit command bar → Choose command bar to edit → Add a button → Drag button to desired location → Configure visual display → Configure action → Manage visibility

Lights 🔦 Camera 🎥 Action 🎬

Configuring actions

Power Fx

- Properties
  - OnSelect
  - Visible
- Creates a component library
  - Ensure this is present in your solution
- Example use cases
  - Access related rows
  - Create and update rows with Patch

JavaScript

- Requires JavaScript web resources

DEMO: Working with the modern command designer

Before you get too excited...

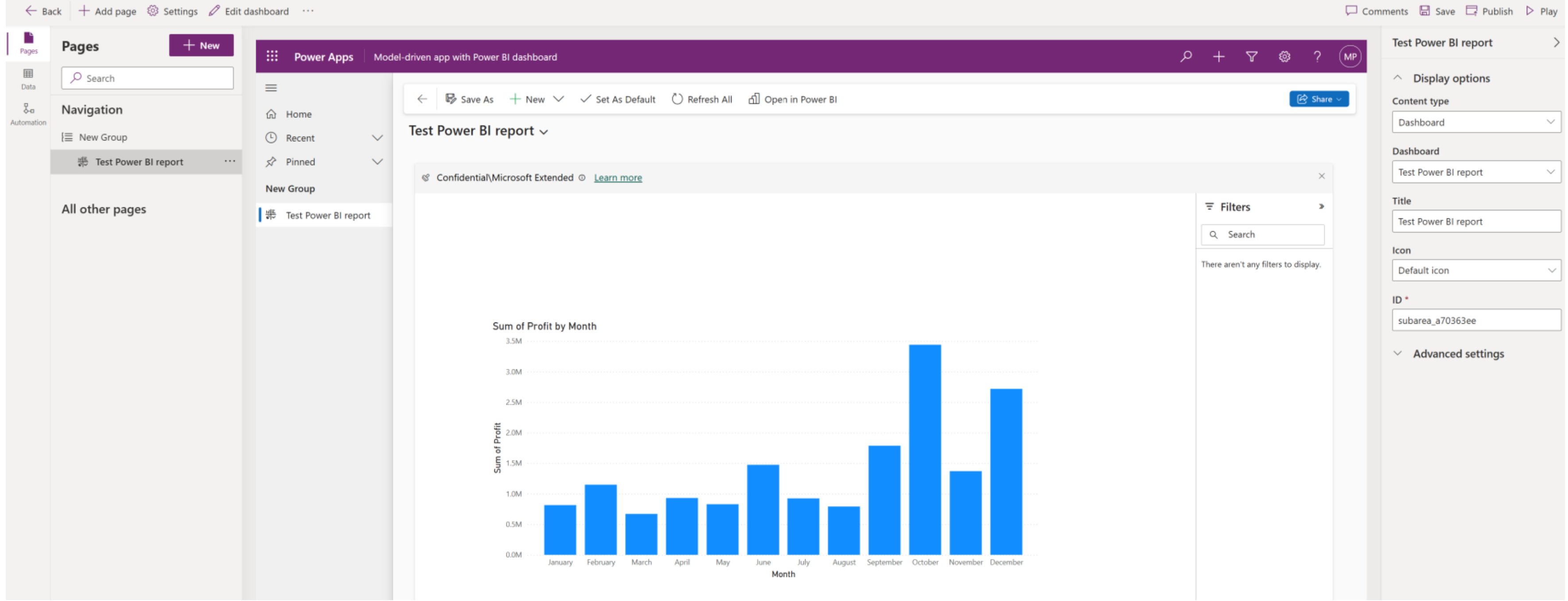
Power Fx commands - Considerations and Limitations

- Visible and OnSelect are the only supported properties
- Specific functions are not supported
- Unable to execute Power Automate cloud flows
  - Consider a Patch() operation and Dataverse connector trigger
- No support for additional data sources
- Power Fx commands do not work in the Dynamics 365 App for Outlook
  - Consider using JavaScript in this scenario

Make that data look snazzy

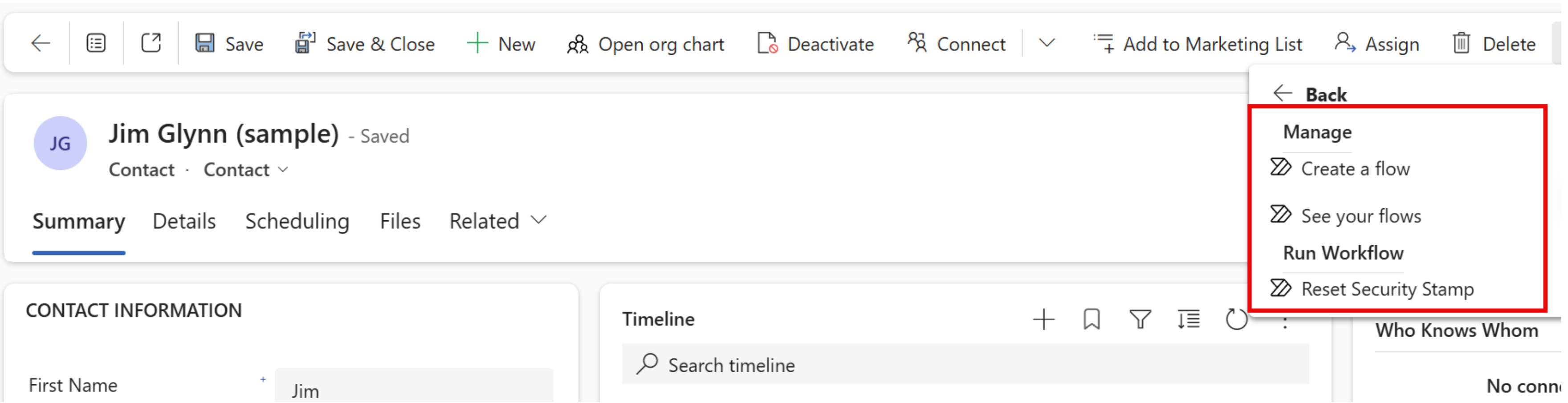
Additional Integration Options – Power BI

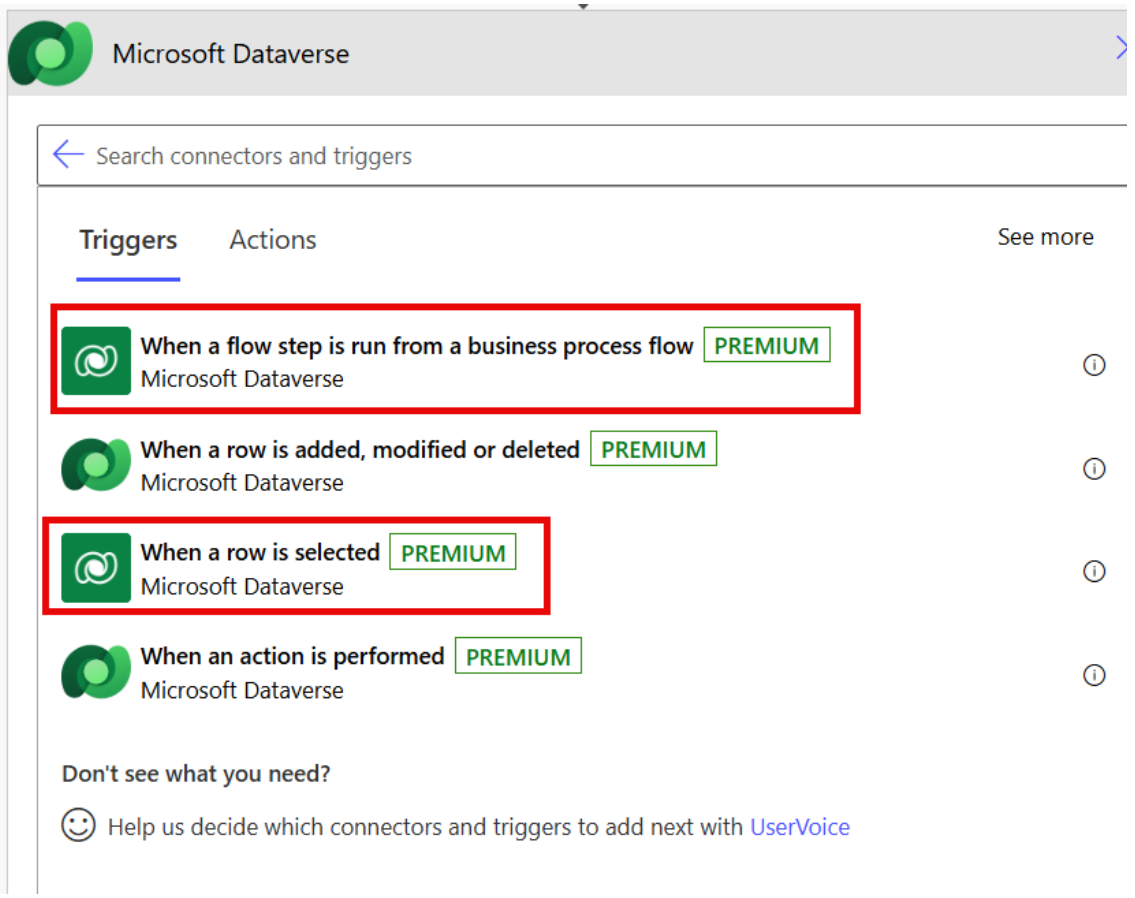
- Embed Power BI content in your apps.
  - Model-Driven
    - Report
    - Dashboard
  - Canvas Apps / Custom Pages
    - Tiles from dashboards only
- Consider ALM implications
  - Separate workspaces
  - Use environment variables



Go with the flow

Additional Integration Options – Power Automate



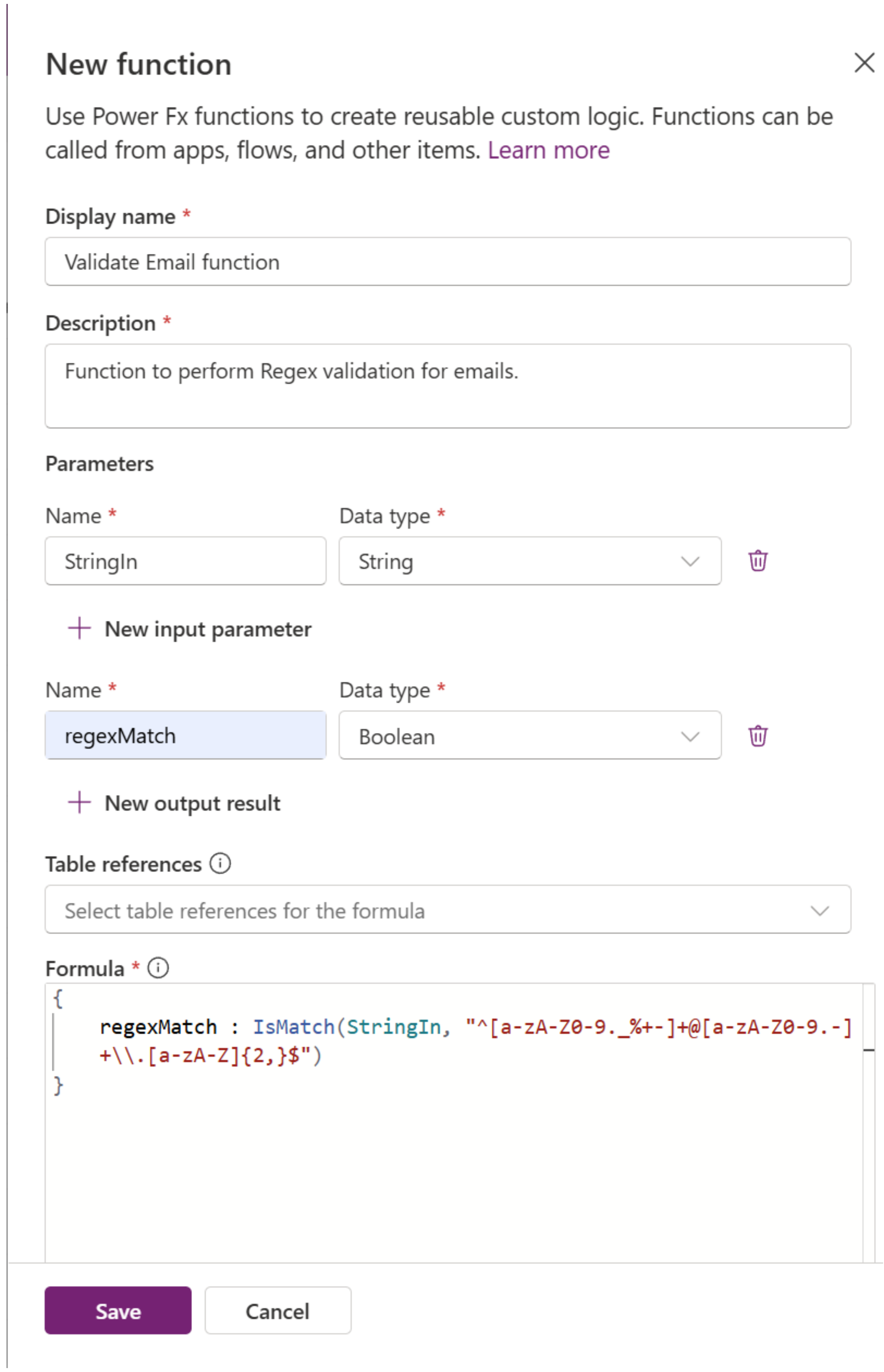


- Execute a cloud flow from
  - Any model-driven app row
  - Within a Business Process flow step
  - Canvas app / custom page:
    - *'My Flow'.Run(parameter1, parameter2)*
- 30+ actions targeting Microsoft Dataverse
- Ideal for more complex scenarios:
  - Complex iterations
  - Working with XML / JSON data
  - Child flows

I’ve got the Power (Fx)

Additional Integration Options – Power Fx

- Power Fx functions:
  - Solution-aware component
  - Support input / output parameters.
  - Can be bound to a record.
  - Some functions not supported
- Low-code plug-ins (preview)
  - Low-code mechanism to trigger synchronous logic in Dataverse
  - Work with via the Dataverse Accelerator app



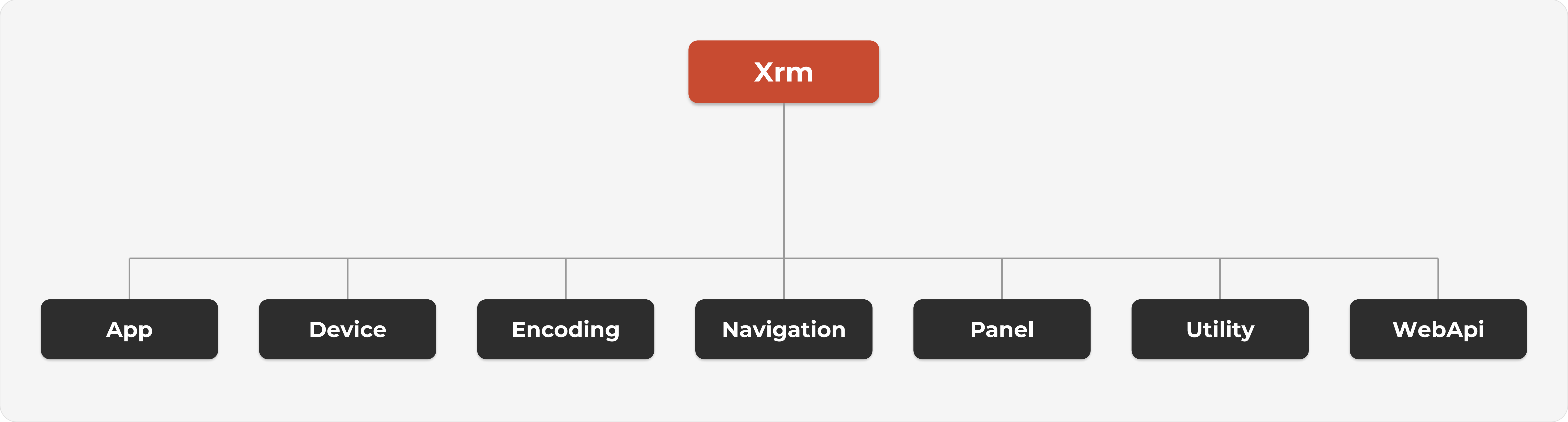
Client-side scripting

JavaScript in model-driven apps via the Xrm object model

What is the Xrm API?
A JavaScript object model in model-driven apps. Interact with forms, controls and records at runtime.

Key namespaces
formContext for form data & UI, Xrm.WebApi for CRUD, Xrm.Navigation for dialogs & URLs.

Why it matters
Validate fields, show/hide sections, auto-populate data and call services — before a record is saved.



Al Gore invented it (kind of)

Web resources

- Upload files to Dataverse
- Virtual file system
- Use in forms and apps
- Use as JavaScript libraries

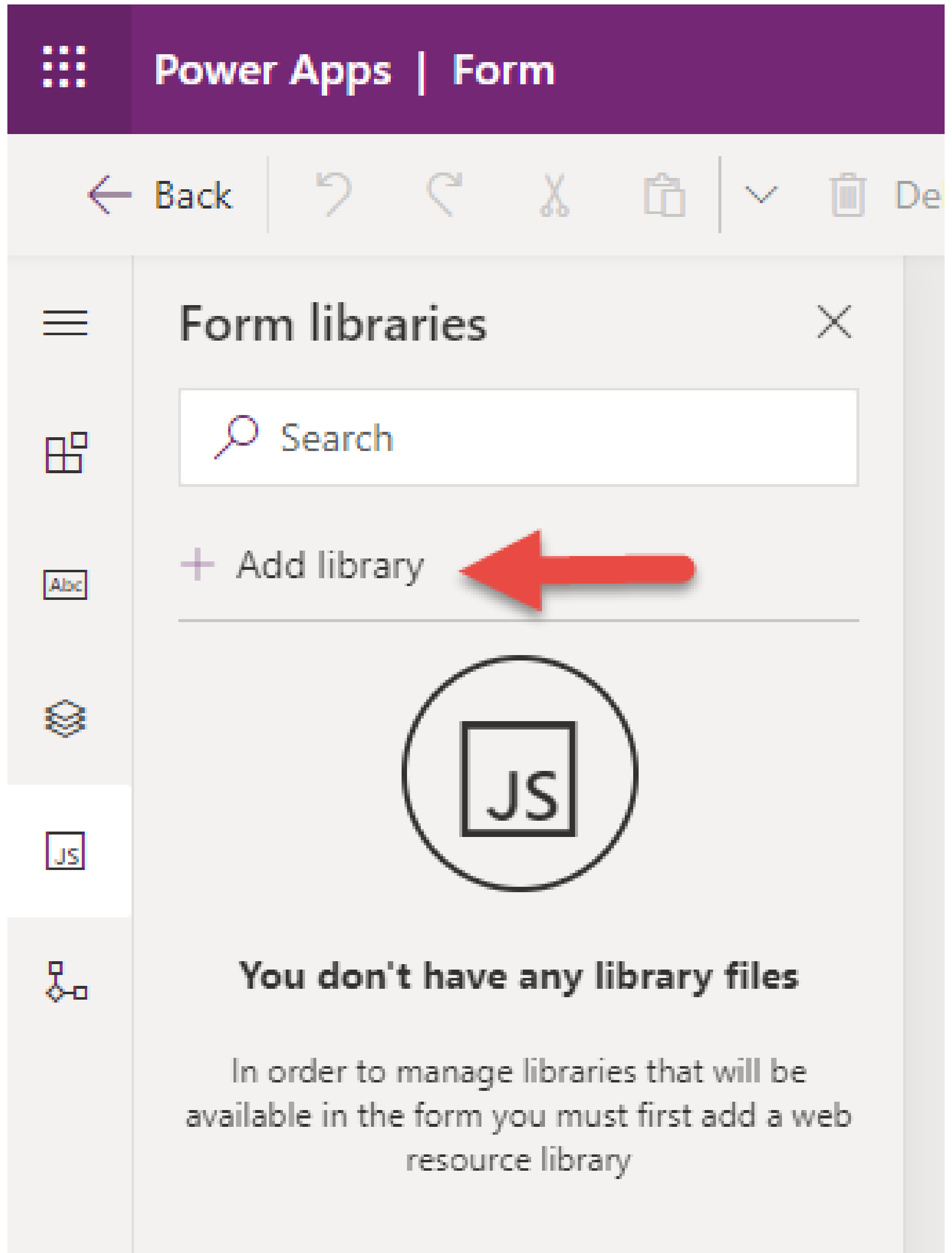
Types:

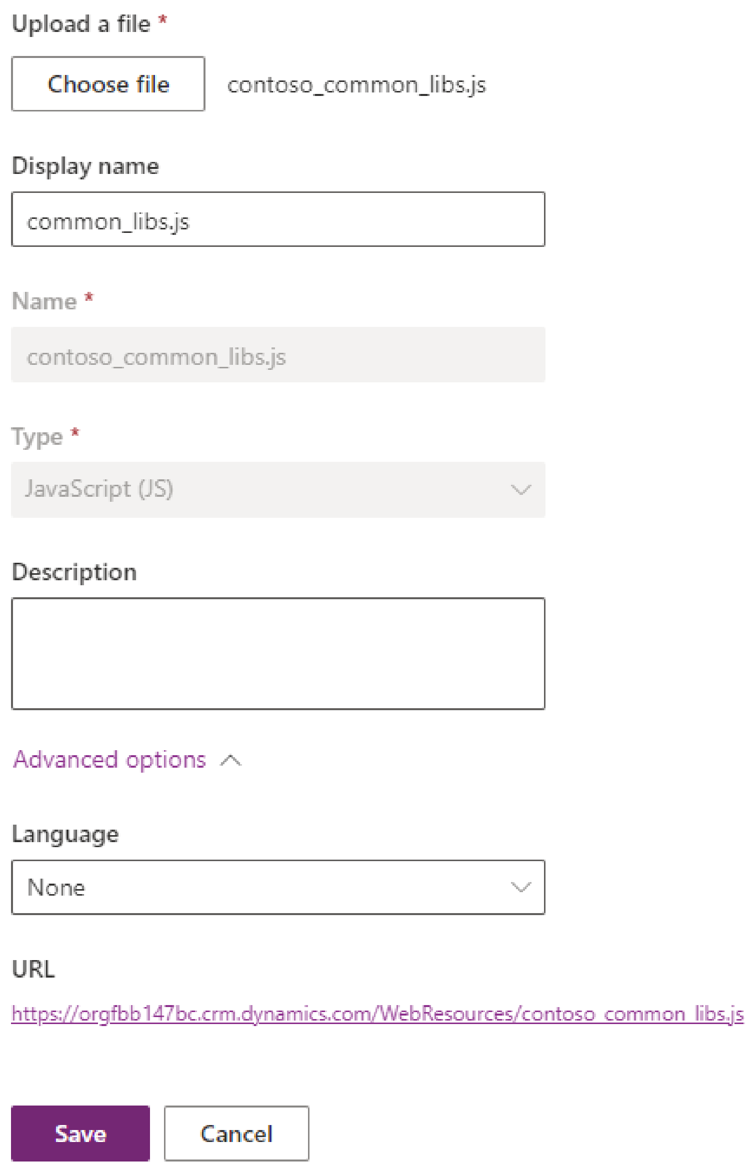
- Script (Jscript)
- Webpage (HTML)
- Style Sheet (CSS)
- Data (XML)
- **Images**:
  - PNG
  - JPG
  - GIF
  - ICO

Totes legal code injection

Upload scripts

To use client scripting on a form the script must first be uploaded as a JScript web resource

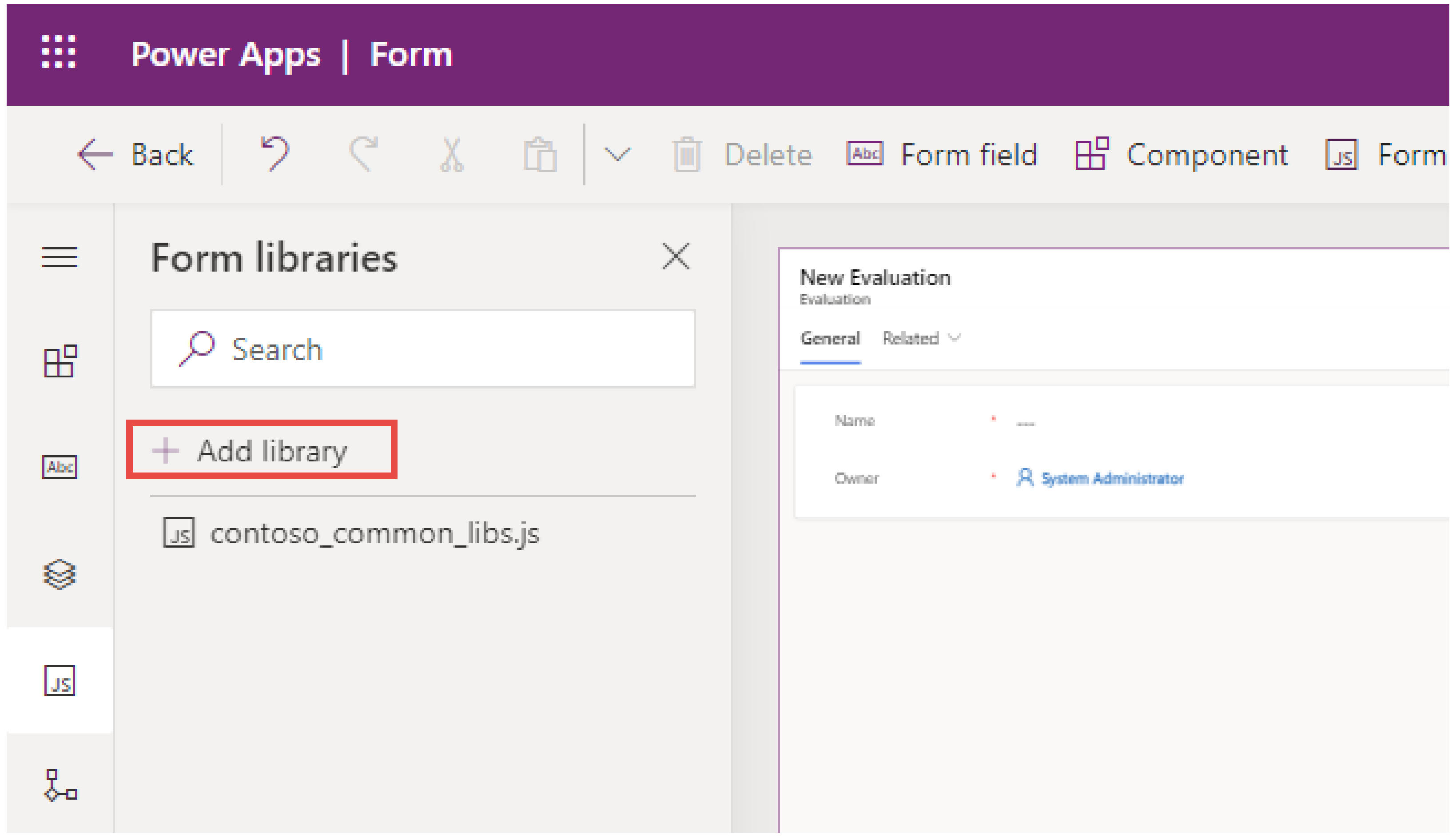


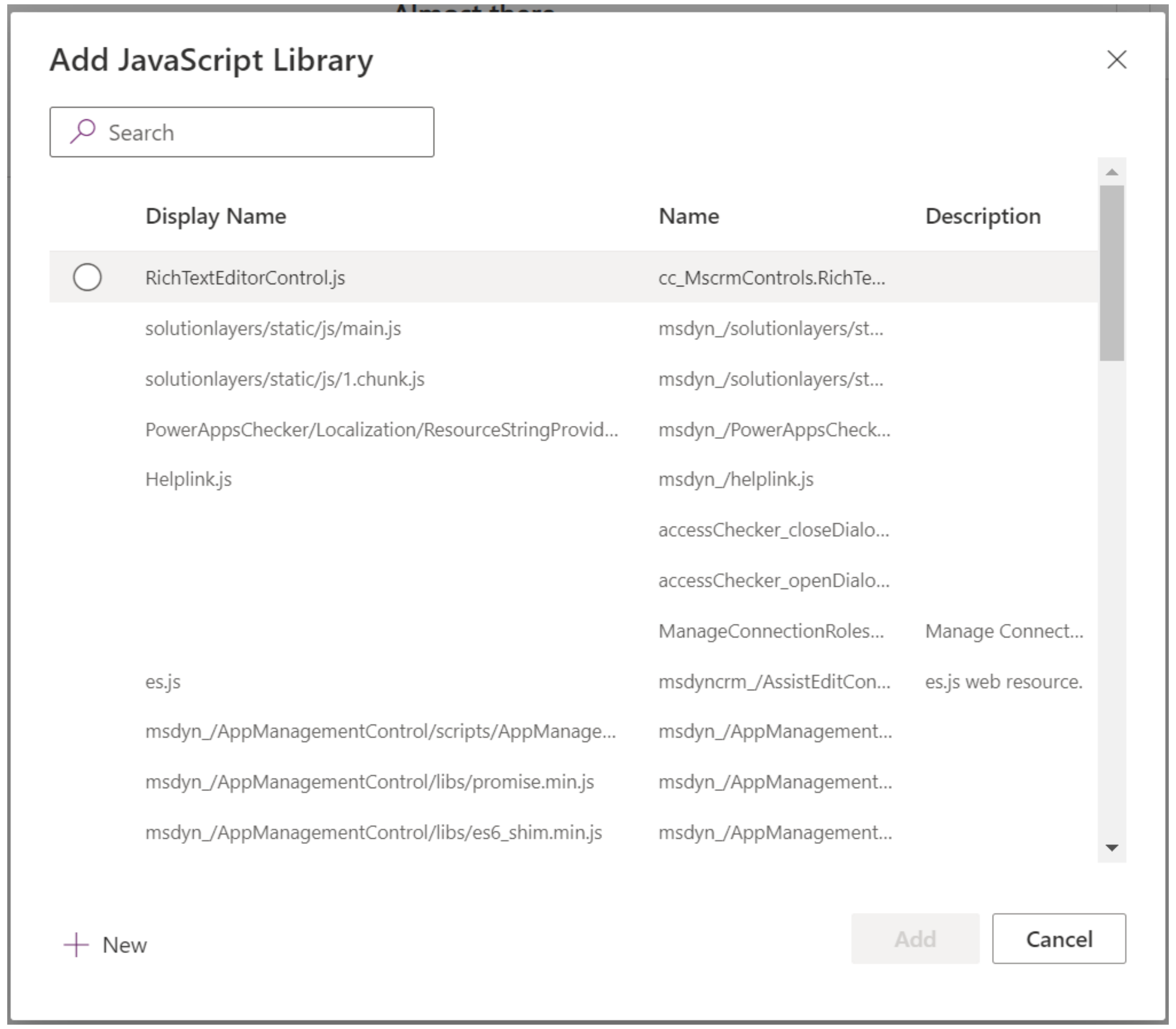


A few itty, bitty steps

Using client script libraries

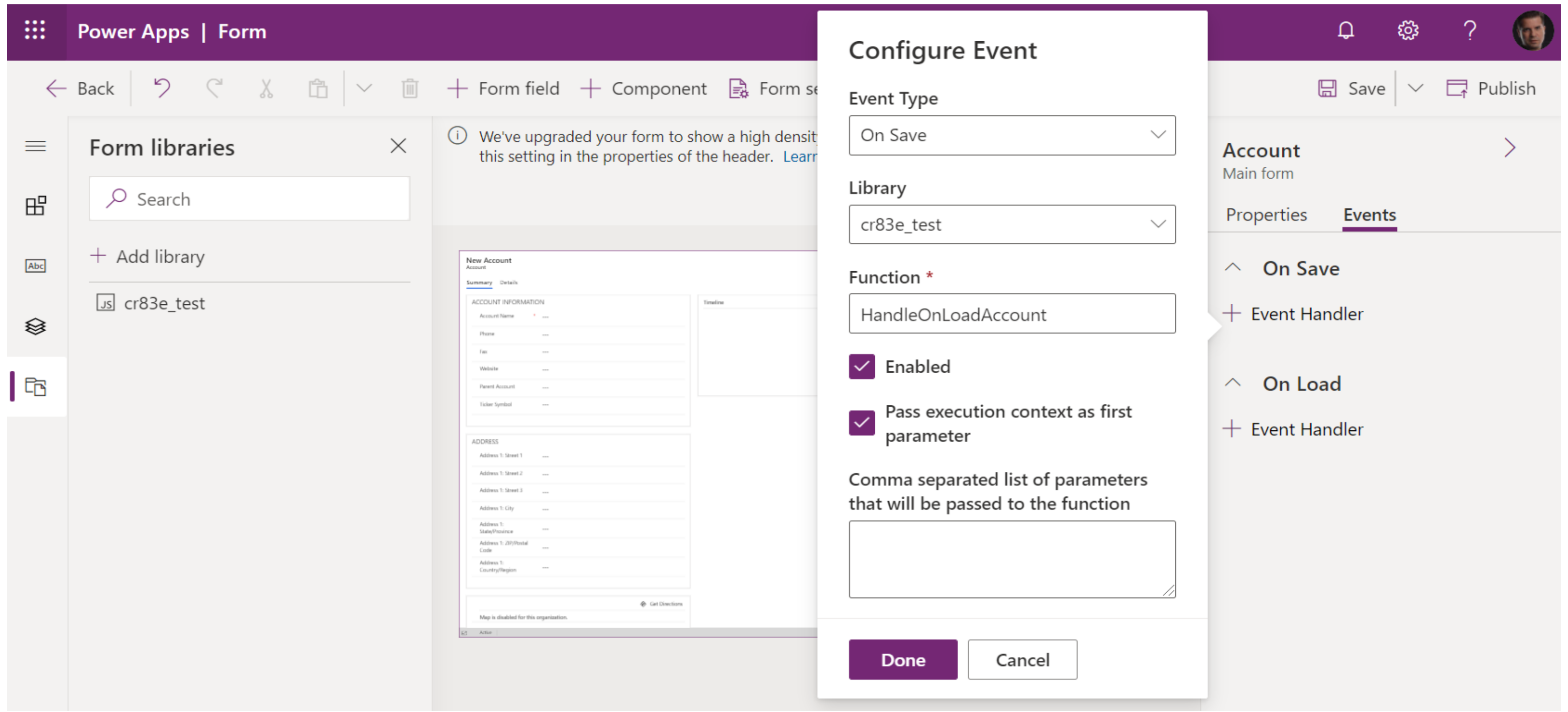
Once configured as a script web resource, client script libraries can be associated with commands and form events





Registration is required

Registering via form properties



Registering event handlers using form properties creates a static configuration of event handlers at design time.

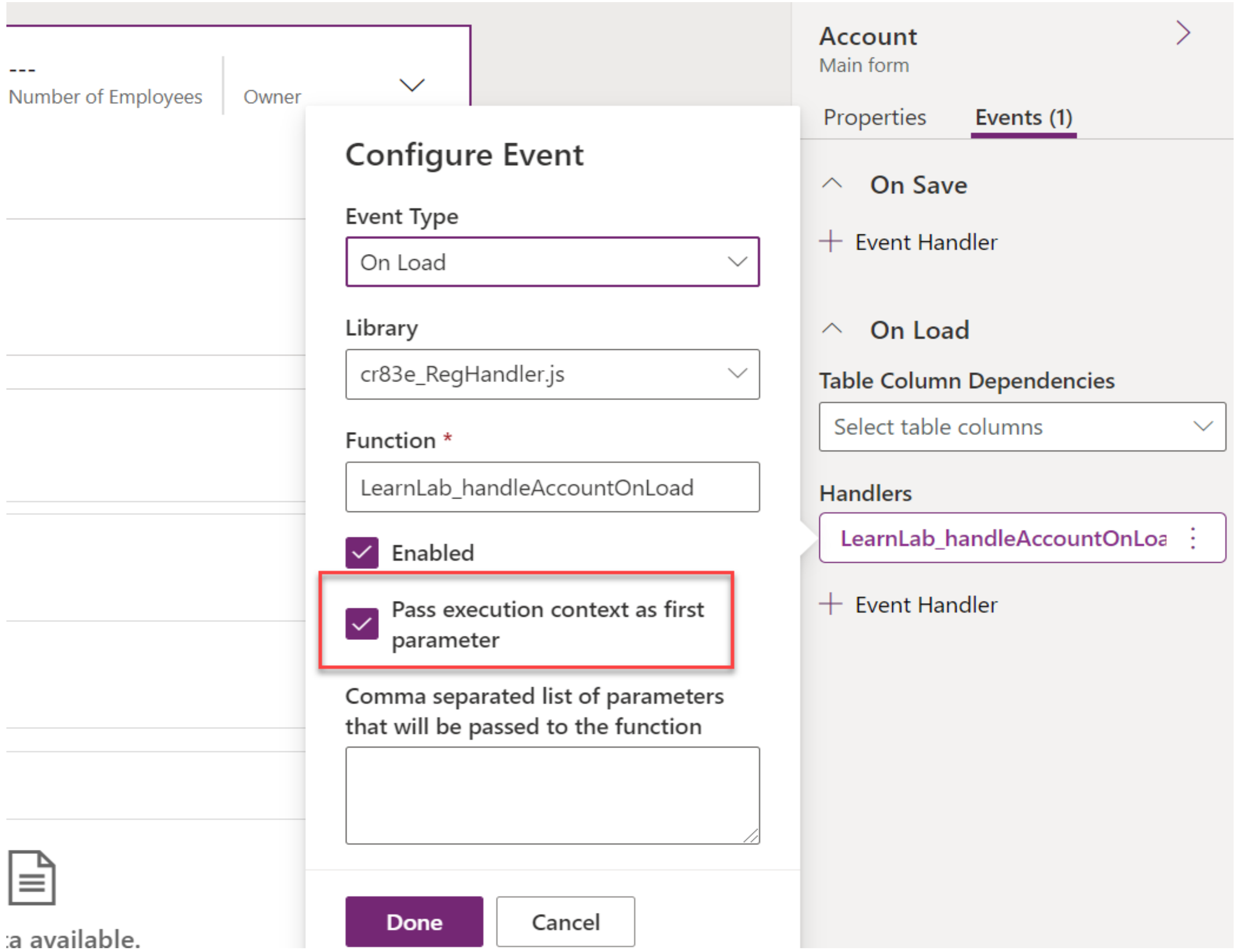
- **Form** – This allows you to register **OnLoad** and **OnSave** event handlers.
- **Columns** – This allows you to register an **OnChange** event handler if column data is changed

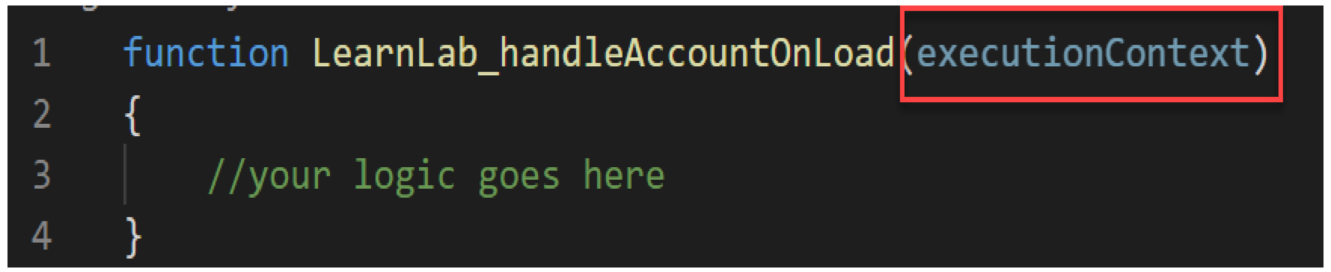
Module 2
Lab 1
Module 3
Lab 2
Lab 3

Give me some context

Execution context

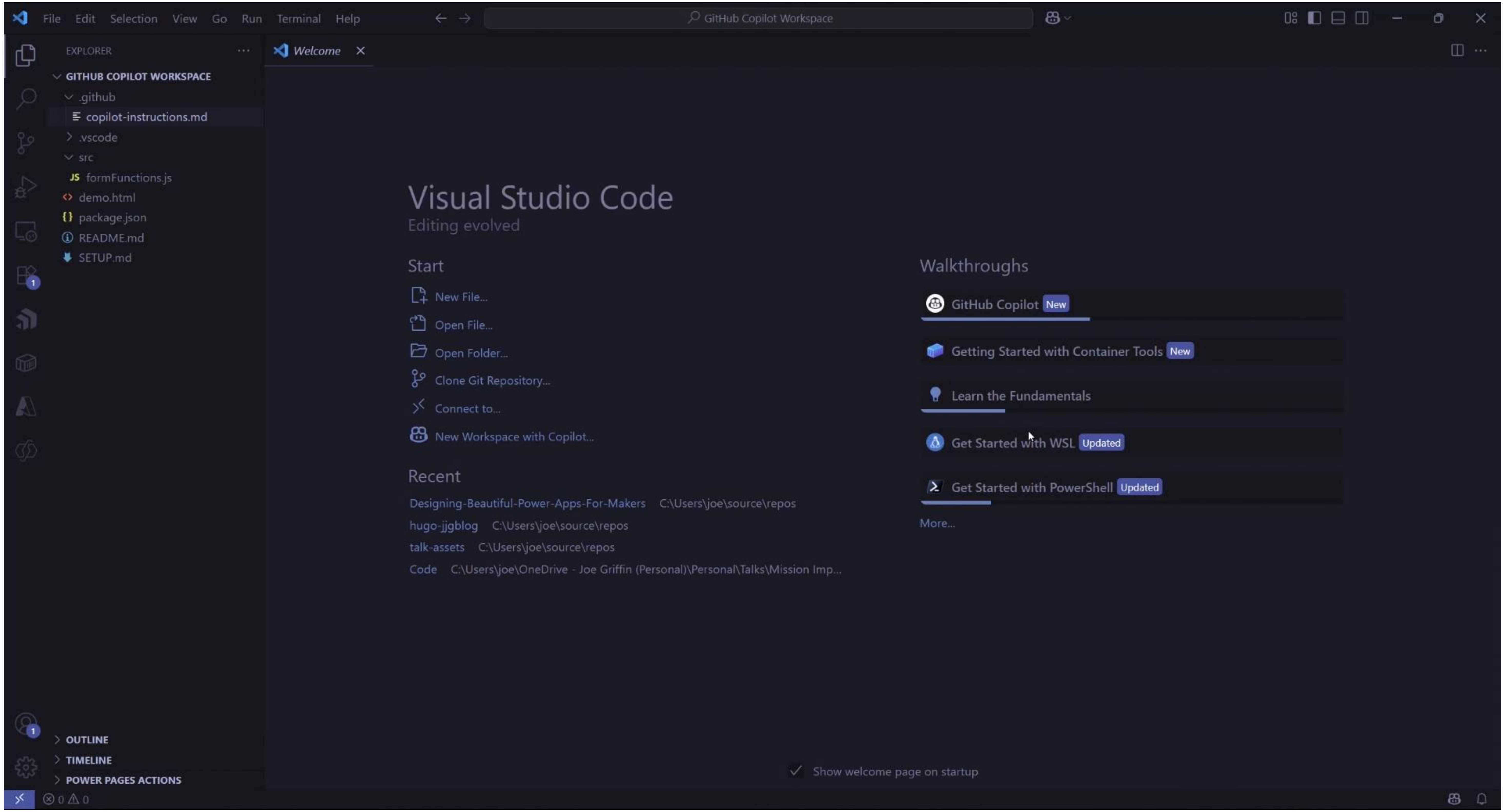
When you register an event handler, you can have the execution context passed as the first parameter.





Ohhh, we vibing now 😎

Use GitHub Copilot to achieve your inner “vibe coder”

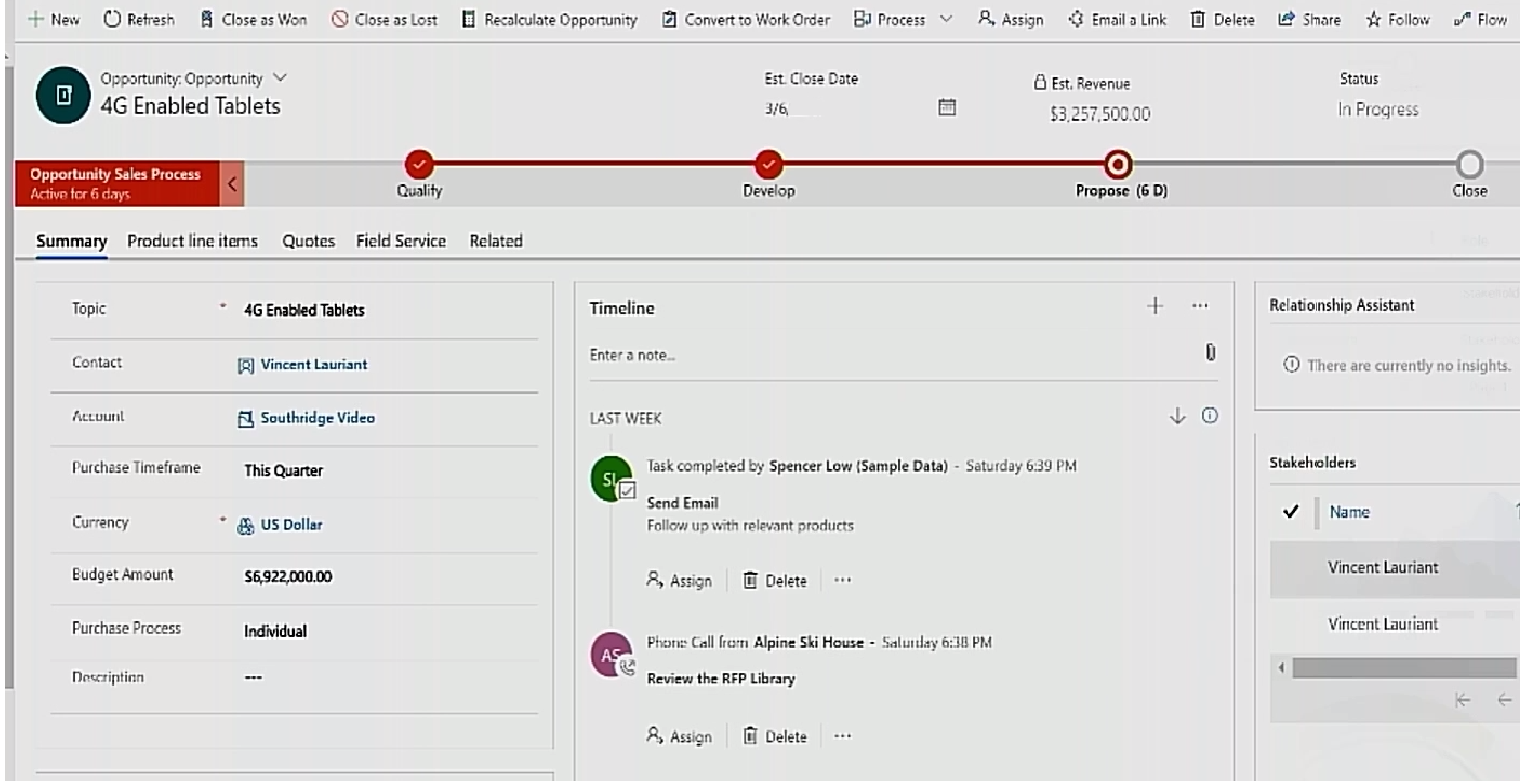


DEMO: Developing a simple form function to open a custom page

That’s a fancy PCF you have there

Introduction to Power Apps component framework

Microsoft Power Apps component framework helps you create reusable components that can be used within your Power Apps applications.



That’s a fancy PCF you have there

Introduction to Power Apps component framework

However, if you reconfigured your app to use custom Power Apps components, your app might look something like the following image.

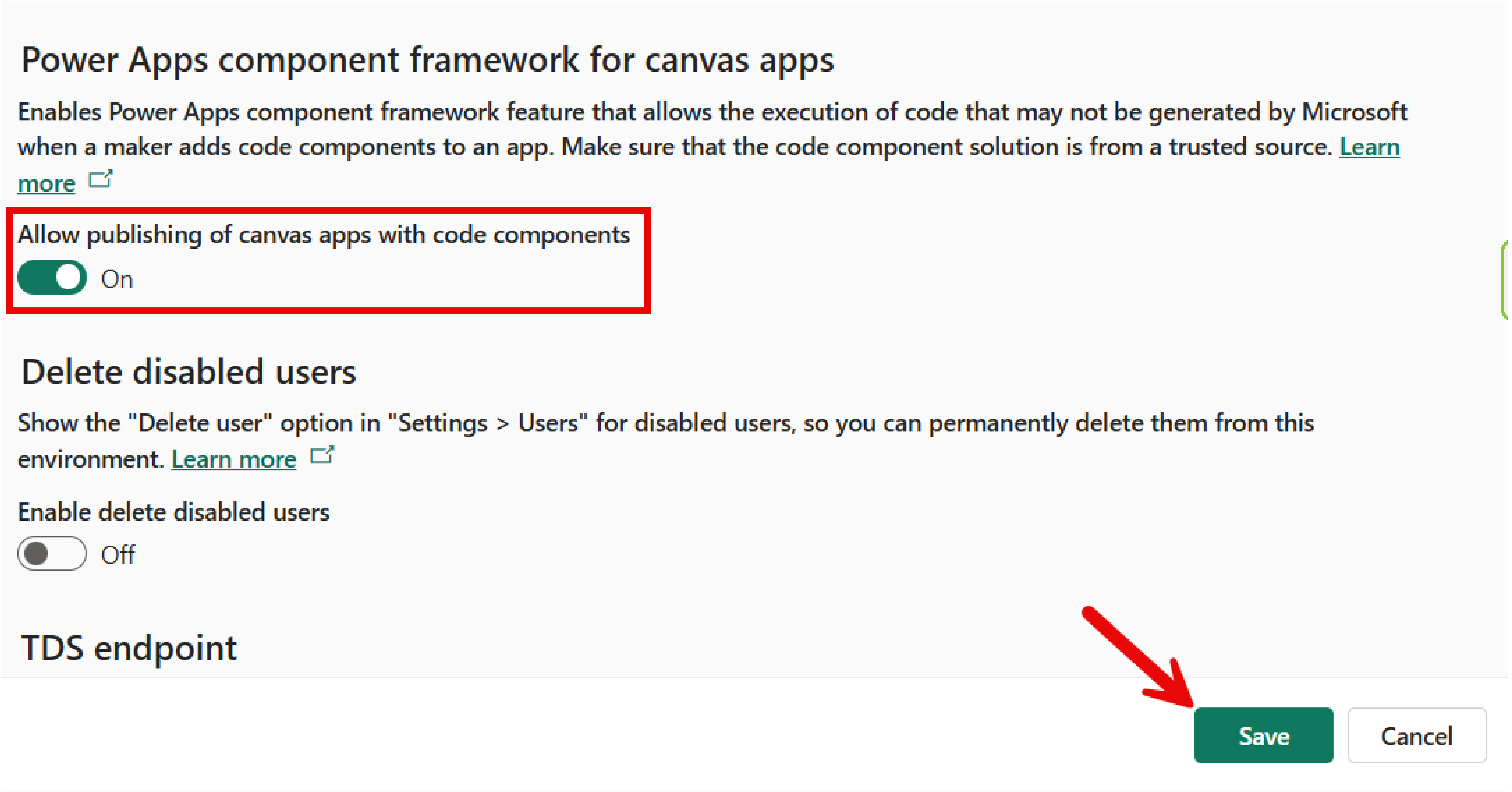
Break this down for me

What are PCF Code components?

1. Reusable rich user interface controls
2. Created by developers
3. Developed using HTML, CSS, and TypeScript
4. Packaged as solutions
5. Can be used in model-driven apps, canvas apps, custom pages and Power Pages sites

Check the defaults

Enable for canvas apps and custom pages



https://aka.ms/ppac

Why should I care?

Power Apps code component framework advantages

A few of the advantages that you are afforded as a result are:

- Access to a rich set of framework APIs that expose capabilities like component lifecycle management, contextual data, and metadata
- Support of client frameworks such as React and AngularJS
- Seamless server access through Web API, utility and data formatting methods, device features like camera, location, and microphone, in addition to easy-to-invoke UX elements like dialogs, lookups, and full-page rendering
- Optimization for performance
- Reusability
- Use of responsive web design principles to provide an optimal viewing and interaction experience for any screen size, device, or orientation
- Ability to bundle all files into a single solution file

Choose your type

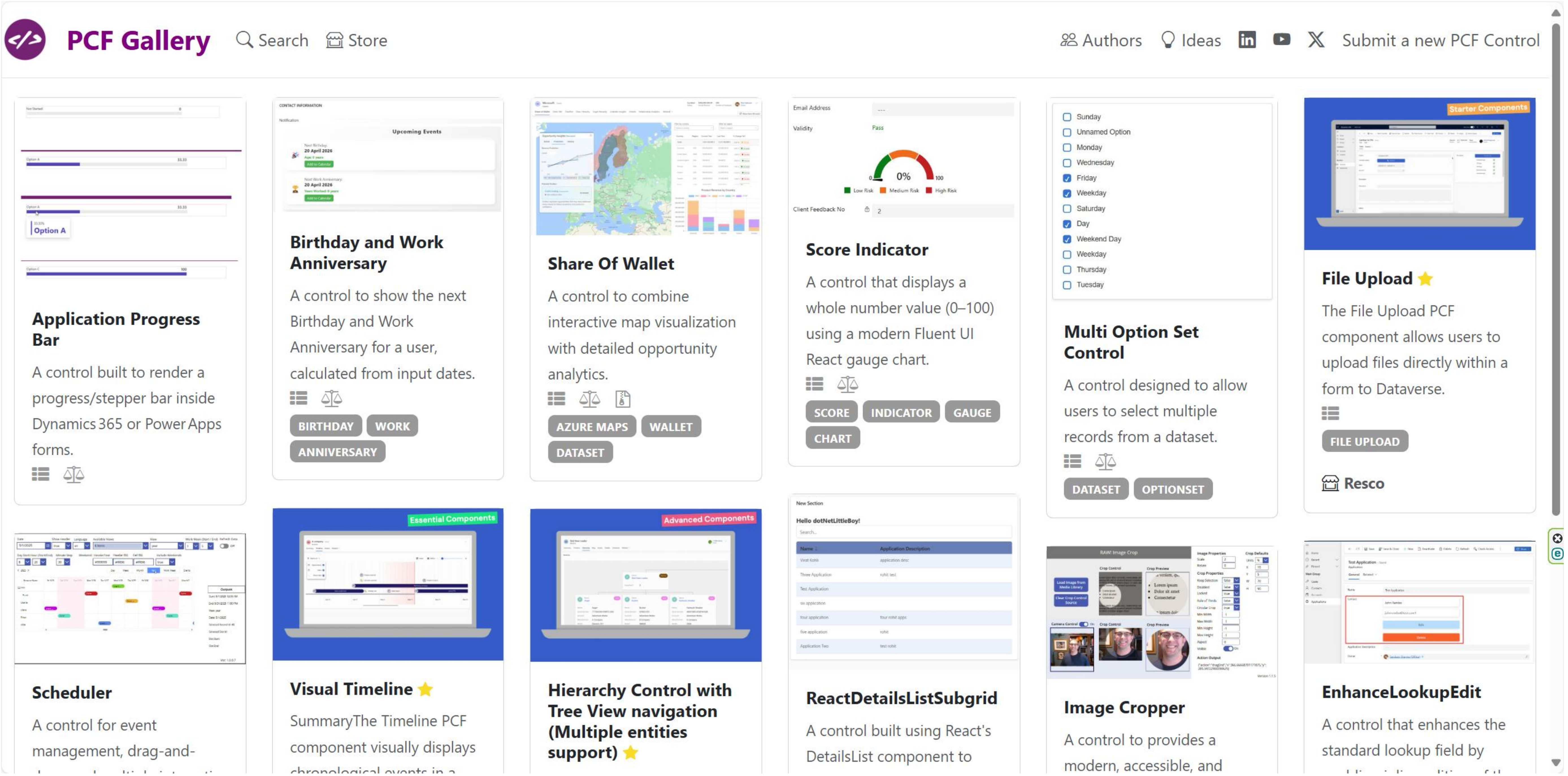
Types of components that you can add

1. **Field:** A custom control for a column on a form.
2. **Dataset:** A custom grid (or subgrid) to display data in a tabular format.
3. A component that displays content from external services

DEMO: Creating and deploying a simple code component

If only there was a community for this...

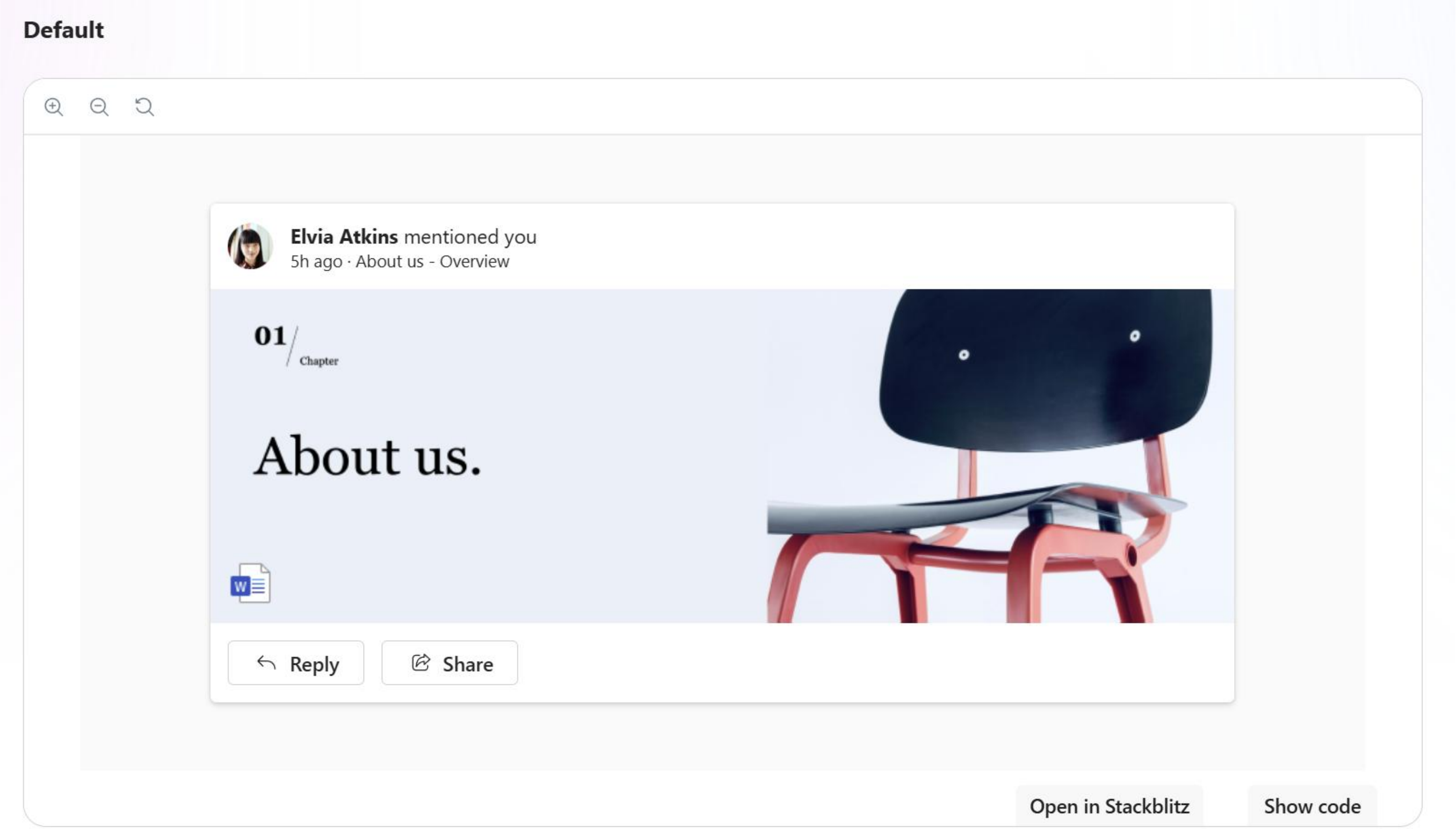
PCF Gallery



https://pcf.gallery

Make your controls super fancy

Fluent UI + React + PCF Controls = Better Together 🤝



https://react.fluentui.dev/?path=/docs/concepts-introduction--docs

Help me, I'm indecisive

JavaScript versus TypeScript versus React

JavaScript	TypeScript	React
Used for client scripting	Used for client scripting and PCF control development	PCF control development only
Not strongly typed	Strongly typed	Strongly typed
Prone to runtime errors; additional testing / debugging often needed	Enforces compile time checks, reducing common errors.	Enforces compile time checks, reducing common errors.
Minimal support for ALM	Supports more complex ALM scenarios	Only contributes in ALM when used in PCF control development
Easy to learn and understand	More complex to learn and master	More complex to learn and master

Lab 1 – CREATE MODEL-DRIVEN POWER APP

In this lab, you will do the following:

- Create a new table called Purchase Order and customise it accordingly.
- Add in the Account and Contact tables to your solution and customise accordingly.
- Create and define the configuration for your model-driven app, before testing it further.
- Add some test records into the system.
- Configure the integration with the backend ERP API.
- Create a cloud flow to generate purchase order request documents, and have this available to trigger from the model-driven app.

Expected completion time: 60 minutes

API Key - Please check the spreadsheet: https://bit.ly/BeautifulPowerApps-Credentials

DEMO: Power Apps types

Module 3: Working with Custom Pages

Model-driven apps + Custom Pages

Increase user experience and **business productivity**

Bridge functional gaps
Merging **structure**, custom logic and design

Extending standard capabilities
Connect to external data sources and access **organisational data**

User-centric development
Display as a side pane, full page or a dialog

Fusing low-code and pro-code
Developed using Power Fx – Enabled by JavaScript – Supports custom code components (PCFs)

It’s all about the use cases

Common scenarios where custom pages are **excellent**

🧭 **Landing zones**
Main screen and dashboard

✨ **Custom UI for business processes**
View PDFs, start approval processes and display record information

🔗 **Integrations**
Connect to other data sources than Dataverse. 1400+ connectors.
Trigger Power Automate.

👉 **Routing**
Optimize the user journey. Open a specific form, redirect to another page and launch websites

Custom Pages Complexity

Cool designs might meet high difficulty

Unexpected behaviour
"It doesn't look like that for me"

Use solutions, be careful with preview features and test *a lot*. DateTime values, language & disappearing controls

Build for performance
Life is too short to wait for unsorted data

Use low-code solutions where low-code is **GREAT!**

Increased difficulty
Adding another layer of complexity with **JavaScript**

Maintenance, web resources and new requirements

User authentication & permissions
Custom pages utilizes a shared connection infrastructure

User consent (connector authentication) spans across **EVERY** custom page in the MDA.

Data – lazy loading 😴

You don't need 1000 records in the app at the same time...

⚠️ **Delegation**
What ever that means

- Off load data processing tasks to the server side
- **Large** datasets – more than **500-2000** records
- Data accuracy – use Filter(), Sort(), StartsWith()
- Locally processed – Collect(), ClearCollect(), Search()

Design considerations

One size **doesn't fit all**

Build responsive apps

Your end-users will thank you

- Utilise containers to align and group controls
- Automatic **resizing** of screen elements
- Parent.Height & Parent.Width
- X, Y, Height and Width – Position and dynamic values

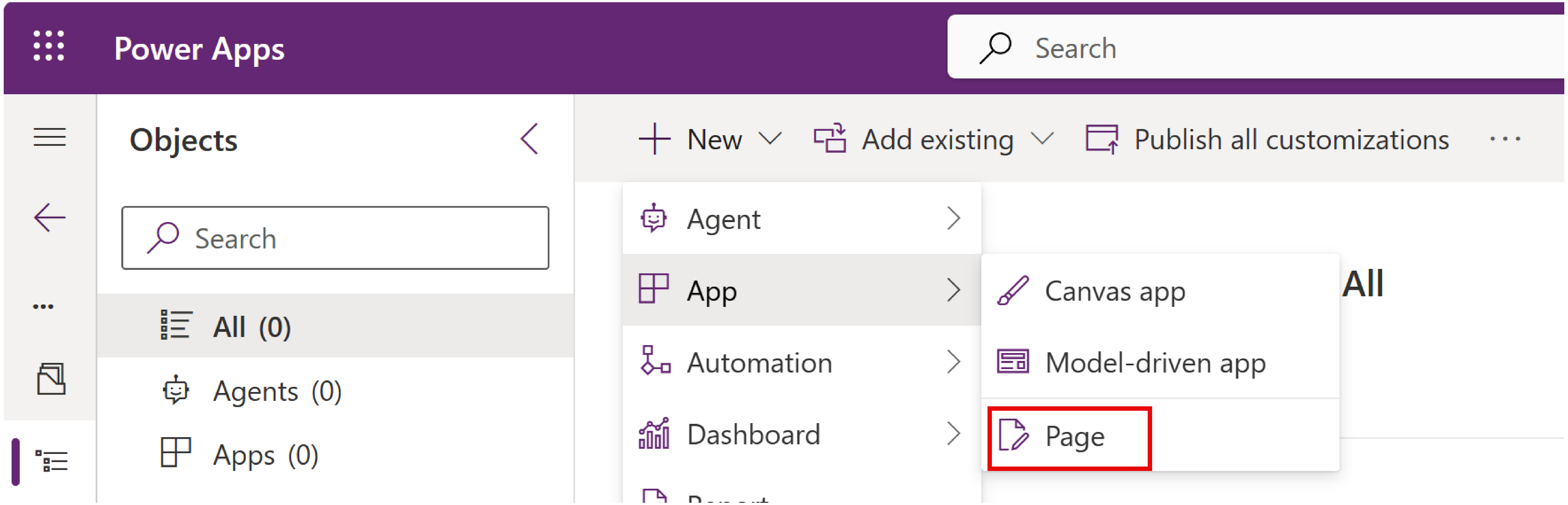
Consistency

Make it look sexy

- Reuse components and YAML
- Color scheme, fonts and company design
- Not every control is **supported** (i.e PDF viewer, barcode scanning, attaching files and taking pictures)

Show me it in action!

Creating a custom page

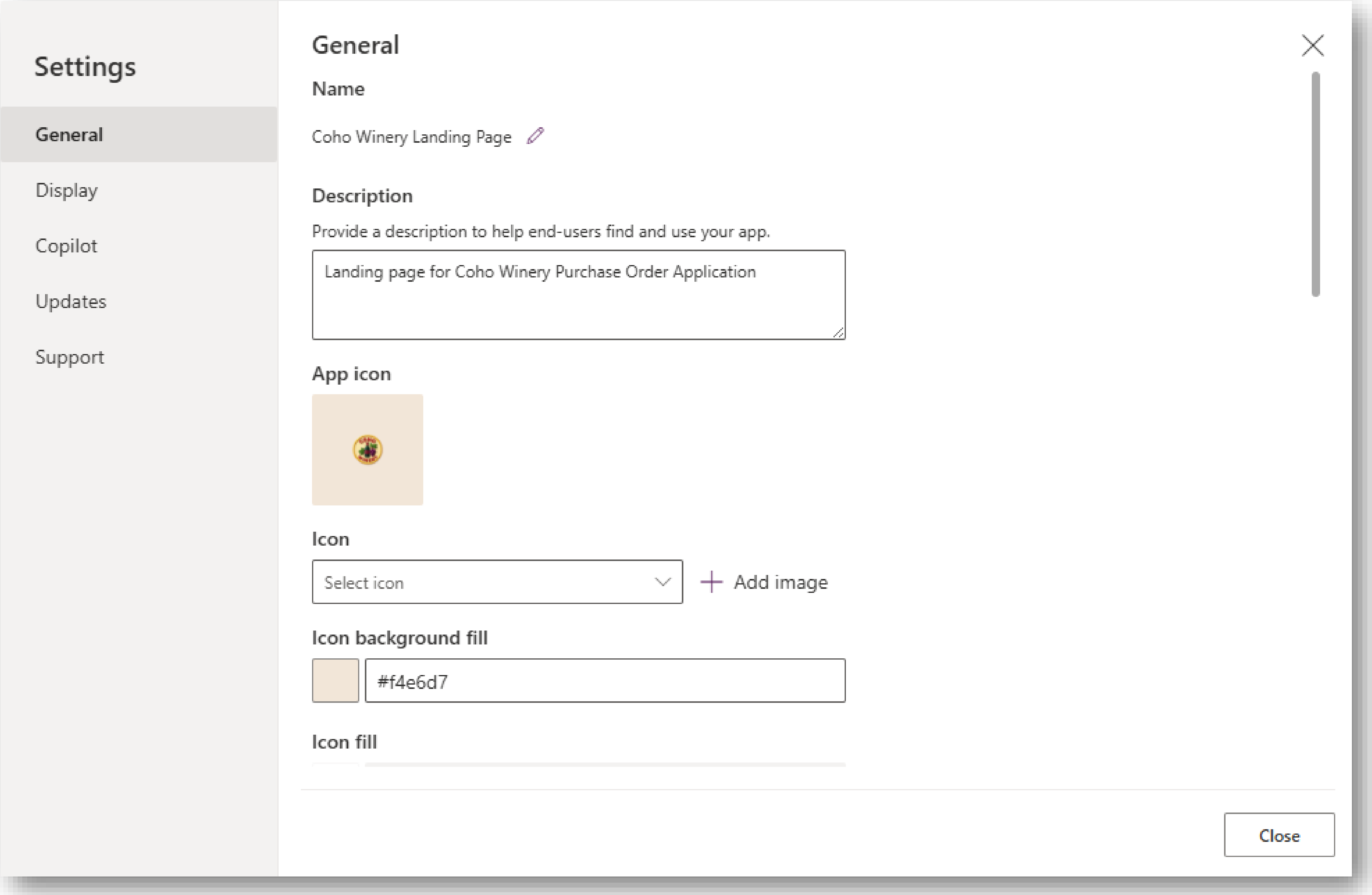


- Multiple ways to create a custom page
  - Solution
  - Model-driven app designer

Soooo many options

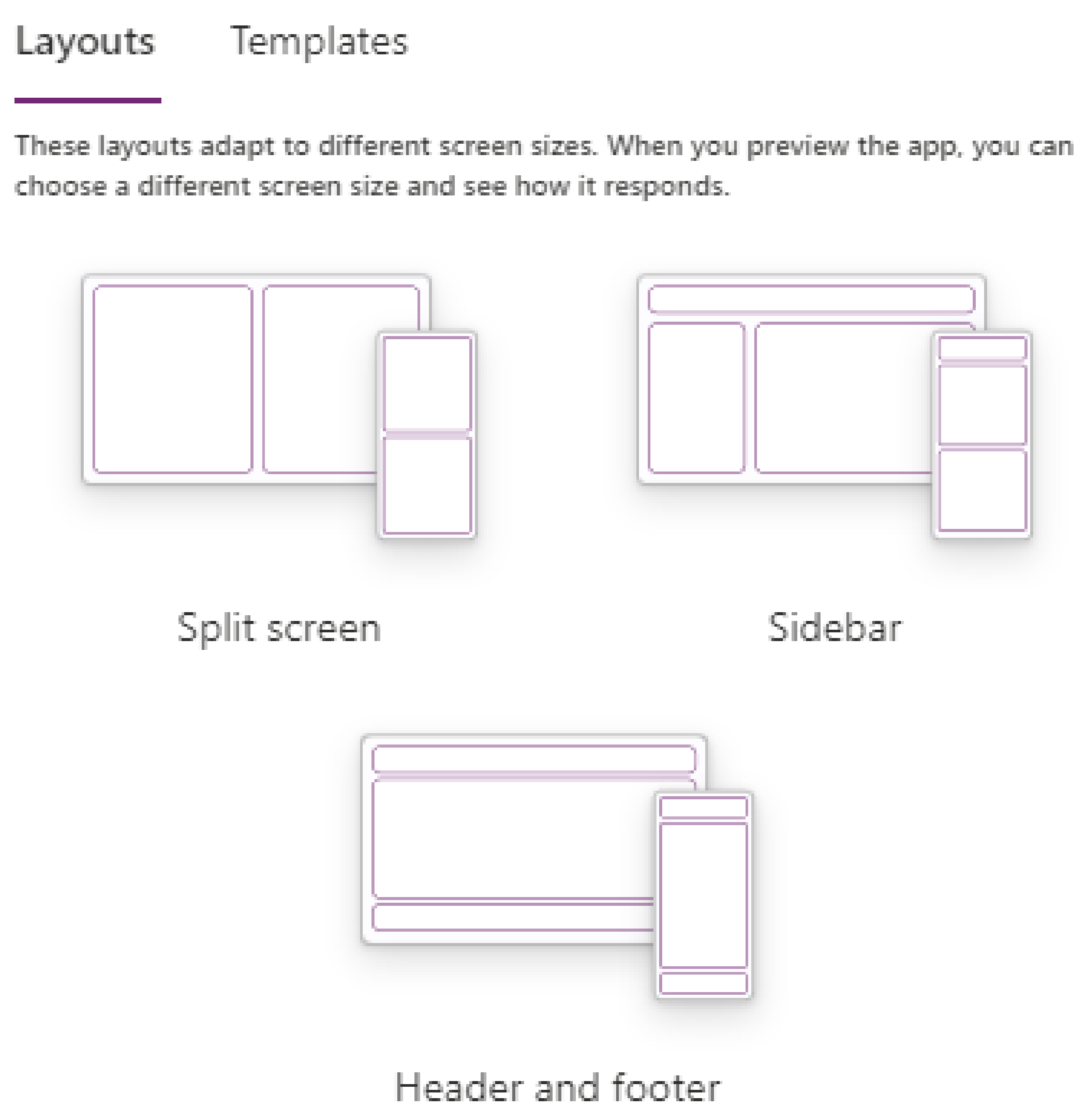
Settings and configuration

- Name your page
- Provide a description
- Set an icon or image (brand logo)
- Set background fill
- This is what shows up when the page is loading



Just tell me how to do it!

Designing a custom page

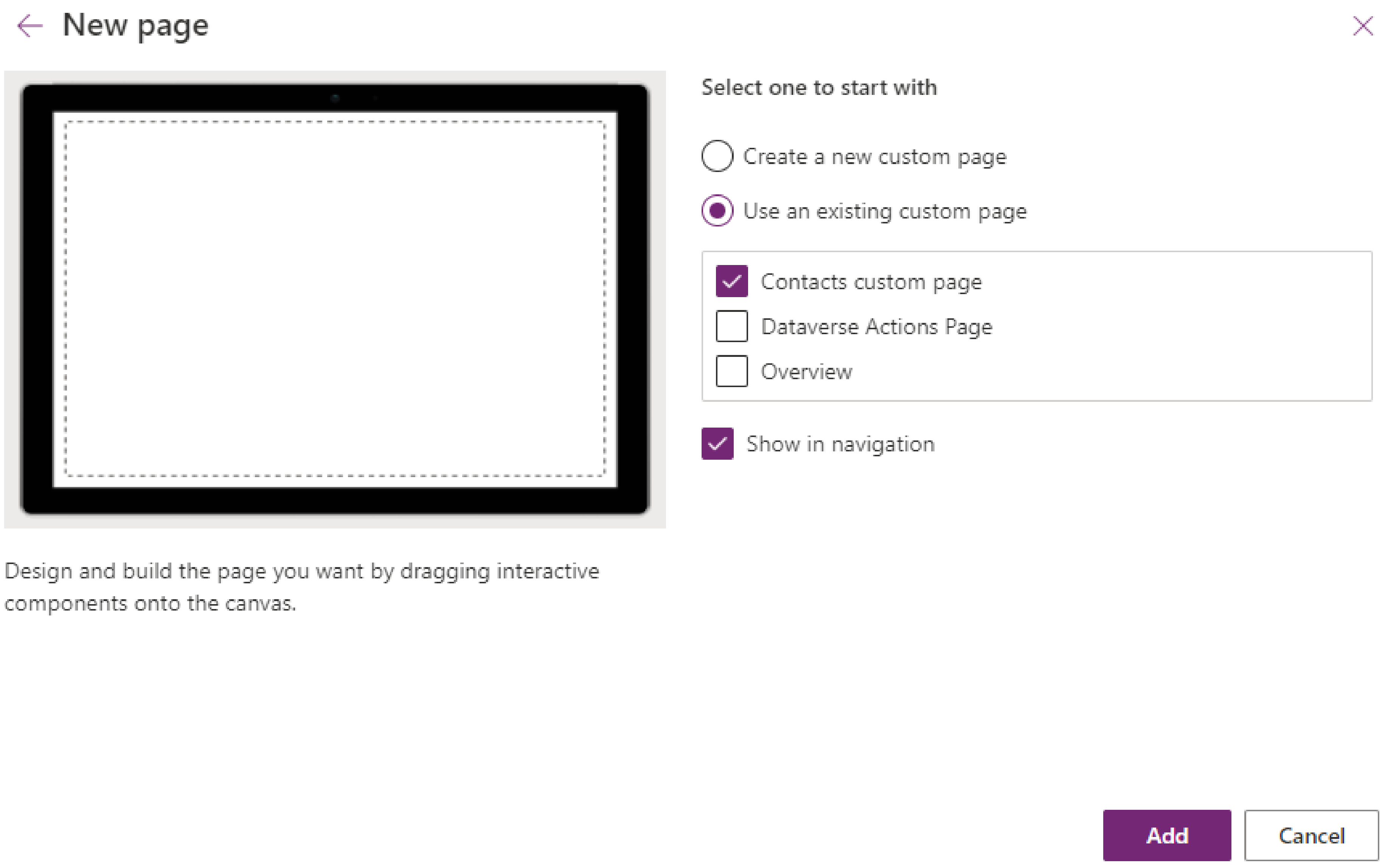


- Responsive design with containers
  - Vertical containers
  - Horizontal containers
- Set properties of containers to handle flexible widths and heights.
- Layouts available when creating a custom page.
- Subset of canvas controls supported.
- Theme set to match theme of model-driven app.

Custom Page embedded

Adding a custom page to an app

- Add custom page
- Add to navigation
- Edit subarea to set the title used in navigation and privileges to access the page
- Publish the app

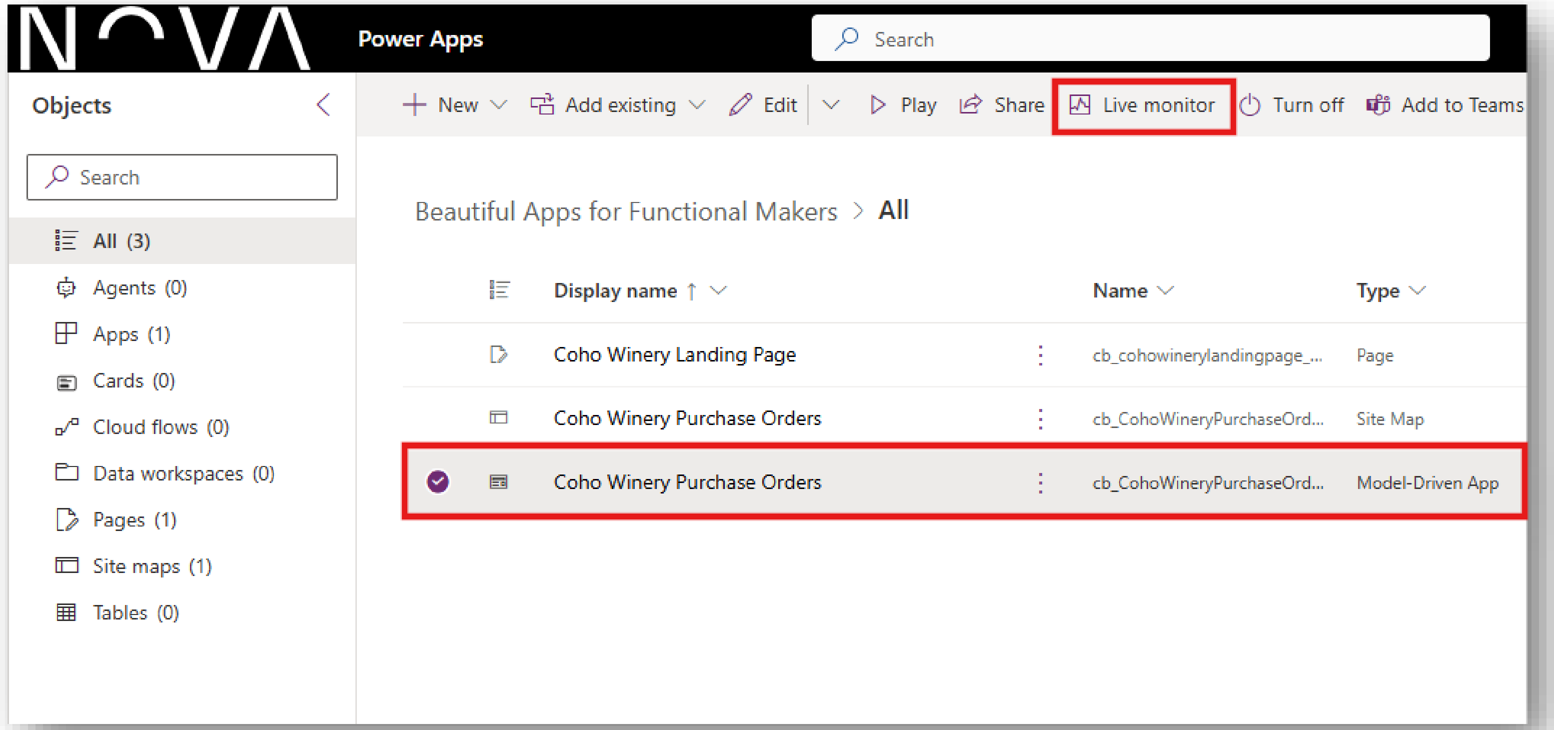


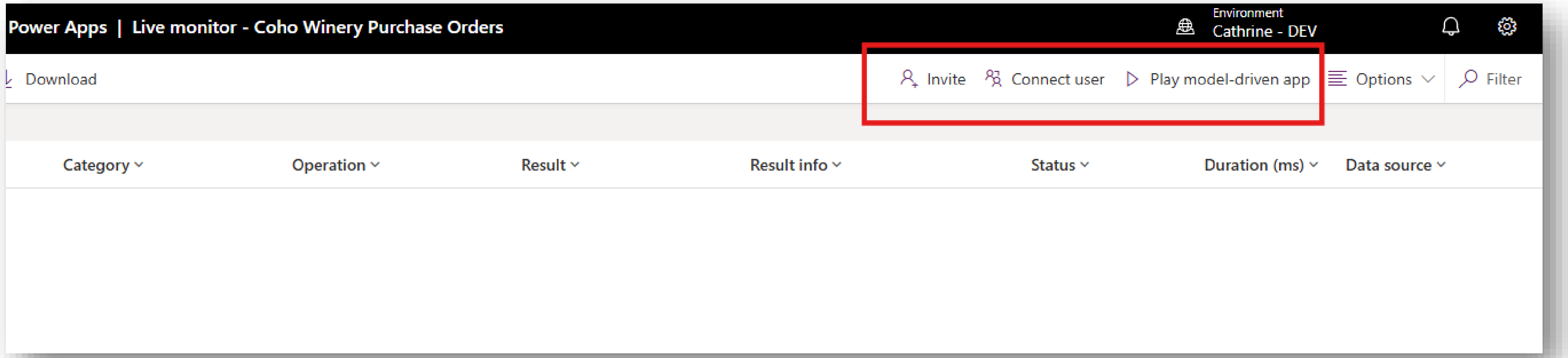
DEMO: Getting started with Custom Pages

I See You

Monitoring

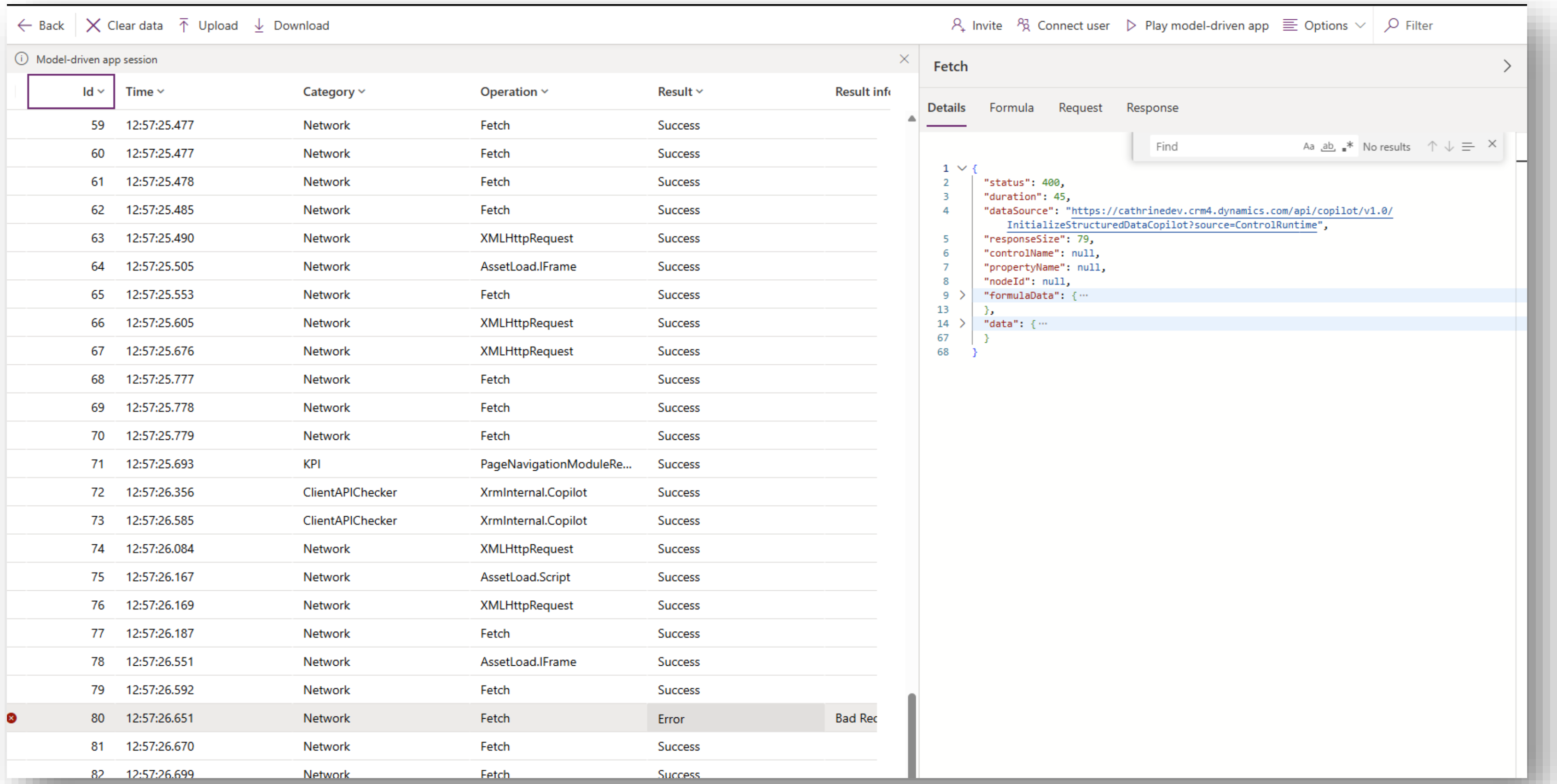
- In the solution, select the MDA and Live monitor
- The monitor will open. Choose to play the app or connect another user to see their details





It’s an infestation 🐜

Debug and improve



- In the **Live monitor** everything the user selects will show up
- Use the information to improve the performance and issues in the app

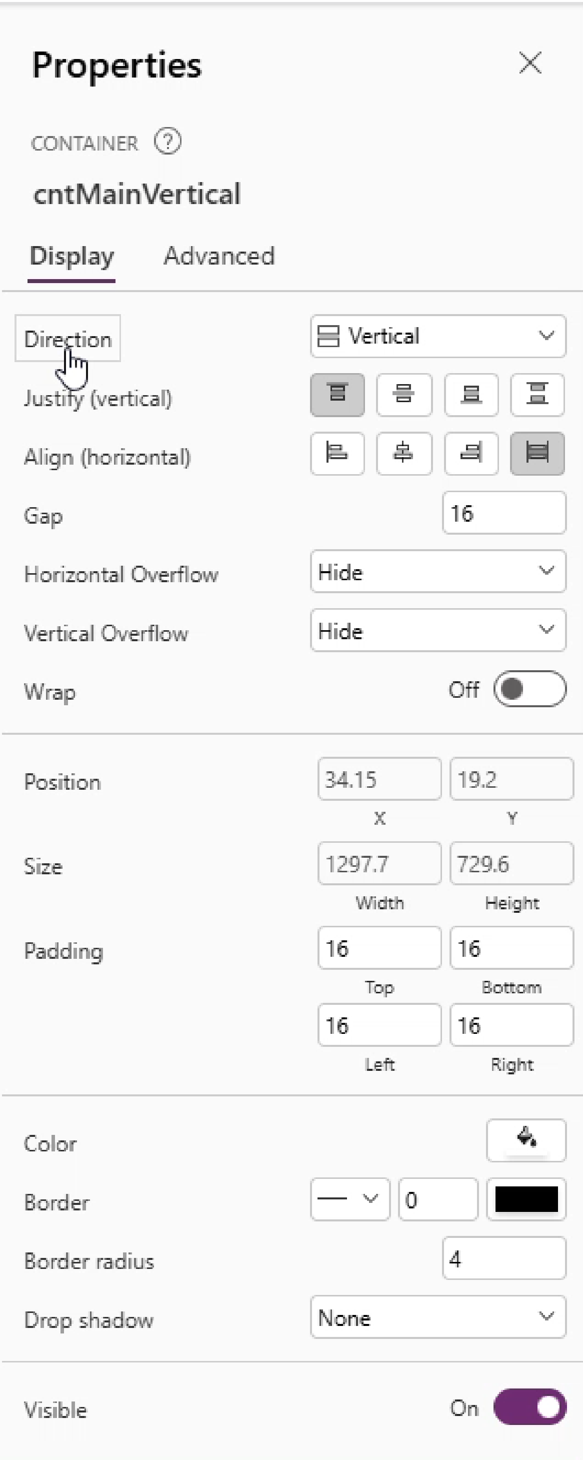
Please contain your excitement

Container properties

- Properties that will take you some time to get used to and master.
- Make sure to test, set values, and play app with different sizes
- Control alignment in containers; no manual control, strict rules for item placement
- Setting the parameters for adjustments and dynamic values, not hardcoded; avoid using drag and drop

Lemme give you a tip there

Container tips



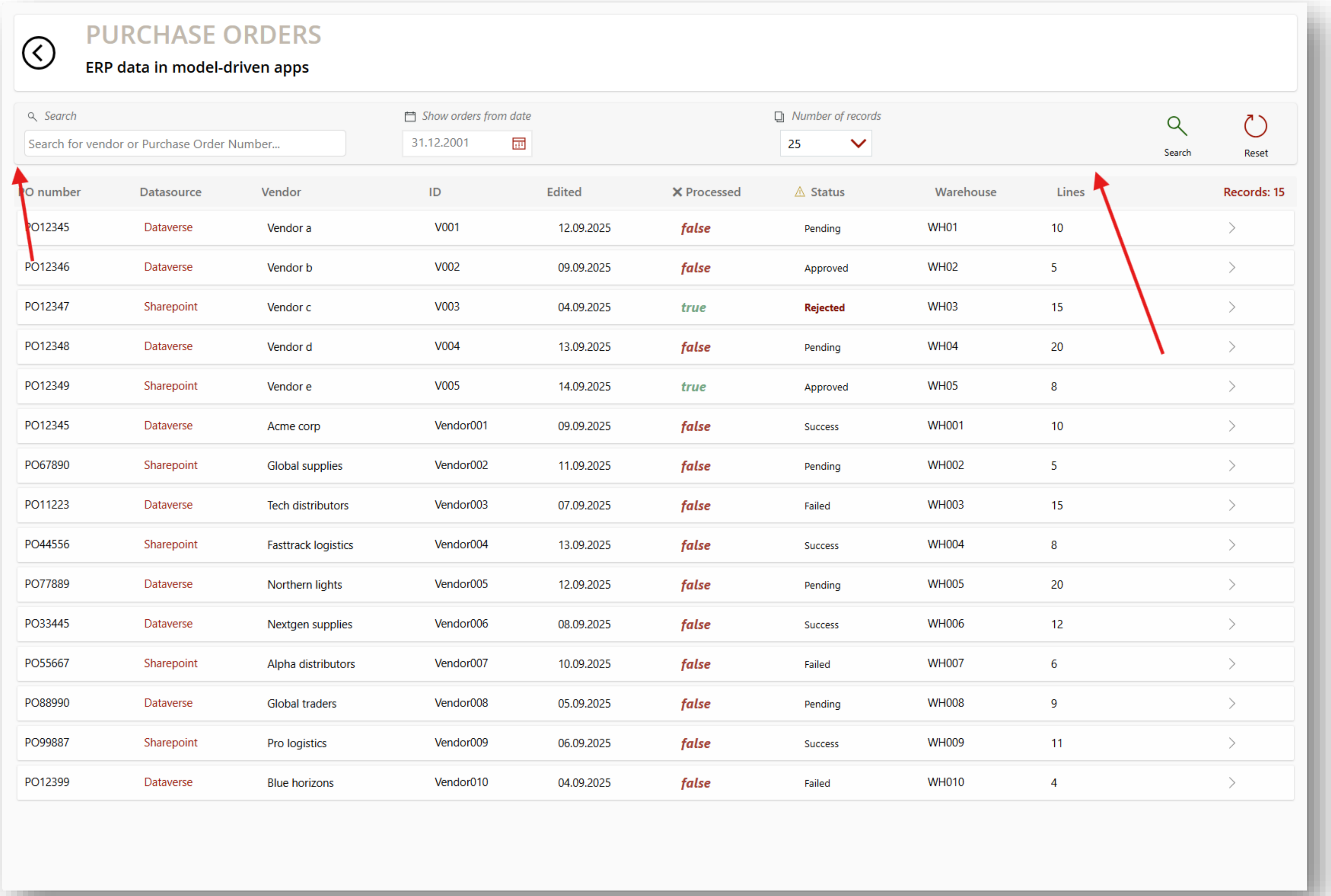
- If you need the container to expand with dynamic text, turn **Flexible height = On** and keep **Vertical Overflow = Hide**.
- If you see unexpected scrollbars, check **Wrap** first; wrapping usually removes the need for horizontal scrolling – Text shows as cath.... instead of cath

  rine
- To get equal column widths, give sibling containers **Fill portions = 1** (or 2, 3...) and set **Align in container = Stretch**.

Less is more, people 😳

Styling containers

- **Fill (Color – Fill)**
  - Background color of the container – can be transparent if you have a background color
- **Border Color / Border Thickness / Border style**
  - Styling properties for borders – try and fail. Usually invisible for main container, but can add depth and highlight content if needed
- **Border Radius**
  - Border roundness of corners – often changed border radius between 4-10
- **Drop Shadow**
  - Shadow intensity – Light is my go-to



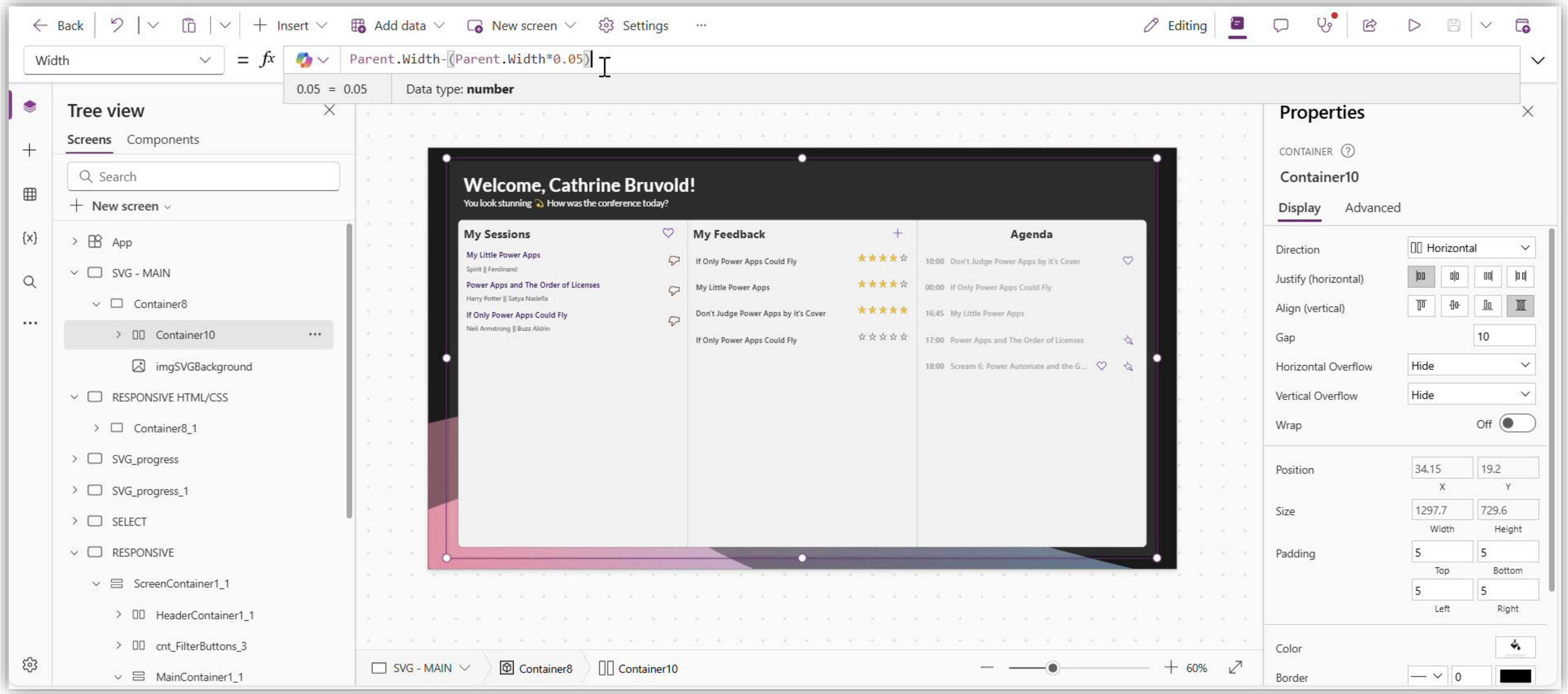
Building From Scratch

Takes foreverrrrrrr so reuse it

Container within container within container - Horizontal, vertical and container

App Width and **Height** are the two properties corresponding to the size of the screen.

Dynamically set width, height and position of controls based on percentage, the Parent./Self. Functions and formulas.



Pivot. PIVOT!!!

Positioning and sizing containers

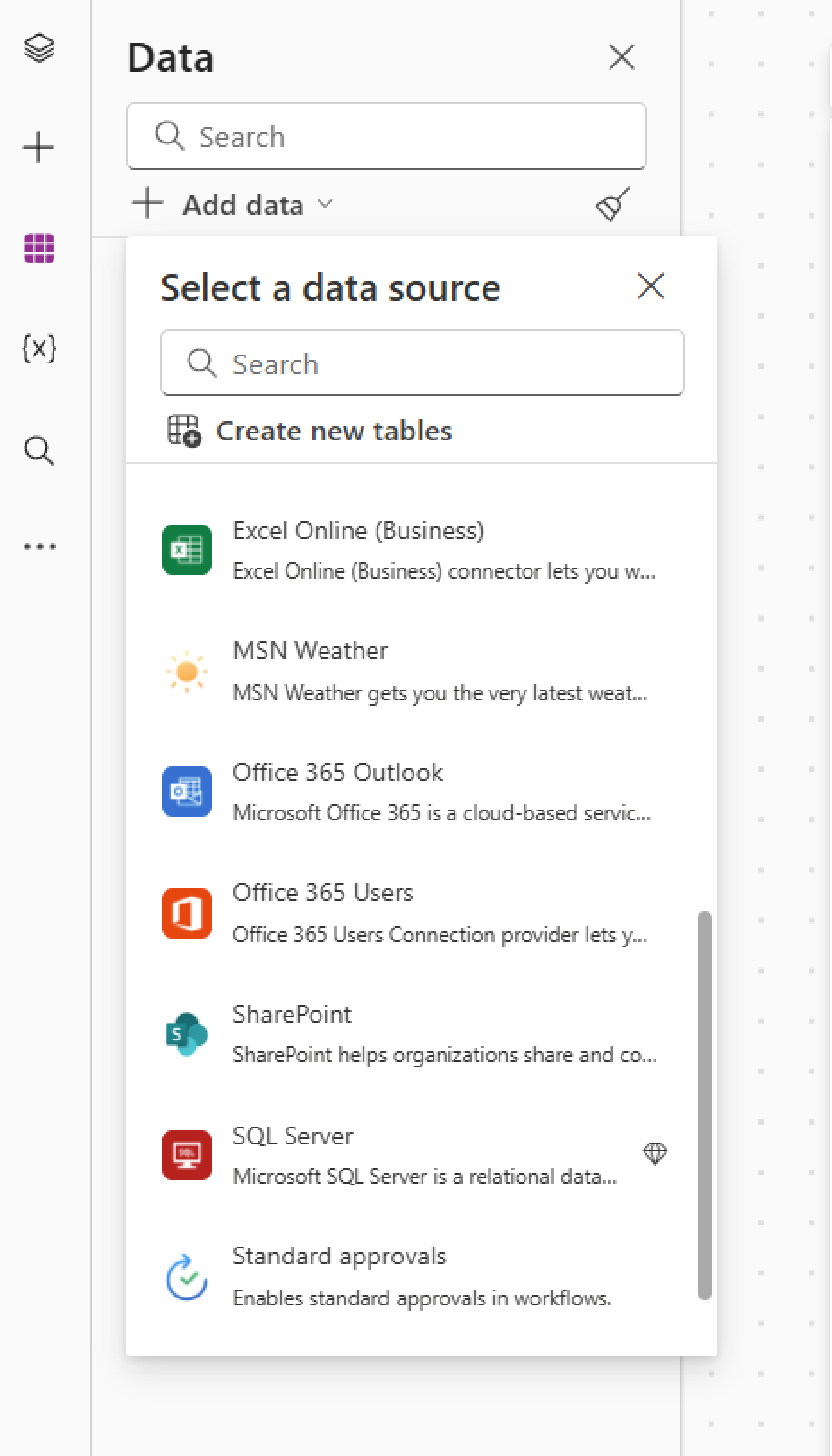
- **Layering and dynamic values**
  - Use **Parent.** Values to automatically scale controls relative to the parent object
- **X, Y, Height and Width**
  - Main properties of controls used to dynamically size elements and position them. When the numbers are grey, a formula is used to set the values. DO NOT MANUALLY ADJUST 😉
- **Flexible height**
  - **On** when you have dynamic text values (will ignore Height/Width property), **Off** when you want to control the size. For instance, main containers that should have a set height.

After you create formulas, be careful not to adjust the elements manually.

Power Fx (center align)
X: (Parent.Width - Self.Width) / 2
Y: (Parent.Height - Self.Height) / 2
Width: Parent.Width-(Parent.Width*0.05)
Height: Parent.Height-(Parent.Height*0.05)

Connect to 1,400+ systems? Why not?

Working with connectors in custom pages



- Full access to both tabular and action-based connectors.
- Consider how data sources are shared and potentially accessed by users
  - Security
  - Fix connections
- Recommendations:
  - Limit number of connectors to 10

Named Formulas

Write reusable code once

Read-only global variables

Automatic recalculation of values.
No timing-dependency or manual variable update required.

Code definition

App.Formulas: define formula name and logic to be executed as nfFormulaName = expression;
Reference the **Named Formula** instead of code duplication

✅ Benefits

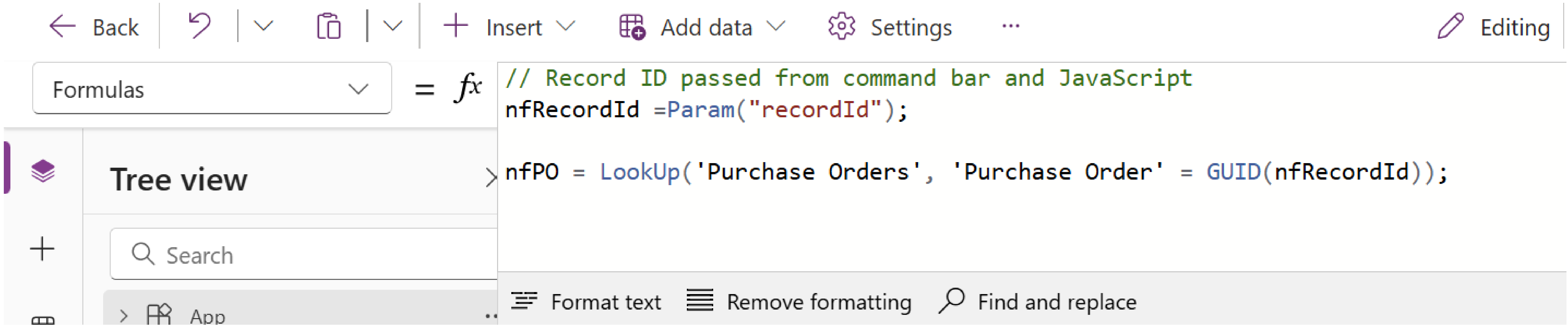
Define once and reuse, avoid redundant code and utilise lazy-loading principles

Named Formulas

Reduce the amount and **repetition** of code

Setting the Named Formula on **App.Formulas** containing the code you want to execute

Calling the Named Formula instead of the entire syntax every time



Syntax and reference

Named Formulas

App.Formulas

NAME DEFINITION

EXPRESSION

```
nfNamedFormula =
ShowColumns(
  LookUp(
       'Data source',
       'Column name' =
SetVariableValue
  ),
  'Field 1',
  'Field 2',
  'Field 3'
  );
```

Control property OnSelect

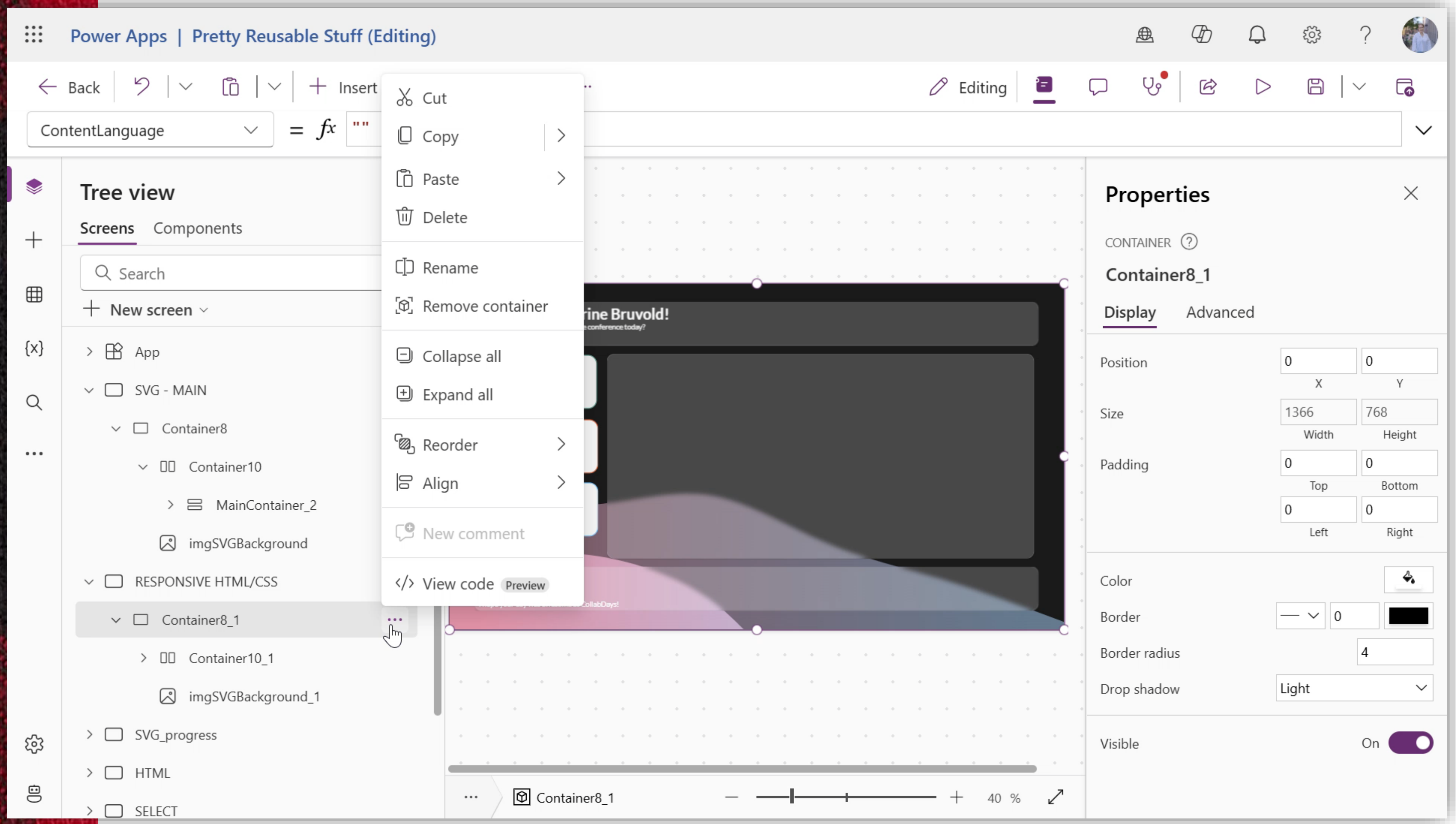
REFERENCE

✅

```
ClearCollect(
    colCollectionName,
    nfNamedFormula
)
```

🚫

```
ClearCollect(colCollectionName,
ShowColumns(
  LookUp(
       'Data source',
       'Column name' = SetVariableValue
  ),
  'Field 1',
  'Field 2',
  'Field 3'
  ))
```



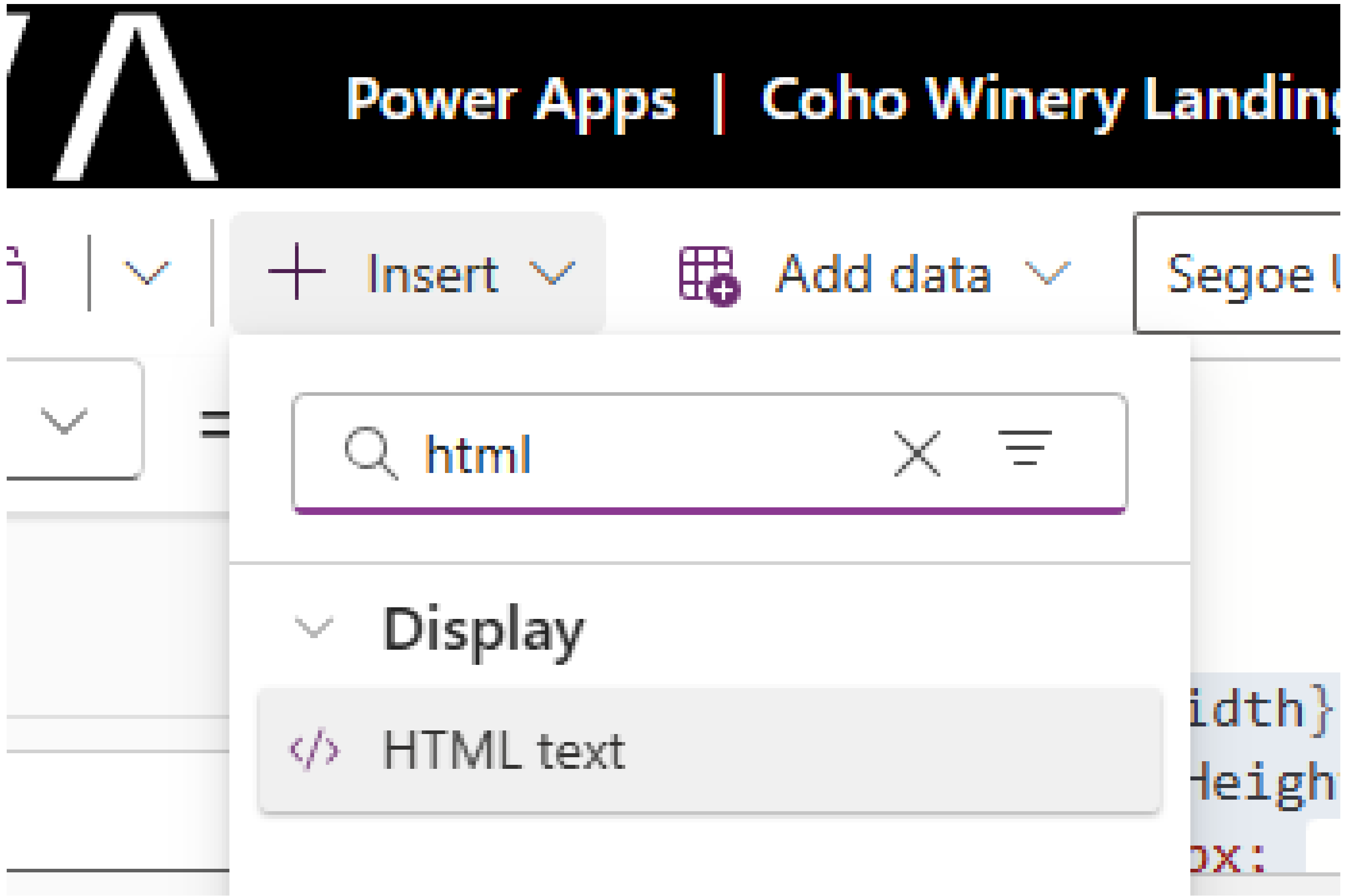
Canvas YAML

- Store and reuse YAML snippets to build apps faster
- Copy and paste YAML code from canvas apps controls and containers
- Use YAML to investigate the code and get a full overview of functions used, naming of controls and where the are located
- Increase reusability and save your favorite components in Visual Studio Code

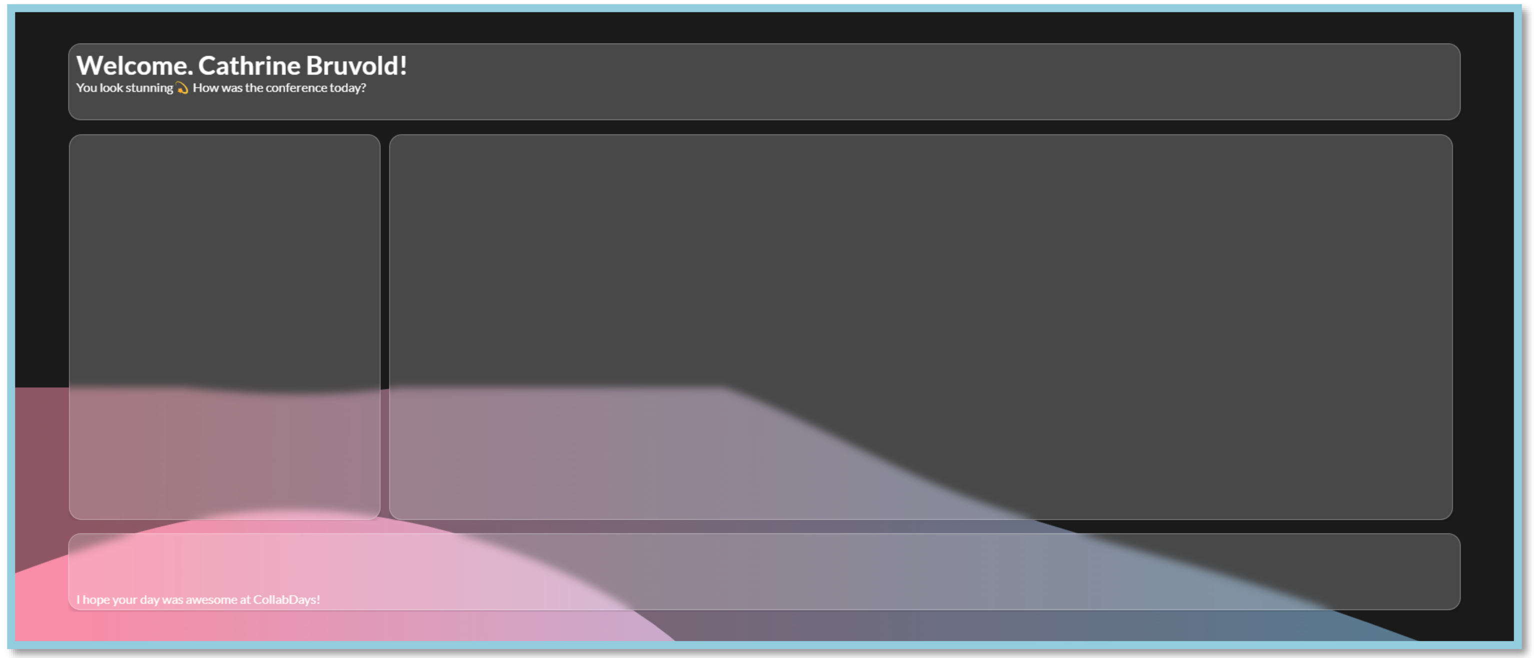
App makeover time 💅

HTML in Power Apps

- Build apps with more depth, customised design and advanced UI
- Use the HTML text control
- Add blur, colours, and custom “code”



Glass morphism



Example of HTML/CSS code snippet used to create Glass morphism features

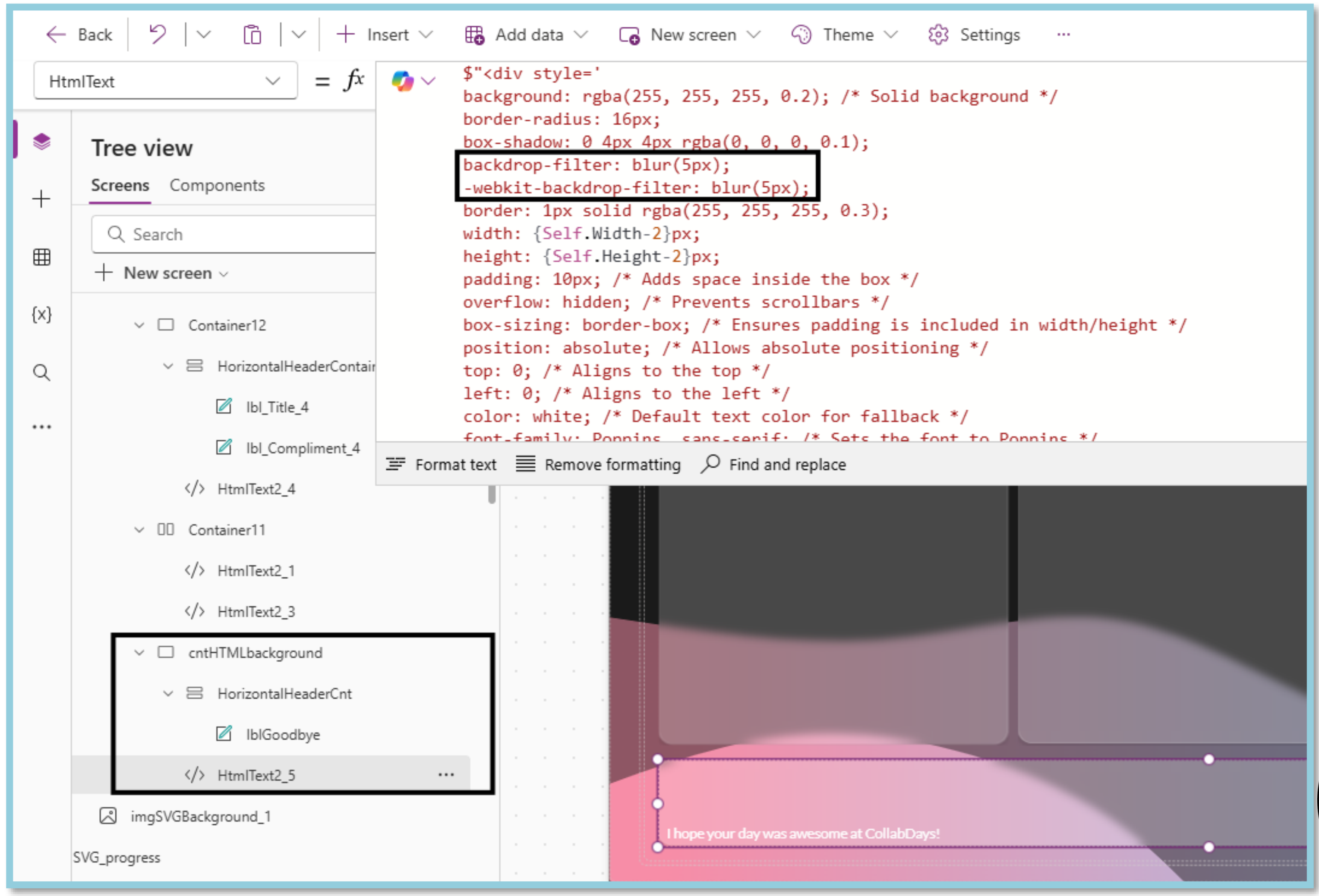
Create simple, yet effective depth to your canvas apps

Add containers to hold the HTML TEXT as a background, adjust width and height to parent.Width & parent.Height (responsive).

Add Horizontal or vertical containers above the HTML TEXT control

CSS generator: https://css.glass/

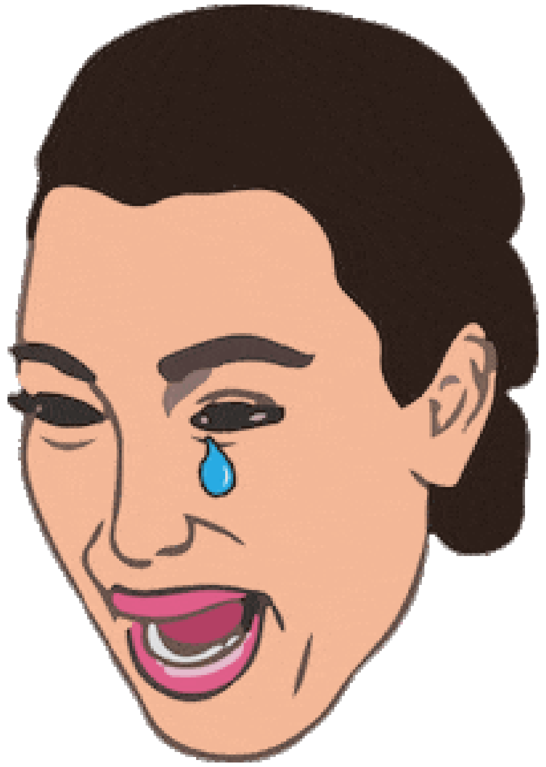
Get inspiration from kristine Kolodziejski's YouTube



Demo: Advanced custom page development

Custom page with external data

Both pretty aaaaaand functional



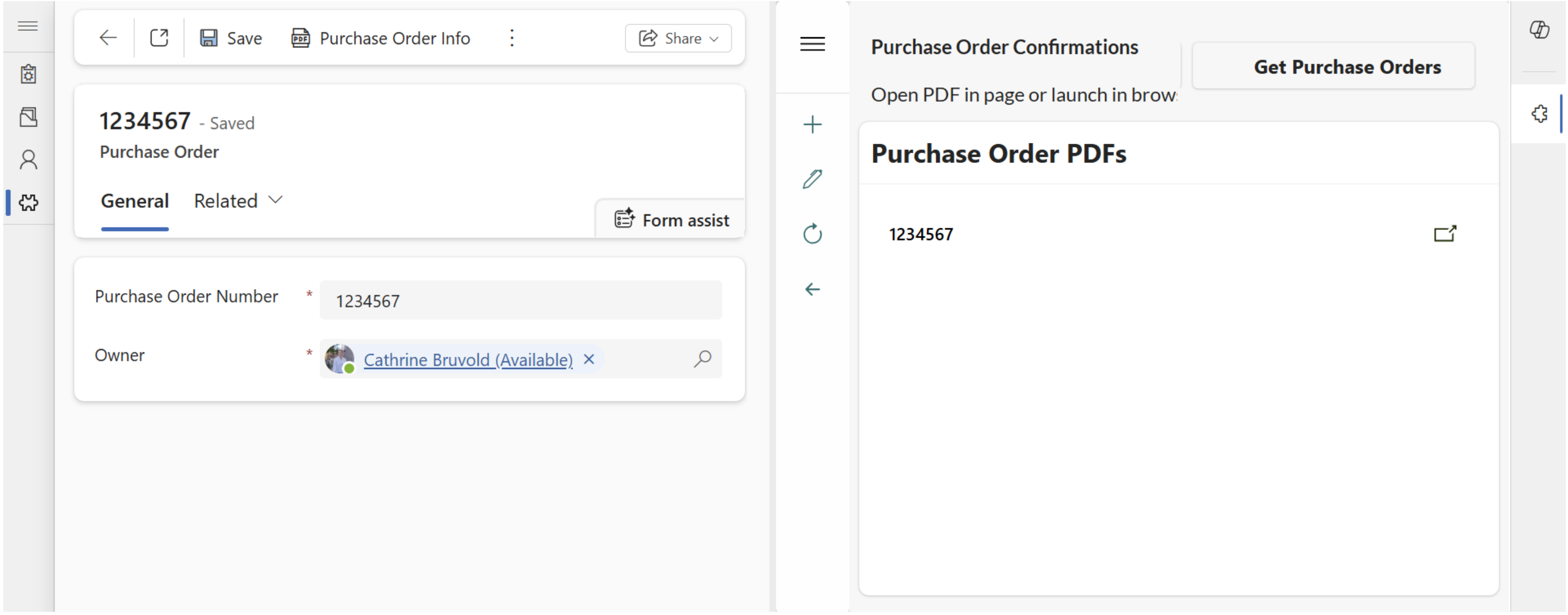
"I just wanted to add some shading to my buttons.
Now I'm debugging regex validation rules and syncing data from an on-prem ERP system"

Custom page in MDA

Custom pages – Example Scenarios

Custom dialogs:

- Ribbon Commands
- Form events (**OnChange**, **OnLoad** & **OnSave**)
- Side pane, dialog or full page



What sorcery is this???

Using JavaScript for custom pages

1. Start building

Create a solution for the custom page, model-driven app and the table where the page will be triggered from.
Save the **logical name** of the custom page.

2. Script

Create a .js file with a function for triggering the custom page Dialog/Side Pane. Use the **logical name** of the custom page as a reference in the script as a parameter.

3. Configure the model-driven app

Add the .js file as a web resource in your solution. Open the **model-driven app designer**, add the custom page and configure the command bar on a Form or the Main grid to reference the web resource.

4. Test & iterate

Trigger the custom page and see what it looks like. Edit the custom page and JavaScript according to your needs (remember to publish)

Lab 2 – CREATE CUSTOM PAGES

In this lab, you will do the following:

- Build Custom Pages to be used as a full page, side pane and dialog in a Model-Driven App
- Connect to relevant data sources
- Fetch record details using the Param() function and connecting to the record context
- Develop responsive apps

Expected completion time: 60 minutes

Lab 3 – INTEGRATE CUSTOM PAGES IN MODEL-DRIVEN POWER APPS

In this lab, you will do the following:

- Embed the custom pages you built in Lab 2 into the model-driven app.
- Preparing the JavaScript web resource that will open the side pane from a command bar button on the Purchase Order form.
- Configure the command bar button to the Purchase Order form to open the side pane and pass the current record’s ID.
- (Optional) Add JavaScript to the **OnLoad** event of the Purchase Order form to automatically open the side pane when the form loads.

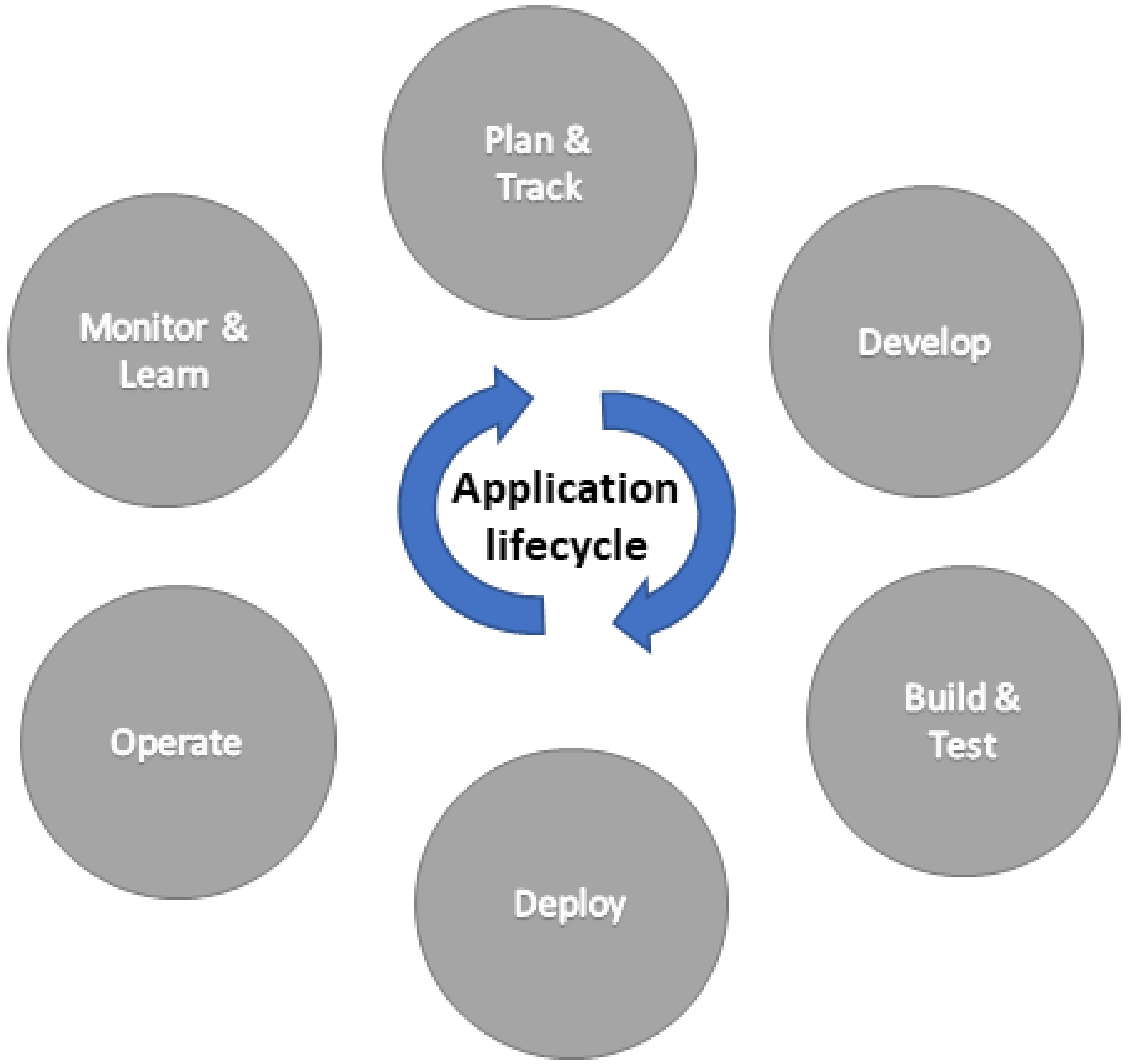
Expected completion time: 60 minutes

Module 4: Going Further - Advanced Concepts and Best Practices

Friday deployment – what could go wrong? 😏

What is Application Lifecycle Management (ALM)?

- The processes and tools to help you deploy apps, automations and other components:
  - Continuously
  - Safely
  - In a scalable fashion (for 5 or 5,000+ users)
  - Automated (optional)
- Helps make your solution fire, rather than putting production on fire 👀 🔥



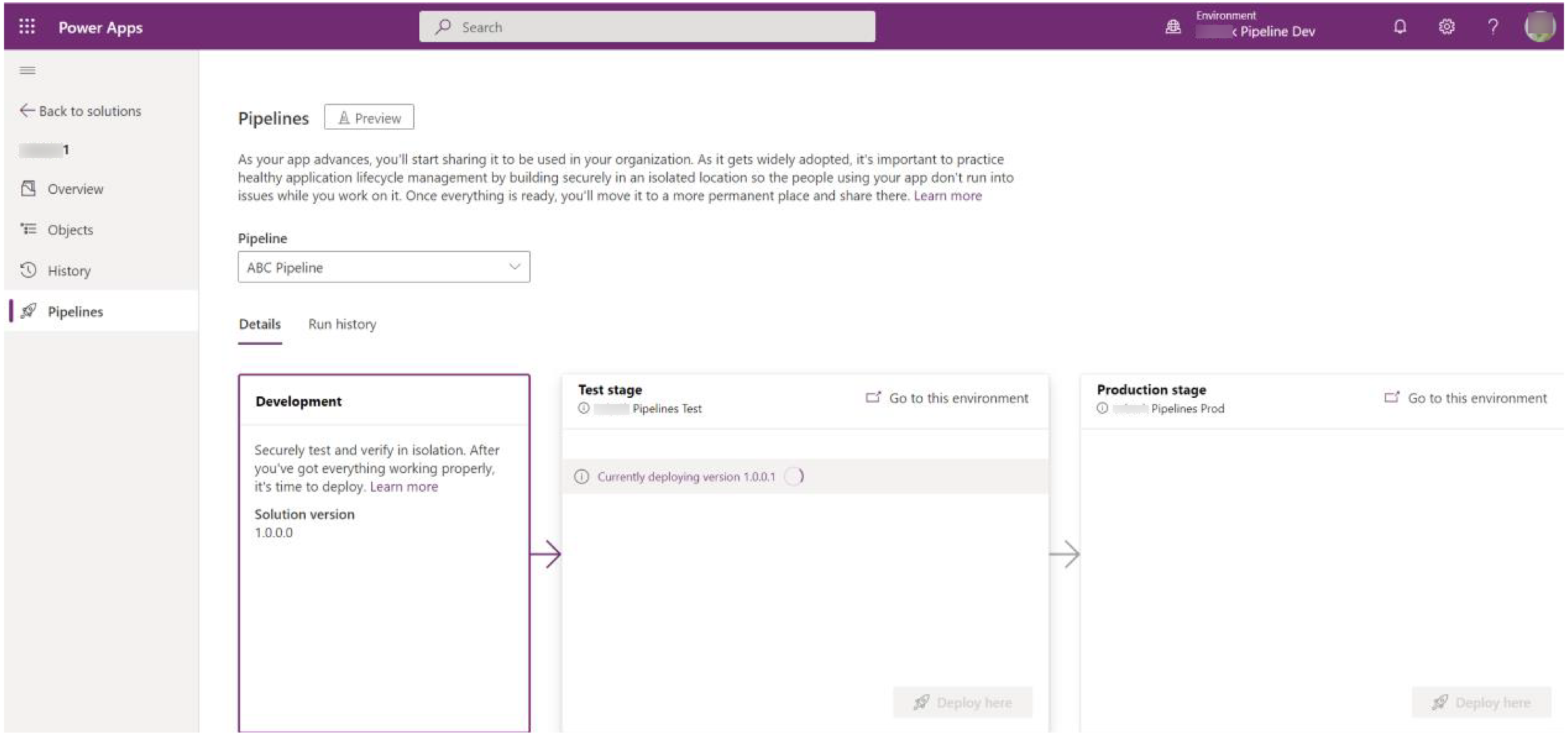
There’s a lot to think about…

Key concepts

- Source (version) control
    - Git repositories
- Solutions / Publishers
    - Managed versus unmanaged
- Environments
- Power Platform pipelines

Now you realise you should have gone to the ALM workshop

Power Platform pipelines



Time for the weigh up ⚖️

Power Platform pipelines

- **Advantages**
  - No external tooling required
  - Supports approval based delegated deployments
  - Multi-stage deployment processes
  - Generate deployment notes automatically using Copilot
  - Deploy immediately or based on a schedule.

- **Disadvantages**
  - Requires Managed Environments; all users accessing the environment must be licensed.
  - Not suitable for complex deployments:
    - Multiple solutions
    - Azure components
    - Configuration data

Demo: Power Platform pipelines

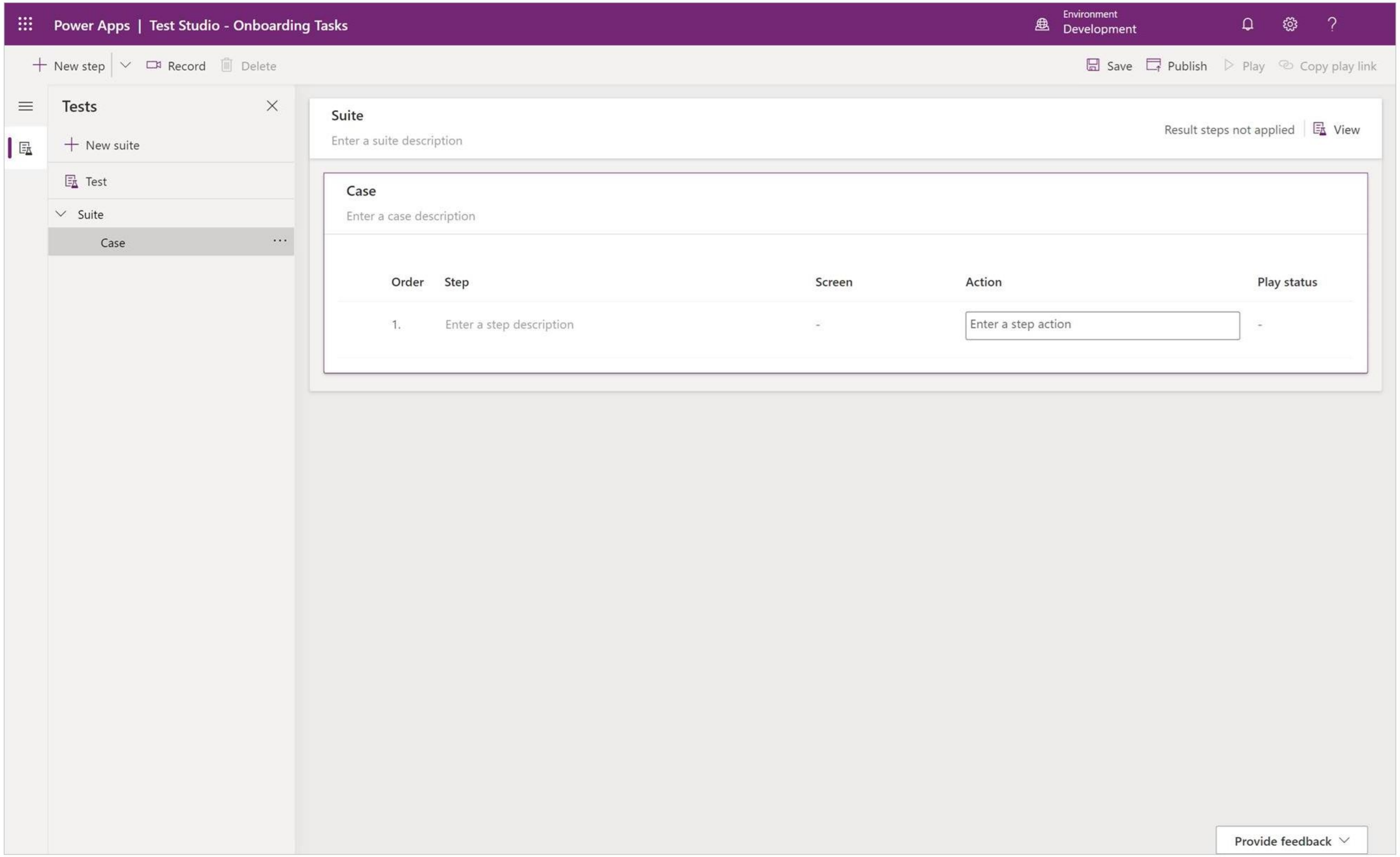
It's a tough job, but someone's got to do it

Testing, Debugging and Troubleshooting your apps

- Unit test your work!
  - Manually
  - Use Test Studio
- Next, perform User Acceptance Testing (UAT)
  - Configure a dedicated environment for this
- Use **Trace()** function in canvas apps and custom pages for additional debugging capability
- Once live:
  - Review analytic reports regularly
  - User having an issue? Jump onto Monitor with them!

Gotta have more studios

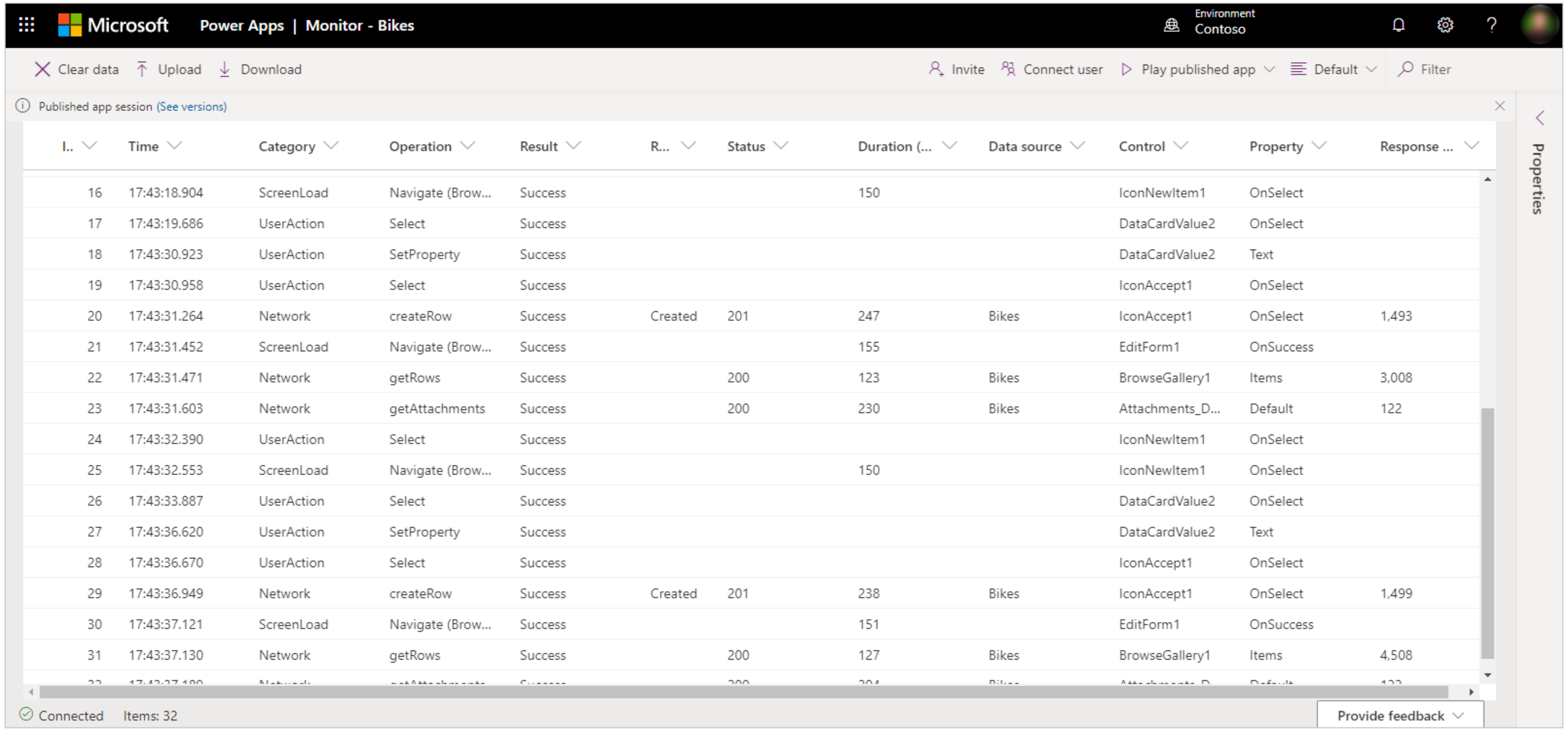
Test studio



Remote debugging FTW

Power Apps Monitor

Monitor is a tool that you can launch from Microsoft Power Apps Studio to help you troubleshoot problems and improve the quality of your apps

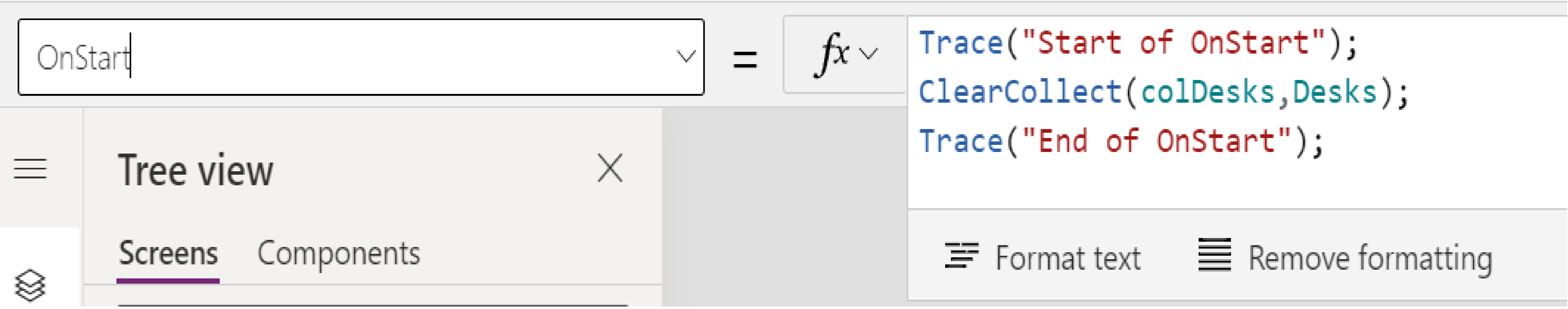


I..	Time	Category	Operation	Result	R...	Status	Duration (...	Data source	Control	Property	Response ...
16	17:43:18.904	ScreenLoad	Navigate (Brow...	Success			150		IconNewItem1	OnSelect	
17	17:43:19.686	UserAction	Select	Success					DataCardValue2	OnSelect	
18	17:43:30.923	UserAction	SetProperty	Success					DataCardValue2	Text	
19	17:43:30.958	UserAction	Select	Success					IconAccept1	OnSelect	
20	17:43:31.264	Network	createRow	Success	Created	201	247	Bikes	IconAccept1	OnSelect	1,493
21	17:43:31.452	ScreenLoad	Navigate (Brow...	Success			155		EditForm1	OnSuccess	
22	17:43:31.471	Network	getRows	Success		200	123	Bikes	BrowseGallery1	Items	3,008
23	17:43:31.603	Network	getAttachments	Success		200	230	Bikes	Attachments_D...	Default	122
24	17:43:32.390	UserAction	Select	Success					IconNewItem1	OnSelect	
25	17:43:32.553	ScreenLoad	Navigate (Brow...	Success			150		IconNewItem1	OnSelect	
26	17:43:33.887	UserAction	Select	Success					DataCardValue2	OnSelect	
27	17:43:36.620	UserAction	SetProperty	Success					DataCardValue2	Text	
28	17:43:36.670	UserAction	Select	Success					IconAccept1	OnSelect	
29	17:43:36.949	Network	createRow	Success	Created	201	238	Bikes	IconAccept1	OnSelect	1,499
30	17:43:37.121	ScreenLoad	Navigate (Brow...	Success			151		EditForm1	OnSuccess	
31	17:43:37.130	Network	getRows	Success		200	127	Bikes	BrowseGallery1	Items	4,508

Console Write Log is soooooo 1996 🤷

Trace

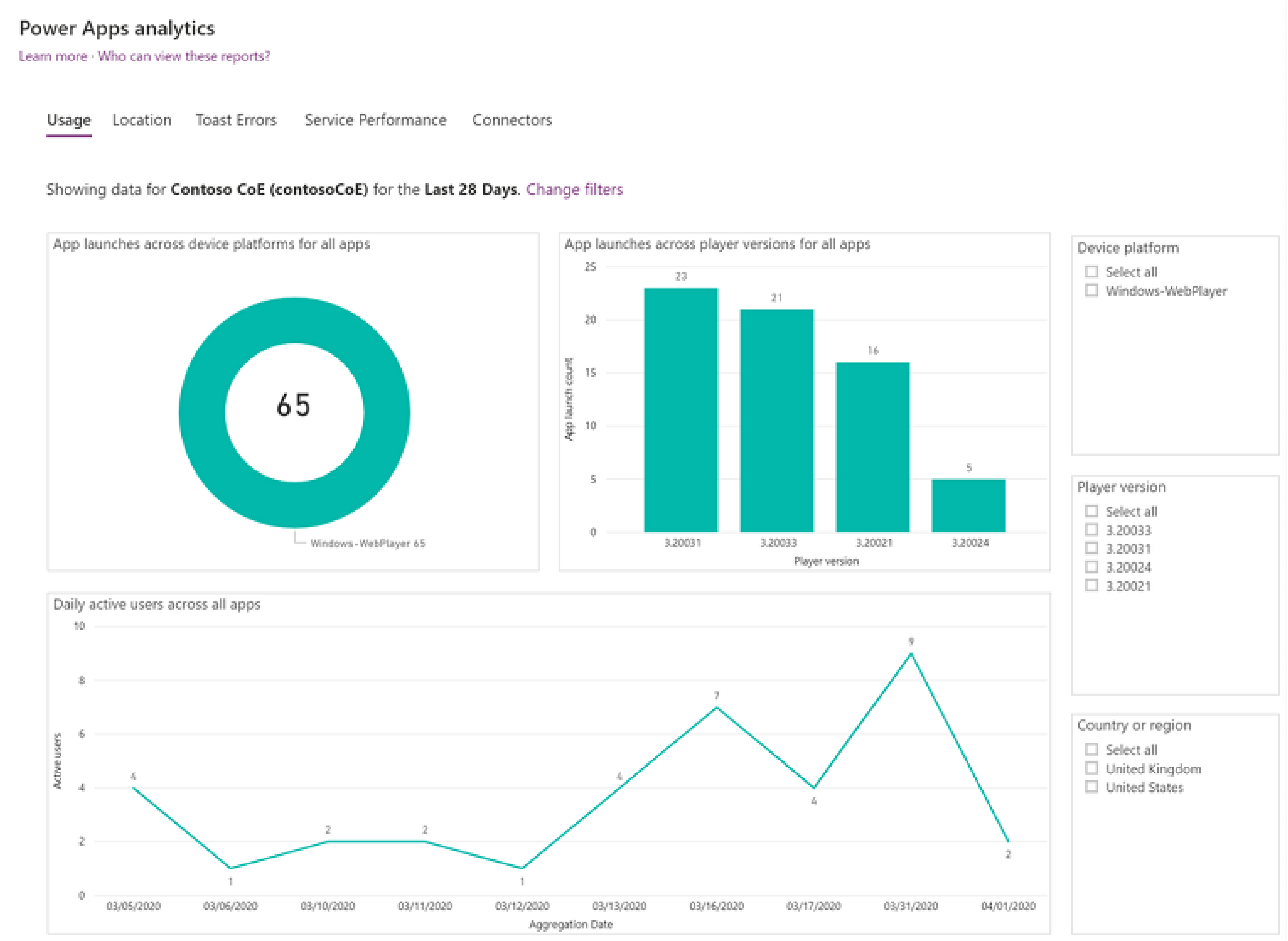
Add custom trace messages for Monitor



Gotta close that feedback loop ➰

Analytics

Power Apps analytics are available via the Power platform admin center: https://aka.ms/ppac

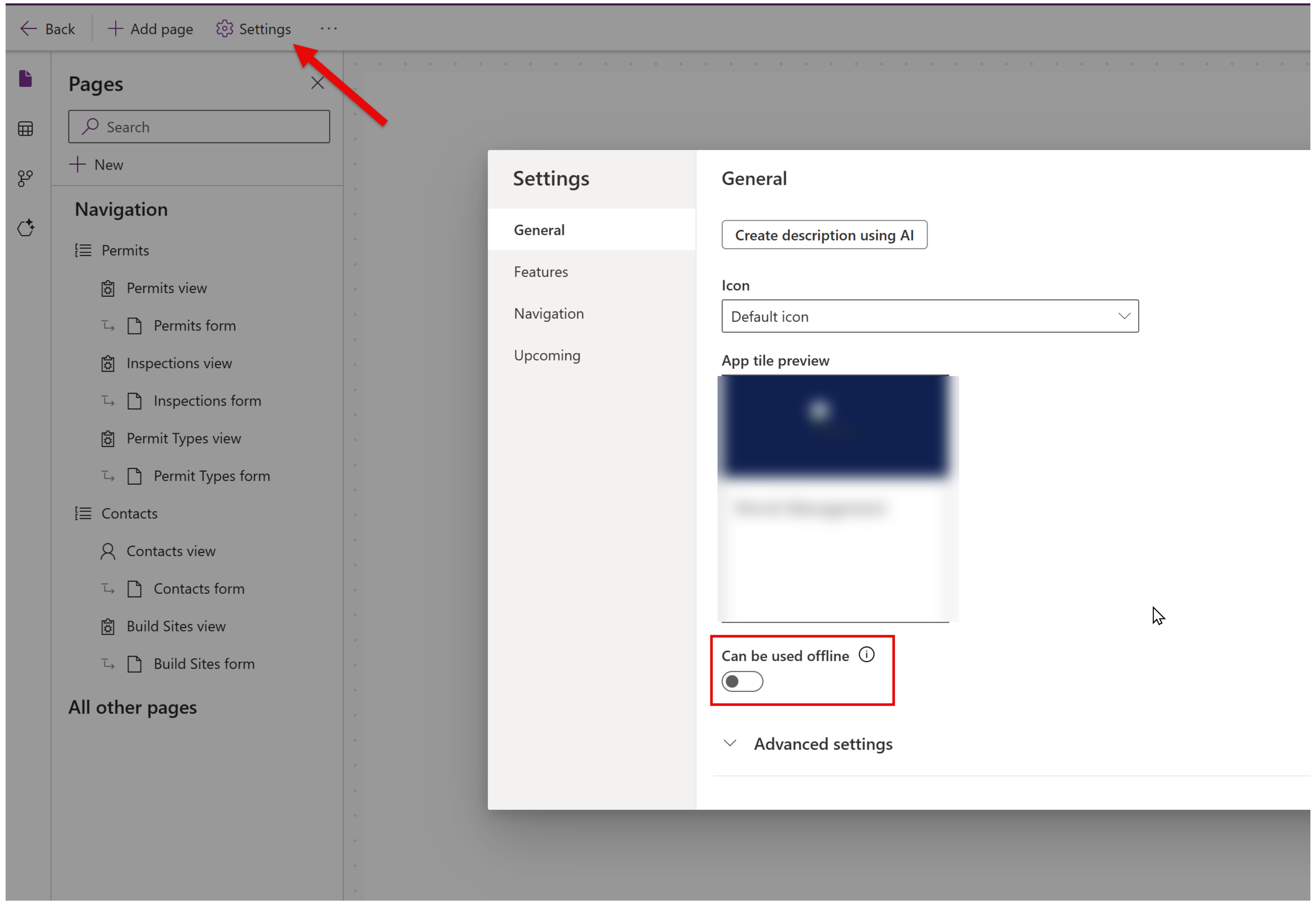


Demo: Test Studio, Monitor and Analytic Reports

Digital detox for your apps 😊

Offline app considerations

- Custom pages / canvas apps:
  - Use **LoadData** and **SaveData** functions:
    - Consider device-level storage limitations (30-70MB)
    - Merge conflicts - how to handle?
- Model-driven apps
  - Each app must be enabled for offline use
  - Configure offline profile to define the table / data to store locally
    - Each table must be enabled for offline first



Thought you could escape this, huh? 😁

Copilot feature overview

- For canvas apps and custom pages:
  - Create and explain Power Fx formulas
  - Copilot chat

Write formulas like a boss

Create and explain Power Fx formulas

```
//Check what day of the week it is - if Monday to Friday, use Color.White. Otherwise, use Color.Black
```

< 1/1 > Accept Tab Make sure AI-generated content is accurate and appropriate before using. See preview terms ··· Otherwise, use Color.Black

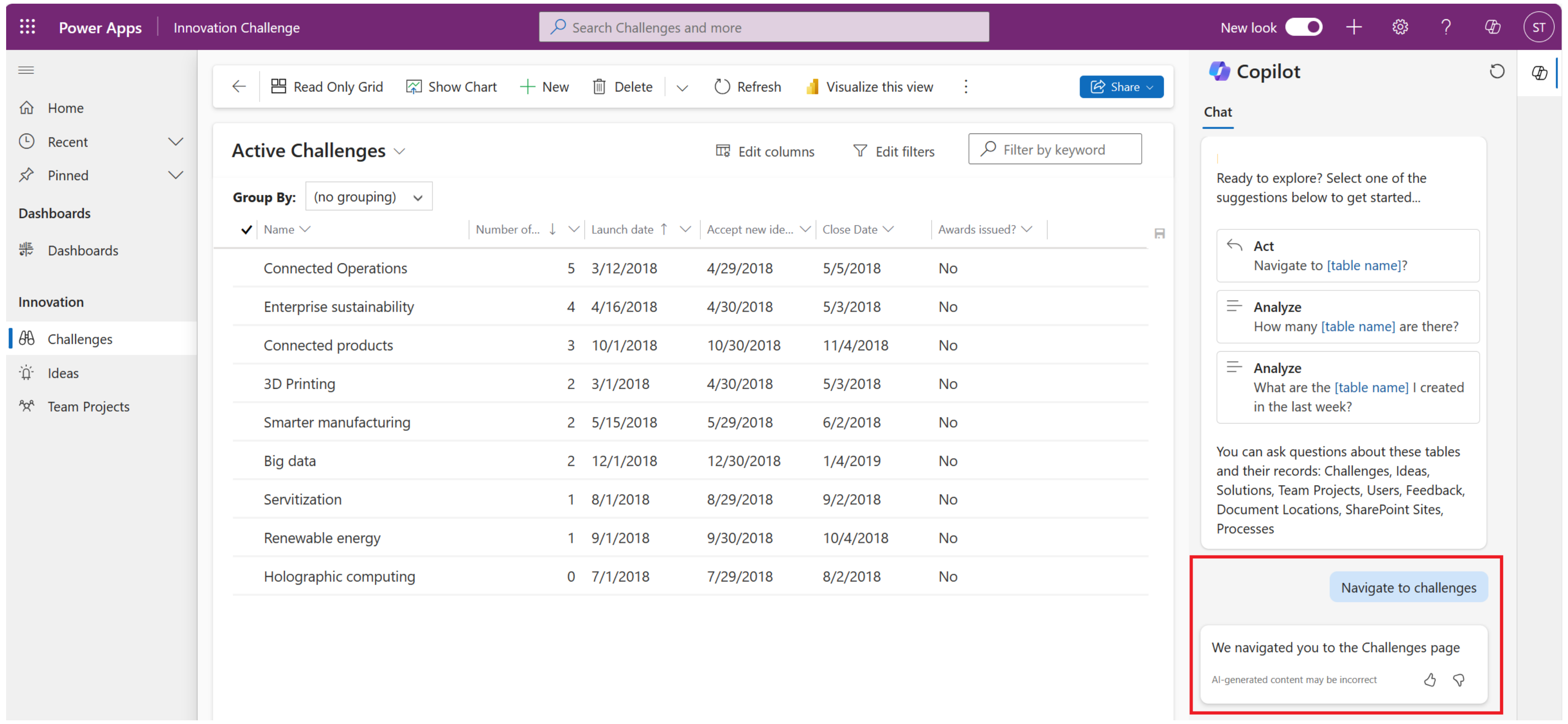
```
If(Weekday(Today()) in [ 2, 3, 4, 5, 6 ], Color.White, Color.Black)
```

- Use Copilot to:
  - Explain a complex formula for you
  - Suggest a formula based on comments.
- Included as part of your licensing; no additional purchase necessary.

Help me, chat!

Copilot chat

- Built-in assistant, directly within your app.
- Key benefits:
  - Ask and answer questions regarding Dataverse data…
  - …or get some hints on potential questions you can ask
  - Quickly open records using the record picker
- Generally available for Dynamics 365 apps, public preview for custom Power Apps



Got some links for you

Additional resources – Custom Page design

- Bootstrap:
  - https://icons.getbootstrap.com/
- CSS blur glass morphism:
  - https://css.glass/

Lab 4 – DEPLOY AND TEST THE SOLUTION

In this lab, you will do the following:

- Export the solution as a managed solution from the Development environment.
- Import the managed solution into the Production environment.
- Test the solution in the Production environment.

Expected completion time: 30 minutes

Closing thoughts

What’s next for your learning experience?

- Liking to use something goes a LONG way to helping with user adoption 🤝
- Convergence of canvas versus model-driven gradually closing; transformative potential for all your user interfaces 💡
- Build solutions that utilises components of Power Platform in the right way
- Low code doesn’t necessarily mean low complexity; strike a balance ⚖️
- Use generative AI to your advantage… 🤖
- …but a bit of knowledge certainly helps when vibe coding 😉
- Go play around with vibe.powerapps.com, Code Apps and generative pages!

How did we do today? 🥹

Your feedback is greatly appreciated!

- Please submit your feedback on today's workshop:
  - What was good 👍
  - What was bad 👎
  - How you rate our (obviously) hilarious slide titles 😂

SESSION ASSETS & SURVEY

https://linktr.ee/canadiansummit

View on mobile

Q&A


WELCOME TO
CANADIAN POWER
PLATFORM SUMMIT
CANADIAN
20
26
SUMMIT

THANK YOU!


CANADIAN POWER PLATFORM SUMMIT
CANADIAN SUMMIT
PLATINUM SPONSOR
mscrm-addons.com
GOLD SPONSORS
Alithya
MNP
CANADIAN POWER PLATFORM SUMMIT
CANADIAN
20
26
SUMMIT
CANADIAN POWER PLATFORM SUMMIT